

2025-02-26

A Framework for Semantic Enterprise Transformat

Reimer, Timothy James

Reimer, T. (2025). A framework for semantic enterprise transformations (Doctoral thesis, University of Calgary, Calgary, Canada). Retrieved from <https://prism.ucalgary.ca>.

<https://hdl.handle.net/1880/120816>

Downloaded from PRISM Repository, University of Calgary

UNIVERSITY OF CALGARY

A Framework for Semantic Enterprise Transformations

by

Timothy James Reimer

A THESIS

SUBMITTED TO THE FACULTY OF GRADUATE STUDIES

IN PARTIAL FULFILMENT OF THE REQUIREMENTS FOR THE

DEGREE OF DOCTOR OF PHILOSOPHY

GRADUATE PROGRAM IN ELECTRICAL AND COMPUTER ENGINEERING

CALGARY, ALBERTA

FEBRUARY, 2025

© Timothy James Reimer 2025

Abstract

Semantic transformation within message exchange between heterogeneous systems provides a challenge for any enterprise. The primary reason for this challenge is that the required transformations are based on arbitrary codes that do not adhere to any grammatical or linguistic structure or rules. These codes are assigned to describe an entity or attribute such as order type or equipment ID.

In any message exchange, the meaning of the data must be preserved. This is done today by applying translation lookups to simple cross-reference tables or files scattered across the computing landscape for each application use case. Such tables are maintained manually and can cause processing delays due to failed transactions that rely on semantic transformation values. The research presented in this dissertation introduces generic transformation patterns and applies these patterns to an underlying graph structure. By generalizing semantic transformations through patterns and graphs (Chapter 5), it is possible not only to determine a traversal path to find a corresponding code value, but to predict missing nodes through the use of graph embeddings (Chapter 7).

The presented architecture is based on microservices. This allows for horizontal and vertical scalability and provides an abstraction to the transformation query such that various forms of queries can be submitted and semantically transformed without changes to the underlying traversal algorithms. The thesis demonstrates two scenarios. The first is a natural language scenario (Chapter 6), and the second is an event stream scenario. In the natural language scenario, a natural language query is provided, and the system returns the expected code-based results. This use case demonstrates the benefits of integrating NLP capabilities for users to search and find transformation codes via a web front-end.

The second scenario demonstrates an IoT case where components of sensor data need to be enriched so that downstream data analytics applications can deliver the correct KPIs and dashboards.

The value of this research is demonstrated by a cost analysis for failed transactions (Chapter 8) and by the benefits the presented semantic transformation architecture would provide to corporations. This research therefore underlines the importance of incorporating semantic transformations into organizations' overall data architecture and strategy.

Acknowledgements

I would like to acknowledge and thank my committee members Dr. Diwakar Krishnamurthy, Dr. Mohammad Moshirpour, Dr. Abraham Fapojuwo, and Dr. Mohammad Keyhani for taking the time to review my research proposal and provide constructive feedback.

I would also like to thank my supervisor Dr. Behrouz Far for his help in organizing the committees and for his insightful feedback during this project.

Dedication

I would like to dedicate this work to my wife Ursula, who has been my greatest supporter throughout this project. In the face of so many roadblocks and challenges, you were always there with me to help scale the wall. No for you was not an answer, and you cheered, encouraged, and challenged me to push on. Without you, this project would never have been started nor ever completed. Thank you, my dear.

List of Tables

Table 1. RDF Statements.	14
Table 2. Semantic Transformation Implementations.....	23
Table 3. Machine Learning Approaches Cross Validation.....	67
Table 4. Machine Learning Approaches Similarity Ratio.	67
Table 5. ML and Graph Algorithm Comparison.	71
Table 6. Solution Comparison.	79

List of Figures and Graphs

Figure 1. RDF Graph Structure.....	15
Figure 2. Message Flow Pipeline Scenario.....	30
Figure 3. PIDX Nomination Transaction.....	31
Figure 4. Canonical Transactions.	31
Figure 5. Material Code Cross-Reference.	32
Figure 6. Material Conditions.	33
Figure 7. Interoperability Architecture.	34
Figure 8. Event Streaming of Sensor Data.....	35
Figure 9. Base Graph for Use Cases.	44
Figure 10. Orchestrator Architecture.	47
Figure 11. Choreographed Architecture with NLP.....	52
Figure 12. Language Parser.	54
Figure 13. Request Message.	55
Figure 14. Response Message.....	56
Figure 15. Event Hub.....	60
Figure 16. Event Hub with Streams.	62
Figure 17. Sample Sensor Output from the Event Hub.	63
Figure 18. Event Log.	66
Figure 19. Missing Node Resolution.	69
Figure 20. Missing Node Retraction.....	70
Figure 21. Error Messages with Associated Barrels of Oil Not Shipped.	74
Figure 22. LLM Architecture.....	83

List of Abbreviations

Abbreviation	Explanation
AIAG	Automotive Industry Action Group
API	Application Programming Interface
B2B	Business-to-Business
BOD	Business Object Documents
CI/CD	Continuous Integration / Continuous Deployment
COIN	Context Interchange
CSV	Comma-Separated Values
EAI	Enterprise Application Integration
EDI	Electronic Data Interchange
EIO	Enterprise Interoperability Ontology
ERP	Enterprise Resource Planning
ESB	Enterprise Service Bus
ETL	Extract, Transform, and Load
HAD	Heterogenous, Autonomous, and Distributed
HTTP	Hypertext Transfer Protocol
IoT	Internet of Things
iPaaS	Integration Platform as a Service
JSON	JavaScript Object Notation
KMPI	Knowledge Management Performance Index
KPI	Key Performance Indicator
LLM	Large Language Models
MMF	Minimal Marketable Features
MVP	Minimum Viable Product
NILE	Networked Infrastructure for Lightweight E-commerce
NLP	Natural Language Processing
NoSQL	Not Only SQL
OAG	Open Application Group
ODESB	Ontology-Driven Enterprise Service Bus
ODSOI	Ontology-Driven Service-Oriented Integration
OWL	Web Ontology Language
P2V	Path to Value
RDBMS	Relational Database Management System
RDF	Resource Description Framework
RDFS	RDF Schema
REST	Representational State Transfer
SAWSDL	Semantic Annotations for WSDL and XML Schema
SDRM	Systems Development Research Methodology
SEAP	Semantic Annotation and Profiling
SMTP	Simple Mail Transfer Protocol
SOAP	Simple Object Access Protocol
SQL	Structured Query Language
STAR	Standards for Technology in Automotive Retail

TXT	Text File Format
UDDI	Universal Description, Discovery, and Integration
URI	Uniform Resource Identifier
URL	Uniform Resource Locator
W3C	World Wide Web Consortium
WSDL	Web Service Description Language
WSML	Web Service Modeling Language
WSMO	Web Service Modeling Ontology
WSMX	Web Service Execution Environment
XML	Extensible Markup Language
XSD	XML Schema Definition
XSLT	Extensible Stylesheet Language Transformations

Table of Contents

Abstract	ii
Acknowledgements	iv
Dedication	v
List of Tables	vi
List of Figures and Graphs.....	vii
List of Abbreviations	viii
1 Introduction and Background	1
2 Literature Review.....	8
3 Research Methodology	27
4 Use Case Description.....	29
4.1 Message-Based Use Case	29
4.2 Event-Based Use Case	35
5 Discovery of Semantic Transformation Patterns	37
5.1 One-To-One Pattern.....	37
5.2 Enrichment Pattern.....	38
5.3 Dependency Pattern	39
5.4 Granularity Pattern.....	41
6 Semantic Transformation Framework	43
6.1 Semantic Transformation Utilizing Graphs	43
6.2 Microservices Architecture	45
6.3 Microservices Workflow Management.....	49
6.3.1 Orchestrator.....	49
6.3.2 Choreographer with NLP	50
6.3.3 Evaluation of Workflow Approaches	56
6.4 Semantic Transformation and Event Streaming	57
6.4.1 Event Streaming Concepts.....	57
6.4.2 Semantic Transformation with Pub/Sub	59
6.4.3 Semantic Transformation with Stream Processing.....	61
6.4.4 Event Stream Architecture Comparison	63
7 Conflict Resolution with Machine Learning Scenarios	66
7.1 Classical Machine Learning Approach	66
7.2 Node Predictions using Graph Methods	68

7.3	Evaluation and Comparison.....	71
8	Realization and Value Contribution.....	73
9	Conclusion	76
9.1	Solution Comparison	76
9.2	Future Research	82
	References.....	85
	Appendix A.....	91
	Appendix B.....	99
	Appendix C.....	107
	Appendix D.....	113

1 Introduction and Background

Any organization that employs any form of information system faces interoperability challenges [1]. This is due to ever-changing technologies for hardware and software. Changing business strategies trigger changes in business processes which in turn drive new requirements for new application deployment, upgrades, or application requirements. To facilitate and support the data exchange among the corporate application suite data, interoperability must be maintained. The definition for interoperability states that interoperability is the “capability to communicate, execute programs, or transfer data among various functional units in a manner that requires the user to have little or no knowledge of the unique characteristics of those units” [2]. This definition helps to set a boundary of a vast interoperability space and focuses on the interchange and sharing of data through various functions and programs.

Due to the diverse technologies (hardware/software) of which applications are composed, the data interchange has always been a challenge since there are no consistent interchange standards for communicating applications to share such data. There is no shortage of proposed approaches on solving or at least easing this problem. The reason that no comprehensive solution exists is that technologies rapidly change, and each proposed approach is always tied to the underlying supporting technological environment. So, one approach might work in a particular technology epoch but will be obsolete once the next epoch arrives. By researching new interoperability approaches and testing their viability, new architecture patterns can be proposed that will improve data reliability and consistency and reduce the overall complexity of application interoperability (keeping in mind that these architecture proposals will eventually be outdated again and replaced by the next generation of technologies).

One of the first discussions of sharing and assimilating information was presented in 1945 by Vannevar Bush in his paper “As We May Think” [3]. Bush's outlook on the potential of machines to support human computing capabilities was visionary; one of these ideas was the concept of a Memex machine that would be able to assimilate information to help humans evaluate, retrieve, and consume that information. What is interesting here is that he stressed that information retrieval should be carried out based on associations rather than strictly indexing. Many such associations are implemented today using taxonomies or ontologies with reasoning.

Even though Bush’s proposed Memex machine can be considered an intelligence repository and does not directly address data interoperability, it can be argued that any time a data repository is established it requires functionality to ingest data from various sources in various forms so that this data can be stored and retrieved. Therefore, the vision of the Memex machine can be considered the first that would require the capability of data interoperability.

With the advent of database technologies, the problem of data interoperability again became relevant. The sharing of data between databases was an issue; and also between different database technologies such as network, relational, and hierarchical databases. Wiederhold in his 1992 paper “Mediators in the Architecture of Future Information Systems” says that “Intelligent and active use of information requires a class of software modules that mediate between the workstation applications and the databases” [4]. He recognized that mediation technology was required to address data interoperability. His three-layered architecture pattern consisted of a user facing layer, a mediation layer, and a database layer. Today this architecture style is considered a layered architecture with a presentation layer (representing the user web interaction), the business layer (containing the application logic), and the persistent layer (providing an abstraction to databases).

The need for mediation functionality between a data source and receiving target became what is known today as middleware.

In 1997 Goh [5] developed a mediated data access strategy that he named COIN (Context Interchange). In this strategy, semantic conflicts between heterogeneous systems are detected and resolved by a context mediator through a comparison of contexts associated with any two systems engaged in a data exchange. His approach is primarily focused on merging data elements across heterogeneous databases. Goh's thesis focuses on connecting databases via semantic mediation rather than using an enterprise integration platform. However, Goh's proposal clearly uses semantic transformation in the form of ontologies to resolve data mediation issues. He distinguishes between scaling conflicts, naming conflicts, and confounding conflicts. Scaling conflicts refer to the adoption of different units of measure of scales in reporting. For example, financial data are routinely reported using different currencies and scale-factors. Naming conflicts consist of synonyms and homonyms among attribute values. For example, the name of a company may be reported differently in different systems. Confounding conflicts refers to conflicts arising from the confounding of concepts which are in fact distinct. For example, the "latest trade price" reported by two data feeds may differ from one another simply because one reports with a greater temporal delay than the other. Goh was one of the first to research semantic mediation in the context of databases.

As the end of the decade approached, many companies were forced to make major changes to their IT applications due to the Y2K issue. The rush on implementing ERP systems in time was driving the need for better integration capabilities. In 2000 Linthicum published his book *Enterprise Application Integration* [6]. It is worth mentioning because his publication first introduced the term EAI (enterprise application integration) to address integration architectures

among heterogeneous applications. He takes a process view of sharing information and defines EAI as the unrestricted sharing of information and business processes among all connected information systems in the enterprise. EAI therefore becomes an integral part of the enterprise not only to connect technologies but also at a higher-level business process. The business user is abstracted from the underlying applications and can perform business processes that reach across diverse applications.

From an EAI perspective, the paper published by Trastour et al. [7] focuses on expanding integration to business partners by integrating various standards (ROSETTANET, PIDX, SWIFT). They differentiate between syntactic and semantic transformation. Most integration standards are syntactically focused. On the semantic integration side, they proposed the ontology standard DAML+OIL (DARPA Agent Markup Language + Ontology Inference Layer), which is a semantic markup language for describing ontologies, particularly in web-based environments like the semantic web. It combines features of DAML (developed by DARPA) and OIL (Ontology Inference Layer) and serves as an important precursor to modern web ontology languages like OWL (Web Ontology Language). Their proposed solution is using reasoners to resolve conflicts in the transformation process. The solution presented uses the ontology concept as the base to transform external data (in this case XML) to an ontology of the partner and then refactor this format back into the receiving XML document. Since it is based on a standard exchange format, it presents a reasonable approach as a solution for business-to-business interchange.

Following the architecture pattern of service-oriented architectures through the implementation of web services with XML, Izza et al. [8] proposed a mediation framework that was based on an Ontology-Driven Enterprise Service Bus (ODESB). With their proposal they wanted to address the integration of heterogeneous, autonomous, and distributed systems (HAD).

They structured the services into data, functional, and business services. Data services expose various data sources; functional services expose application systems; and business services are a combination of data and functional services to support business processes. The semantic mediation process is achieved with services that use local, domain, and global ontologies with reasoning to drive inferences. Izza et al. recognized that a single set of ontology standards is almost impossible to create. Therefore, their approach defined ontologies for the different service levels such as data, function, and business. The mediation process would then use the service description to derive a semantic transformation by mapping the service against the respective technology. Their approach achieved interoperability if the message structure was maintained in XML and supported web services with the SOAP protocol.

In 2008 Bouras et al. [9] proposed a new approach for semantic integration. They proposed to enrich the standard web service with semantic annotation. This semantic annotation would then be used to upcast the XML message to a domain ontology level. This ontology could then be transformed into a global or enterprise ontology. On the receiver side, the reverse process would take place whereby the message would be downcasted to match the expected receiver structure.

The transformation between the different ontology levels would be performed using XSLT. The semantic enrichment would be performed by using the proposed SEAP (Semantic Annotation and Profiling) tool. This Windows-based tool allows the enrichment of the message structure for each web service with ontology-related terms to aid the dynamic transformation with the domain/global ontologies.

In 2017 Vuković et al. [10] presented another approach which is characterized by first finding mapping candidates. These mapping candidates are elements in the data structure that can be mapped to the receiver by annotating elements with certain ontology definitions. Therefore,

Vuković et al. propose several mapping criteria. For example, the Equal Name criterium describes elements that are semantically equal in source and target. Another criterium is Generalization, where the sender describes a general term “address”; however, the receiver is more specific in classifying it to be “home address.” Once the mapping is enriched through these criteria, a conflict resolution process is performed that uses reasoning to resolve differences. This is followed by an expression builder which is the input to build the actual mapping code.

Since Vuković et al.’s paper was published, enterprise integration has again undergone major changes. Cloud computing has become a standard utilization in any enterprise, and with that, the concept of serverless computing. The new term iPaaS (Integration Platform as a Service) has emerged [11]. It describes the concept of moving all enterprise integration to the cloud. Given that the scope of enterprise integration has expanded, the complexity of integration scenarios has also increased. Besides business-to-business integration and standard message exchanges within an organization, iPaaS platforms must now handle interoperability across diverse cloud platforms and must be able to integrate various integration platforms which still reside on premise. Parallel to integration platforms, data analytics has emerged as a vital capability for organizations. Platforms built on Apache Spark [12] process very large data sets for analytics purposes. Transformations are used to ingest data from various applications in order to harmonize, categorize, and aggregate data for these machine learning tasks. The use cases for these types of ingestions require syntactic and semantic transformation.

In addition, IoT has become a significant factor with capturing millions of events and passing them on for further processing. In many scenarios, companies deploy event brokers such as Apache Kafka [13] to process these large data sets. Since the original captured event is often only a minimal

data structure, data enrichment is required to enhance the meaning and evaluation capability of such events.

All these technological changes justify taking a fresh look at enterprise integration—namely, how to best expand its functionality through applying modern architectures and new technological trends, while still incorporating the learnings from previously described initiatives. The result of this research is therefore a proposed new approach to define a modern architecture that will address the semantic transformation aspect within the new ecosystem of enterprise integration. Conflict resolution within semantic transformation is an integral part of interoperability. Therefore, the research will address methods and processes that will handle conflicts as part of the semantic transformation process. The research questions can therefore be formulated as follows:

- What modern architecture could be applied to handle semantic transformations and is generic enough to handle a variety of use cases and technologies?
- What methods or processes could such an architecture utilize to handle semantic conflict resolution automatically or at least semi-automatically?

2 Literature Review

Interoperability between applications has a long history. It started with shared libraries and subroutines in code, followed by remote procedure calls. With the maturing of the Internet, interoperability between business partners to exchange trading documents became essential. The service-oriented architecture pattern introduced a new type of service design. Based on its standardized definition, and utilizing existing Internet protocols, it enabled communication via the Internet. Costly value-added networks as were used for EDI (Electronic Data Interchange) could now be avoided. Web services provided a new technique for integrating applications within the enterprise and for communicating and connecting business partners to establish integrated business processes. Each web service provided unique functionality through a set of custom functions and therefore allowed integration of both syntactic and semantic transformation.

To support its interoperability, a web service is characterized by three distinct features [14]:

- The service can describe itself, its functionality, and communication contract.
- It is discoverable by other applications, so it can be consumed by other services and applications.
- It can be invoked by any other application that wishes to use its functionality.

These three characteristics are implemented through the standardized Simple Object Access Protocol (SOAP), Web Service Description Language (WSDL), and the Universal Description, Discovery, and Integration (UDDI) specification [15]. The exchange structure is W3C's XML schema specification (www.w3.org/XML/Schema) which provides a standard language for defining the document structure and the XML structure data types.

The SOAP protocol can be implemented on top of other protocols like HTTP or SMTP, and therefore provides great usage versatility. Its message structure consists of an envelope that wraps the full SOAP message. Within each envelope, multiple blocks can be defined, of which each will contain a header and a body structure. The SOAP message also contains a description of data types within the message and a description or convention on method invocation. Finally, it provides a description on binding to a communication protocol.

WSDL is the specification that describes the web service. The WSDL document is structured in two parts. The first part describes the abstract portion of the service. It includes the structure of the message with its data type definitions, functions, and ports required for an invocation and message exchange. The second part contains the concrete description with a definition of any bindings, the actual service invocation, and necessary port implementations. From an implementation perspective, the client imports the WSDL document of the service and, with that, has all the parameters necessary to invoke and communicate with the service.

Web services can be registered in a central register for discovery. To communicate with such registration, the UDDI specification provides the standard. It defines the information about each service in the registry and the way to query the registry so that the published services can be consumed [16].

Another type of web service called RESTful service (Representational State Transfer) [17] is a lightweight and cost-effective alternative to SOAP-based services. RESTful services use the basic built-in HTTP interaction methods (GET, POST, PUT, and DELETE) to access URI-referenceable resources. A resource could be any data item, document, image, or tweet, for example. RESTful services can be described with four characteristics [18]:

- Resources represent an abstraction for server-side application states and entities.
- Each resource is addressable via its uniform resource identifier (URI).
- All resources share a uniform interface via the HTTP methods.
- All interactions with resources are stateless.

Because of their ease of implementation, small footprint, and cost effectiveness, RESTful services have become popular in the development of interoperability solutions either for the B2B space or mobile applications. Since RESTful services are stateless, server-side state management is not required. The lightweight and small footprint of RESTful services is ideal for the communication of microservices, as will be demonstrated.

Services can be composed to service mashups. In general, there are three different types of such service mashups [18]:

- Data-oriented mashups, responsible for converting, transforming, and enriching data items.
- Process-oriented mashups, to enhance specific business functionality to business processes.
- Consumer-oriented mashups, which are exposed to the public to customize views and dashboard.

In the context of this research, we will focus on data- and process-oriented mashups.

Architecturally, web services provide a way to loosely couple functionality and apply a distributed computing environment which can span internal applications but also business partners. To expand

interoperability of web services, the sharing of meaning among processing endpoints is required. This requires the creation and definition of a vocabulary that can be shared amongst participating parties. Such a vocabulary is a conceptual representation of a domain, also referred to as an ontology [19]. To incorporate ontologies into web services to enhance their semantic capability, the WSMO [20] definition was created. WSMO (Web Service Modeling Ontology) is a formal ontology for describing various related aspects of semantic web services. It defines a coherent definition to allow semi-automated discovery, composition, and execution of web services based on logical inferences [21]. WSMO defines four main modeling elements for describing semantic web services.

The first element is ontologies, which provide domain-specific terminologies for describing the other elements. Their purpose is twofold [21]. First, describing the semantics of the information; and second, linking human and machine technologies. These descriptions include concepts such as classes, relations, functions, and axioms; as well as non-functional properties, imported ontologies, and used mediators. Mediators allow for solving terminology differences. WSMO uses the WSML (Web Service Modeling Language) [22] as the underlying language based on different logical formalisms.

The second element is web services, which connect computing devices using standard web-based protocols to exchange and assimilate data based on functional requirements. Web services are distributed across the web and constitute a distributed environment. Each web service contains specific definitions of functionality which can be reused and incorporated into larger, more complex web service networks. WSMO defines a web service by its non-functional properties, its imported ontologies (which provide a link to ontologies in other domains), its mediators, its capability, and its interfaces. The capability encapsulates its functionality, and the interface

definitions provide the contractual agreement for communication and collaboration. A web service can have multiple interfaces but can have only one capability.

The third element is the definition of goals related to the web service consumer's desire for the requested functionality as opposed to the provided functionality described in the web service. As with the web service definition in WSMO, goals also reference non-functional properties, imported ontologies, and used mediators; as well as requested capabilities and requested interfaces.

The final element is mediators with their objective to overcome structural, semantic, or conceptual mismatches that might appear between different components of a WSMO description. Currently, the specifications address four different types of mediators:

- OOMediators – import target and source ontologies by resolving all the representation mismatches between source and target.
- GGMediators – connect goals that are in a relation of refinement and resolve mismatches between them.
- WGMediators – link web services to goals and resolve mismatches.
- WWMediators – connect several web services for collaboration.

Once a web service is described with WSMML, it needs an execution environment so that consumers can utilize its functionality. Haller et al. [23] introduced WSMX (Web Service Execution Environment) as a platform that enables the creation and execution of semantic web services based on WSMO. Their proposed WSMX architecture consists of the following components.

The adapter provides connectivity to heterogeneous source and target systems that natively do not support WSMML. The adapter therefore transforms the received and sent messages to/from

WSML into the native format of the connected application. The compiler component is responsible for verifying the WSML document. During web service registration, the compiler reads the service description, validates it, and stores it. Through execution the compiler reads the goal description and passes it to the management component for resolution. The matchmaker is responsible for finding an appropriate web service that matches the described goal. The data mediator uses the incorporated ontologies from source and receiver and provides the appropriate transformation. The choreography and composition engine orchestrates the execution of the web services based on a predefined execution to achieve the stated goals described as part of WSMO. Finally, the communication manager manages the invocation of these web services to retrieve the results based either on synchronous or asynchronous calls. The latter requires more complex message management since web service state needs to be considered.

The presented semantic web service approach and definition assumes that interoperability is primarily achieved using web services, which is of course unrealistic. Many different message formats and connectivity requirements exist beyond just web services. Even though components such as a full implementation of mediators is still missing [23], the WSMX architecture utilizes the concept of incorporating ontologies into the message transformation to add knowledge to the overall integration process; and with that, attempts to resolve semantic transformation within integration.

Semantic web services are built upon web services technology and their associated standards which do add restrictions to implementing semantic transformation in enterprise integration scenarios. Another approach in solving interoperability with semantic transformation is to examine the semantic web and its standards and definition.

The main idea of the semantic web is to support a distributed web at the level of the data rather than at the level of presentation [24]. Instead of having only pages linked together, a secondary linkage is implemented by connecting data items using a global reference called a Uniform Resource Identifier (URI). The data model of such linkage is being defined through a Resource Description Framework (RDF) [25]. With URIs, the RDF can name relationships between things as well as between two endpoints of such a link. This connection is referred to as a “triple.” Using this simple triple relationship model allows for structure and semi-structure data to be shared across different applications. The linked structures can be expanded upon to form a directed labelled graph where the edges represent the named link. The graph representation is easy to understand and makes RDF representable.

RDF is therefore the meta data description language of the semantic web [26]. It is made up of resources, properties, and statements [27]. A resource in RDF can be compared to an entity in a data model. Each resource is identified via the URI. Combined with the URL of the web location as namespace, the URI fulfills a resource identification requirement that must be globally unique. Properties are resources with a name that can be used to describe some specific aspect, characteristic, attribute, or relation of a given resource. The statement defines the relationship and comprises the basic RDF building blocks, which are:

<SUBJECT> ---< PREDICATE> --- <OBJECT>

Subjects can be linked via predicates to other subjects, which then become objects. These linkages then build a graph structure with unique nodes. To demonstrate, Table 1 shows several statements in RDF format.

Table 1. RDF Statements.

Subject	Predicate	Object
---------	-----------	--------

Plant	Resides in	Toronto
Material	Is in a	Plant
Material	Is an	Asset
Plants	Have	Assets

This table can now be transformed into the following graph structure (Fig. 1):

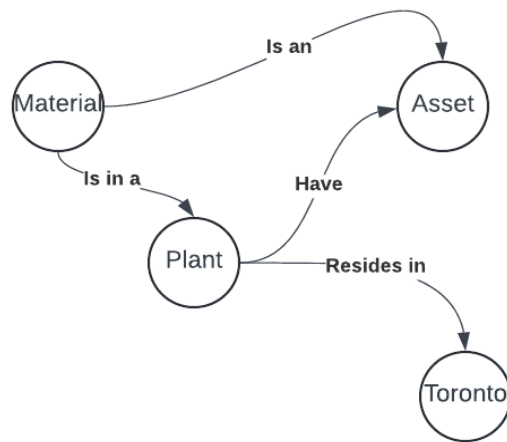


Figure 1. RDF Graph Structure.

By using RDF as the semantic web descriptor and by using its statements, data can be connected and presented as a graph.

The next step is to associate a structured vocabulary to the RDF structure. This structured vocabulary is the definition of an ontology within the semantic web [27]. The linkage between RDF and the domain ontology is the RDF schema (RDFS) definition. RDFS is a lightweight schema language and is part of the semantic web technology stack. It allows for the description of classes, subclasses, and properties of RDF resources. The RDFS language works in conjunction with RDF statements and therefore can add semantics to RDF predicates and resources. It defines

the meaning of a given term by specifying its properties and what kind of objects can be the values of these properties. As with RDF statements, RDFS statements can also be expressed in triple form which can be transformed into a graph structure.

The Web Ontology Language (OWL) [28] is based on RDFS. It provides further expression for the definition of an ontology. OWL and RDFS have the same purpose in defining classes, properties, and their relationships. The additional expressiveness of OWL over RDFS is in presenting richer and more complex relationships between classes.

In summary, two semantic interoperability solutions have been discussed. One focuses on the use of web service technologies and their related standard definitions. The second is a more generic approach which utilizes the definitions of the semantic web in general and is therefore not linked to any specific technology implementation. The semantic web definitions with RDF and RDFS provide a way to present data in a graphical format compared to standard table structures.

Based on the two design approaches presented for semantic transformation, the following will provide a discussion on several proposed implementations within the literature. Even though they each use different implementation architectures and technologies, the overall intention of providing a solution for semantic transformation is the same. The discussion therefore illustrates the complexity of the problem and the various attempts to find the right solution.

As already mentioned, Trastour et al. [7] are specifically focused on processing ROSETTANET-style documents as part of an integration solution. The input are XML-type documents. However, the schema definition of XML, which is XSD, handles syntactic constraints but not semantic ones. By converting the XML to an RDF format, the constraints identified in the XML document can be stored in a knowledge base. For each new business partner that is to be onboarded into the transformation ecosystem, a new constraint definition must be developed. The

NILE tool (Networked Infrastructure for Lightweight E-commerce) that is being proposed by Trastour et al. will perform the transformation from XML to RDF. Based on the agreed constraint definition between partners, the incoming XML ROSETTANET document can be transformed syntactically as well as semantically.

Trastour et al.'s paper does not specifically discuss the underlying processing technology but only provides a conceptual description of the process. There is no mention of how the received XML documents are ingested into the NILE tool, or how the output is generated. The NILE tool also does not seem to allow for other message structures; it only supports ROSETTANET. One can assume that since RDF and the development of an ontology is proposed, then the transformation between XML and RDF can be configured such that other types of XML documents can be processed. However, since the NILE tool can only transform XML to RDF, it cannot process other types of formats such as CSV, TXT, or databases. It is also unclear whether this transformation process is of a synchronous or asynchronous nature. In addition, it is not mentioned how the constraints for a specific transformation/partner are stored so that at execution time the right transformation map can be retrieved and executed. Even though the solution incorporates semantic web concepts, it is not generic enough to provide an overall solution for interoperability in the enterprise application integration space.

Izza et al. [29] extensively researched the use of ontologies as part of the enterprise application integration space. Their proposed approach is to use a layered ontology concept which they name urbanization of ontologies. The idea is the introduction of abstraction between different ontology layers. This allows for more flexibility in maintaining various ontologies on the different layers. Their three primary layers are local ontologies, which focus on specific functions; domain ontologies, which cover more of an application scope of subject area; and a global ontology that

encompasses the whole enterprise. The implementation of these ontologies can be either a single (global) ontology or can comprise multiple local/domain ontologies connected to establish a network of semantic definitions.

Once the ontology structures are defined, a mediation process is applied [30] that will relate or map semantic entities from the source to the target. A prerequisite is a structure transformation of an XML/XSD-based document to OWL-S+ which is an additional extension to OWL. The mapping process will utilize the various levels of ontology granularity from local to global ontologies as they exist. Once the mediation process is completed, the result will be transformed into an XML document for the receiving system.

The described approach is captured in the proposed ODSOI framework (Ontology-Driven Service-Oriented Integration) [8]. The framework comprises a central ontology service bus that integrates various data, functional, and process services including a service registry. In addition, it invokes the respective semantic services for semantic mediation. Data, functional, and process services handle the syntactic and semantic transformation required.

Izza et al.'s [8] framework provides a tight integration between web service architecture and semantic transformation through the Ontology-Driven Enterprise Service Bus (ODESB). The conclusion is that this bus will become the central integration hub for an enterprise. Architecturally, there seems to be no possible integration to existing integration platforms since the ODESB will manage both syntactic and semantic transformations.

The development of ontology definitions is a time-consuming task; and even though the layered ontology architecture approach is recommended, it provides additional complexity in connecting ontologies in such a way that the mediation approach can be successful. Since the proposed solution is web service based, it can only process XML-type documents. It is also

important to note that the different functional layers such as data, functional, and process combined with semantic service inadvertently add latency to the overall process of ingesting an XML document from source to target.

Roberto et al. [31] provide another interesting example. Their proposal discusses an e-government portal implementation where user queries are deconstructed into web service calls. These web services are semantic web services and use a WSMO ontology definition. The ontologies are structured into different domain ontologies such as life-event, e-government, and services. The primary objective of the portal is to help users identify which government service is the right one to use. Life-event portals [32] assume that users don't necessarily know which service applies to which life situation; therefore, a knowledge base portal helps the user navigate to the right service offering. The portal allows for registration of new web services through a desktop front-end as well as to respond to stated goals within the message. These goals are part of the WSMO definition, to provide back recommendations.

The presented solution does not have the traditional characteristics of an enterprise application integration pattern, such as message ingestion, transformation, or forwarding. However, it provides an example of how interoperability can be applied in use cases where direct user interaction is required. Architecturally, the concept of a middleware that routes and responds to user requests demonstrates the variety of use of semantic web services; and illustrates that the problem of interoperability can take on different forms with different solutions.

As already mentioned, Bouras et al. [9] proposed to address the interoperability issues by annotating the standard web service definition with ontology components. The semantic components would be drawn from an enterprise interoperability ontology (EIO). The annotation would be expressed in W3C standard SAWSDL (Semantic Annotations for WSDL and XML

Schema). The XML message definition embedded in the WSDL description will be semantically annotated. These annotations help define the meaning of data by using references to the enterprise interoperability ontology. The SEAP tool is used via a graphical user interface to annotate the standard web service with semantic extensions. The output is an XSLT transformation that enhances the WSDL to SAWSDL and back to WSDL; this is called up-lift/down-lift. These transformations are stored in a repository for processing through the invoked integration process.

The approach differs from Izza et al.'s [8] since it applies a web service annotation approach rather than using WSMO which is a different web service construct. By using this approach, there is no change to the original web service design, but rather an up-lift/down-lift process to integrate ontology concepts into the process. This is where the SEAP mapping tool is used to create a link between the standard web service and the semantic annotation. One could argue that since the original web service structure is maintained, an implementation of the approach is easier to realize since only up-lift/down-lift maps need to be created with the SEAP tool.

Vuković et al.'s paper [33] introduces a domain-specific language for semantic-aided automation of interface mapping in enterprise integration. The idea is to annotate the data input structure with a domain-specific language to address semantic differences. This is achieved by identifying mapping criteria that link to the structure candidates. Such criteria can include equal names, aggregate elements, and generalization elements. Once the common mapping elements are identified, a reconciliation of differences is analyzed and resolved. The result is a transformation map in programmable code that can be used to perform the full integration. The prototype use case uses a CSV input structure with a web service back-end to an ERP system.

Compared to other discussed solutions which all require XML structures for input, this proposal uses a standard CSV file. The CSV structure is imported into the mapping tool which is an extension of Eclipse. Developed adapters provide the linkage via web services to the back-end ERP system and extract the target data structures. The overall integration process is then scheduled as a job to be executed. Due to the ingestion of a CSV and back-end connection via web service, this integration solution reflects a more realistic scenario than using pure web services. One restriction of Vuković et al.'s approach is the use of the domain-specific language, which seems not to be a generally accepted standard.

Anicic et al.'s [34] proposal demonstrates a two-step semantic web approach. The first step is considered the ontology design time, and the second step is the actual transformation of messages. In the scenario described by the authors, two organizations exchange messages. The messages of the first organization are based on the Standards for Technology in Automotive Retail (STAR). The second organization's message documents are based on the standard of the Automotive Industry Action Group (AIAG). In return, both standards are based on Business Object Documents (BOD) of the Open Applications Group (OAG). At ontology creation time, the BOD, STAR, and AIAG documents are transformed into OWL-type ontologies with additional XML schemas for both the STAR and AIAG organizations. As a follow-up step, the ontologies are merged and checked for unsatisfiable conditions. A comparison of the schemas of both standards (STAR & AIAG) with the OAG-created ontology is performed. Since the schemas are based on the OAG standard, the assumption is made that there will be a high degree of commonality. The created ontologies are then referenced in the following transformation step.

In the transformation step, the XML document received in STAR format will be transformed to an OWL representation (XML2OWL). In addition, the target definition of AIAG is retrieved

and compared using an OWL integrator. Any differences are identified and resolved with the applied reasoner. Finally, the output document for the AIAG-type document is created and forwarded.

With the architecture described above, the advantages of applying semantic web concepts becomes evident through the fact that standard document types in XML format can be transformed based on their XSD definitions into an ontology representation. Compared to semantic web services, it eliminates the need for the intermediary step of creating a WSDL-defined document. From this perspective, the semantic web solution provides more flexibility.

From an execution perspective, semantic web services can be invoked by a client, whereas in this solution an ontology creation and transformation orchestration functionality must be provided. This orchestrator would manage the sequence of execution of the various XSLT transformations in both the ontology creation and the message transformation process.

The proposed solution by Haller et al. [23] was previously discussed and described. As part of the project evaluations, Haller et al. propose an integrated solution built on WSMO/WSML. Their solution comprises a multi-tier architecture. However, given the list of service offerings within the architecture, the solution does not provide any integration to existing integration platforms and must be considered a standalone approach. The ontology definitions also reside inside the WSMO definition which are tightly coupled to the respective web service functionality. Standards such as RDF and OWL are not applicable and therefore could provide challenges in integrating to other environments.

As the last project, Shi et al. [35] describe an approach where two heterogenous systems have to share a bill of materials. The key concept is to use multi-layered ontologies to map and merge meaning for the source and target applications. Their described approach is very similar to Izza et

al.'s [8]. Shi et al. also distinguish the ontology creation process from the actual mapping process, similarly discussed in Anicic et al. [34]. The solution discusses the creation of ontologies and the mapping process between two document types; however, it is lacking in a detailed implementation and execution description. Therefore, it's difficult to assess how feasible and practical it would be to integrate the architecture into a respective business process.

Table 2 summarizes the evaluation and findings of the nine discussed projects. Most of the projects are semantic web services and are therefore preconditions to a specific underlying technology platform. Mecheri and Souici-Meslati [36] provided a survey of semantic web services solutions. The approach presented in Table 2 differs from their approach in that it includes proposed solutions of the semantic web. Compared to web services approaches, constructs from the semantic web such as RDF/RDFS and OWL are more generic and are not tied to a specific technology definition; and should therefore be preferable. Using service components to distribute functionality such as ontology creation, binding, and mapping is essential. But services can be published in ways other than just by using SOAP and WSDL.

Table 2. Semantic Transformation Implementations.

Title & Author	Year	Standards	Model	Semantic Integration	Pros	Cons
Trastour et al. [7] Using Semantic Web Technology to Enhance Current Business-to-Business Integration Approaches	2003	DAML-OIL XML	NILE	Semantic web	- Use of RDF - Use of knowledge base - Able to onboard business partners with new constraint definitions	- Only addressed ROSETTANET - No clear description of underlying technology. Is the process synchronous or asynchronous?
Izza et al. [8] A Framework for Semantic Enterprise Integration	2006	DAML OWL-S+	ODSOI	Semantic web services	- Use of RDF and DAML - Mediation approach is based on services	- Multi-level ontologies provide great flexibility, but also add complexity

Title & Author	Year	Standards	Model	Semantic Integration	Pros	Cons
					- Central Ontology Service Bus	- Only XML based - Not clear if the ontology service bus can handle synchronous message processing since the different layers will add to latency
Bouras et al. [9] Dynamic Data Mediation in Enterprise Application Integration	2009	SAWSDL	SEAP	Web service with semantic annotation	- Using the SAWSDL standard - Support for different standard message structures through import into SEAP tool - Special enterprise interoperability ontology	- Require annotation maps with XSLT for up-cast/down-cast - Not clear if the web service configuration can handle synchronous processing - Development and maintenance of an enterprise interoperability ontology could be complex and time consuming
Vuković et al. [10] Semantic-Aided Automation of Interface Mapping in Enterprise Integration with Conflict Detection	2016	RDF & OWL-S	SAIL	Semantic web services	- Input uses CSV formats which is a common structure - Using predefined mapping functions to resolve differences and annotate data elements with semantic terms - Back-end call to ERP via web service	- Example only shows CSV processing which assumes asynchronous integration - Domain-specific language developed for semantic-aided automation of interface mapping
Roberto et al. [31] A semantic web service-based architecture for the interoperabilit	2005	WSMO	Government portal	Semantic web services	- Use of WSMO - Use of ontology domains - Scenario describes a government	- Integration applicable for the scenario but can only handle semantic web services with WSMO

Title & Author	Year	Standards	Model	Semantic Integration	Pros	Cons
y of e-Government services					- portal integration - Functionality extension through new services	- Other standards not supported which could be an issue when providing portal access to other technologies
Anicic et al. [34] An Architecture for Semantic Enterprise Application Integration Standards	2006	OWL/XSD/XSLT	STAR-AIAG OAG	Semantic web	- Use of semantic web transformation with XSLT - Common document standard OAG BODs - Ontology generation from schema	- Lack of synchronous process - Not clear if unsatisfiable conditions between ontologies need to be resolved manually - Requires orchestration manger to manage invocation of XSLT maps
Haller et al. [23] WSMX – a semantic service-oriented architecture	2005	OWL/XML	WSMX	Semantic web services	- Multi-tier integration approach by providing various functions such as orchestrator, mediator, goal searcher	- Based solely on WSMO/WSML defined web services - No semantic web components
Shi et al. [35] Integration framework with semantic aspect of heterogeneous system based on ontology and ESB	2014	OWL/XML	PDM/CAPP	Semantic web services	- Multi-level ontology development - Use of enterprise service bus with multi-tier architecture	- Solution is specific for predefined use cases - No discussion on resolving ontology differences

Using a layered form of ontologies such as local, domain, and global ontologies is also important since it breaks down complex subject areas into a manageable scope rather than building large complex structures that are difficult to maintain. Overall concepts such as distributed service architecture, ontology representation through triple stores (i.e., RDF), abstraction of architecture

components, and generic interface definitions are all characteristics to be considered in the design and development of a semantic transformation solution for interoperability.

3 Research Methodology

There are numerous research methodologies that are suitable for and specific to information systems research [37]. The main differentiator of these methodologies is the proposed process of execution through various phases to deliver the research results as artifact. Most of the methodologies are presented as a sequential approach with minimal direct feedback or continuous improvement loops. Based on our research questions we opted for the Systems Development Research Methodology (SDRM) [38]. SDRM provided the best fit to deliver the research life cycle from framing the research question to creating a workable solution or product. The research methodology is a phased approach structured as follows: defining the research question; establishing the associated requirements; designing a conceptual framework; defining a proposed system architecture; designing and constructing the solution; and finally, testing, observing, and evaluating the constructed research product.

SDRM is a sequential approach that evaluates research results in the final phase of observation. Since there is no endorsed official agile research approach [39], we opted to incorporate a frequent feedback cycle and incremental development in our research. New research discoveries were tested and integrated into the code base if these new increments contributed to the overall solution in addressing the defined research question. This iterative approach contributed to publishing a minimum viable product (MVP) early in the research cycle to validate research progress. We identified several epics that would encompass a feature set for research. These epics included the discovery of common semantic transformation patterns, research on conflict resolution within semantic transformation, and collaboration of services. Each of these epics would contribute to delivering an MVP that could be presented, reviewed, and observed. Such an MVP for pattern

discovery and conflict resolution was presented in our publication [Appendix A]. The subsequent step, accomplished by the findings and discoveries in this paper, is to refine the research from the MVP into minimal marketable features (MMF) [40]. Part of the delivered solution requires the identification of enablers that allow for the construction of the MMF as a solution. One such enabler is a suitable architecture with a supporting technology stack. It is therefore a requirement to define an architecture on which the MMF can be delivered. This process involves refining the requirements from the MVP and mapping them to architectural characteristics, which drives the selection of an appropriate architecture pattern. Along with the selected architecture pattern, a service catalogue needs to be defined that lays out the right service decomposition and communication strategy for the solution. Throughout this process a continuous improvement cycle is adopted and implemented that uses the research findings to make necessary adjustments both in design and construction of functionality. An example of such an iterated research approach is the application of NLP concepts in analyzing choreographed service workflows, which was not considered in the initial design iterations. Our solution therefore supports an incremental research approach that is based on continuous validation of findings while making necessary adjustments to the final MMF.

4 Use Case Description

The bases for this research project are two use cases. The data for each use case represents a real-world scenario and will be analyzed and used for testing and validating the developed concepts. The first use case describes the interchange of pipeline transactions to schedule, ship, and receive pipeline batches. The second use case describes an event scenario where temperature readings from sensors are processed at different plant locations and transmitted for data analytics purposes to a back-end analytics system. Both use cases are independent of each other and have no relationship either in data or process; even the underlying technologies are different. Where the first use case is based on traditional messaging, the second use case uses events. This ensures that the developed architecture solutions can be applied to any number of diverse scenarios.

4.1 Message-Based Use Case

The pipeline use case has three actors that need to communicate the delivery status of pipeline batches with each other. The first actor is the carrier who either has pre-reserved pipeline space or is the operator of the pipeline. The carrier ensures the delivery of the batch from shipper to receiver. The second actor is the shipper who is requesting the shipment of a batch. The third actor is the transaction broker who manages the transaction exchange between carrier and shipper. In this case the shipper is also a receiver. This is the case when crude oil is shipped from exploration to refinement or when refined products are shipped to storage locations.

The structural format of the transactions is standardized by the definition of the PIDX standard [41] and are defined as XSDs. This use cases utilizes three transactions. The first transaction is the nomination, which is a request by the shipper to the carrier to reserve pipeline space in the future.

The second transaction is the shipment transaction, which informs the actual shipping date of the batch. The final transaction is the confirmation of the shipped batch. All actors are business partners. Fig. 2 depicts the transaction flow between the business partners. As a transaction is forwarded to the next end point, a transformation occurs. Each transformation has syntactic and semantic components. The syntactic transformation addresses any structural changes that the receiver requires. The semantic transformation deals with maintaining the meaning of data values between sender and receiver. The shipper receives the transactions from the broker and first passes them to production planning. After that, the transactions are passed to the refinery as a production schedule of refined products.

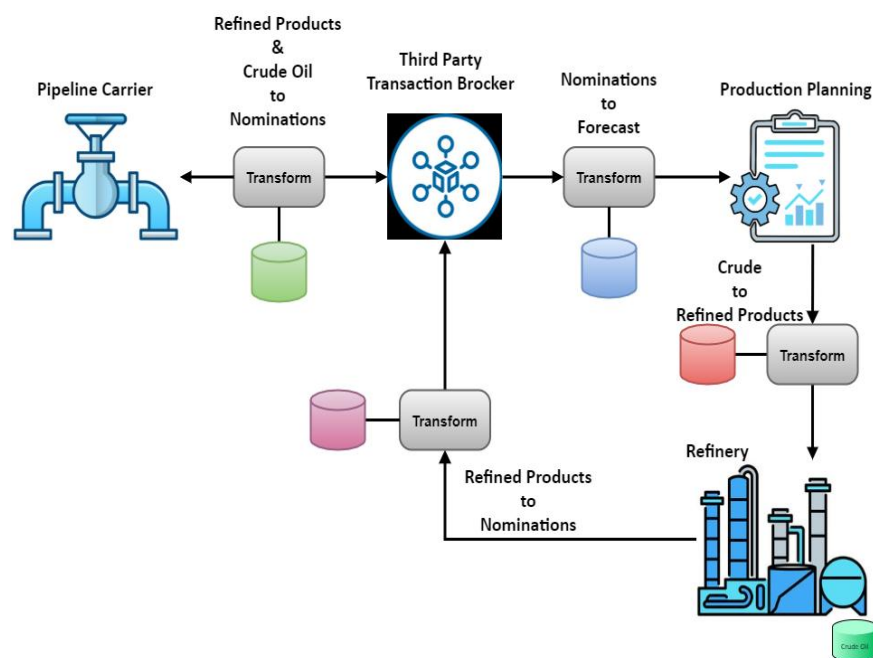


Figure 2. Message Flow Pipeline Scenario.

From a syntactic perspective, the transaction needs to map from an internal format to the PIDX format. On the receiving end the reverse happens. The following (Fig. 3) is a sample PIDX “Nomination Create” transaction.

```

<pidx:PipelineNominationCreate xsi:schemaLocation="http://www.pidx.org/schemas/v1.51
https://www.transport4.com/xml/schemas/PipelineNominationCreateV1-51.xsd" pidx:documentIdentifier="74844"
pidx:transactionPurposeIndicator="Add" pidx:version="1.51">
  <pidx:PipelineNominationCreateProperties>
    <pidx:CreatedByPartner>
      <pidx:PartnerInformation partnerRoleIndicator="Shipper">
        <pidx:PartnerIdentifier partnerIdentifierIndicator="AssignedByCarrier">HUS</pidx:PartnerIdentifier>
      </pidx:PartnerInformation>
      <pidx:EventDateTime>2018-06-03T00:00:00</pidx:EventDateTime>
    </pidx:CreatedByPartner>
    <pidx:PartnerInformation partnerRoleIndicator="Shipper">
      <pidx:PartnerIdentifier partnerIdentifierIndicator="AssignedByCarrier">HUS</pidx:PartnerIdentifier>
    </pidx:PartnerInformation>
    <pidx:PartnerInformation partnerRoleIndicator="Carrier">
      <pidx:PartnerIdentifier partnerIdentifierIndicator="AssignedByCarrier">BPL</pidx:PartnerIdentifier>
    </pidx:PartnerInformation>
    <pidx:ProductInformation>
      <pidx:ProductIdentifier assigningOrganization="AssignedByMarketplace">190</pidx:ProductIdentifier>
    </pidx:ProductInformation>
    <pidx:PipelineCycle>23</pidx:PipelineCycle>
    <pidx:PipelineCycleYear>2018</pidx:PipelineCycleYear>
    <pidx:PipelineSequence>1</pidx:PipelineSequence>
    <pidx:PipelineSCD>M</pidx:PipelineSCD>
    <pidx:ReferenceInformation referenceInformationIndicator="ShipperNominationNumber">
      <pidx:ReferenceNumber>74844</pidx:ReferenceNumber>
      <pidx>Description>Lifting Batch Record</pidx>Description>
  </pidx:PipelineNominationCreateProperties>
</pidx:PipelineNominationCreate>

```

Figure 3. PIDX Nomination Transaction.

In addition, the shipper might have multiple applications, of which each have their own proprietary format. To ease the transaction exchange within the shipper's application portfolio, a standard structural format has been defined which is also called a canonical structure. Fig. 4 shows a sample of the three transactions with transaction type "N" for nomination, "S" for shipping notification, and "T" for shipping confirmation in a canonical format.

<pre> <record> <SourceSys>Transport4</SourceSys> <SourceLoc>DS</SourceLoc> <CarrierCode>BPL</CarrierCode> <ShipperCode>BPL</ShipperCode> <Tankage>SPL</Tankage> <Line>SN2</Line> <PurposeIndicator>Replace</PurposeIndicator> <PlantID> </PlantID> <AssetID> </AssetID> <DataSourceName>2122034</DataSourceName> <ImportExportID>2122034Imp8842</ImportExportID> <StartTimeUTC>2019-11-09T19:00:00</StartTimeUTC> <EndTimeUTC>2019-11-10T01:15:00</EndTimeUTC> <VesselName>S BPL-HU2-182-46P-M</VesselName> <ImportExportType>2</ImportExportType> <Amount>15000</Amount> <UOM>BR</UOM> <MaterialName>182</MaterialName> <TransType>S</TransType> </record> </pre>	<pre> <record> <SourceSys>Transport4</SourceSys> <SourceLoc>CDP1</SourceLoc> <CarrierCode>INP</CarrierCode> <ShipperCode>INP</ShipperCode> <Tankage>BHH</Tankage> <PurposeIndicator>Add</PurposeIndicator> <PlantID> </PlantID> <AssetID> </AssetID> <DataSourceName>6047079</DataSourceName> <ExternalID>1914.0.0</ExternalID> <ImportExportID>6047079Exp5342</ImportExportID> <StartTimeUTC>2019-11-09T11:55:00</StartTimeUTC> <EndTimeUTC>2019-11-09T14:36:00</EndTimeUTC> <VesselName>T INP-BPO-S83CB27-456-INW</VesselName> <ImportExportType>2</ImportExportType> <Amount>12000</Amount> <UOM>BR</UOM> <MaterialName>S83CB27</MaterialName> <TransType>T</TransType> </record> </pre>	<pre> <record> <SourceSys>Transport4</SourceSys> <SourceLoc>LA</SourceLoc> <CarrierCode>BPL</CarrierCode> <ShipperCode>BPL</ShipperCode> <Tankage>HUS</Tankage> <PurposeIndicator>Add</PurposeIndicator> <PlantID> </PlantID> <AssetID> </AssetID> <DataSourceName>7903249</DataSourceName> <ExternalID>1098933873</ExternalID> <ImportExportID>7903249Exp6514</ImportExportID> <StartTimeUTC>2020-04-03T04:00:00</StartTimeUTC> <EndTimeUTC>2020-04-03T08:00:00</EndTimeUTC> <VesselName>N BPL-HUS-182-144-M</VesselName> <ImportExportType>2</ImportExportType> <Amount>15000</Amount> <UOM>BR</UOM> <MaterialName>182</MaterialName> <TransType>N</TransType> </record> </pre>
--	---	---

Figure 4. Canonical Transactions.

The material code which is named Material Name in the transaction is the code received from the broker and does not match to material codes used by the refinery. In addition, Plant ID and Asset ID are missing but are required by the production planning and refinery application. Therefore, a semantic transformation must occur to determine the corresponding material code and to populate the Plant ID and Asset ID fields.

Fig. 5 describes the relationship between the received material code and the internal material code for the refinery. It clearly shows that the material code in the transaction of “S83CB27” must be converted to “LRC – CBOB Reg (15psi).”

referencenotes character varying (255)	mtxloc character varying (255)	sourcematerialcode character varying (255)	targetmaterialcode character varying (255)
	ML-000002	073	H-ULSD Summer
	ML-000002	510	H-Jet A
	ML-000002	889	H-Alky Butanes
23Dec2019:email from LIMA: Use ?Reformat - E...	ML-000002	899	Reformat - Example Grade
	ML-000002	35	H-ULSD Winter
	ML-000002	018	LRC - CBOB Reg (15 psi)
	ML-000002	003	LRC - CBOB Reg (9 psi)
	ML-000002	S83CB27	LRC - CBOB Reg (15 psi)
	ML-000002	009	LRC - CBOB Reg (9 psi)
	ML-000002	013	LRC - CBOB Reg (15 psi)
	ML-000002	032	H-Diesel Canadian A
	ML-000002	030	H-Diesel Canadian B

Figure 5. Material Code Cross-Reference.

To add additional complexity, some received material codes could map to multiple material codes in the refinery application. In Fig. 6 below, for example, material “017” has two possibilities: either “LRC – Gasoline Pre-91 (11.5 psi)” or “LRC – Gasoline Pre-91 (13.5 psi).”

sourcemtrcode character varying (255)	targetmtrcode character varying (255)	effectivedatefrom timestamp without time zone	effectivedateto timestamp without time zone
016	LRC - Gasoline Pre-91 (15 psi)	2019-08-01 00:00:00	2019-09-30 00:00:00
018	LRC - CBOB Reg (11.5 psi)	2019-08-01 00:00:00	2019-09-29 00:00:00
018	LRC - CBOB Reg (13.5 psi)	2019-06-21 00:00:00	2019-07-31 00:00:00
014	LRC - Gasoline Pre-91 (13.5 psi)	2019-06-21 00:00:00	2019-07-31 00:00:00
013	LRC - CBOB Reg (13.5 psi)	2019-06-21 00:00:00	2019-09-30 00:00:00
014	LRC - Gasoline Pre-91 (15 psi)	2019-08-01 00:00:00	2019-08-30 00:00:00
013	LRC - CBOB Reg (11.5 psi)	2019-10-01 00:00:00	2019-11-30 00:00:00
017	LRC - Gasoline Pre-91 (11.5 psi)	2019-08-01 00:00:00	2019-08-31 00:00:00
017	LRC - Gasoline Pre-91 (13.5 psi)	2019-06-21 00:00:00	2019-07-31 00:00:00
016	LRC - Gasoline Pre-91 (13.5 psi)	2019-06-21 00:00:00	2019-07-31 00:00:00
090	H-Diesel Canadian B	2019-06-21 00:00:00	2019-07-31 00:00:00
S83CB27	LRC - CBOB Reg (13.5 psi)	2019-06-21 00:00:00	2019-07-31 00:00:00

Figure 6. Material Conditions.

The distinguishing factor is a condition. The transformation needs to determine under which condition the right target material code applies. The condition in this case is a time window; but it could also be a numeric range. Other transformations could be plant identifications, location codes, tankage codes, pipeline segments, asset codes, and units of measure. For a unit of measure, a semantic transformation could be from “bbl” to “BR” for barrels.

Fig. 7 depicts the underlying architecture of the transformation between endpoints. It consists of the capability to send and receive messages as well as integrate a cross-reference database that serves as a lookup for the previously described semantic transformation scenarios. The middleware component can be either custom logic, an enterprise service bus, or any other software that can perform the required transformations. The primary communication in this use case is SOAP using web services, partly because the PIDX format is based on XML. All web services function synchronously, which means that the client will wait for a response. Other message scenarios could use data structures with other API types [42] such as REST, WebSocket, or gRPC. It is also

possible to have direct database connections or file ingestions where files can have various formats other than CSV.

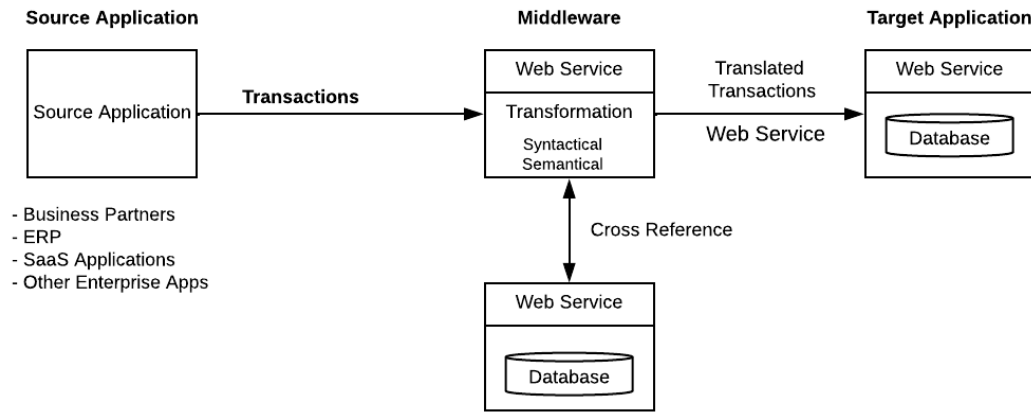


Figure 7. Interoperability Architecture.

The cross-reference database can be an individual or shared database. In many cases, these are individual databases that have been created to address the semantic transformation scenarios for the specific transformation. Maintaining these cross-reference tables and keeping them current is a challenge, given the dependencies within the data (i.e., material records with condition records). Inaccurate or missing reference data will cause transaction failure and therefore disrupt the overall transaction flow. A disruption of transaction flow will not only impact the communication between internal applications and their business processes, but can also have a ripple effect on business partners who rely on the transaction exchange.

This use case demonstrates a typical scenario that in this case is taken from the oil and gas industry. However, the same data flow can be found in banking or other industries and therefore represents an excellent scenario for this research to generalize a solution for semantic transformations.

4.2 Event-Based Use Case

Where the previous scenario is a message- or transaction-based scenario, the second use case (Fig. 8) describes the scenario of streaming event data from several sensors in a plant to measure and then analyze that data. Another scenario could be the trigger of alarms when certain metric thresholds are reached or passed. In most cases the sensors are identified via a tag ID. This tag ID, however, does not map with the actual sensor identification in many of the production planning applications, and must therefore be translated to the respective plant ID for the production planning system to be able to process the data.

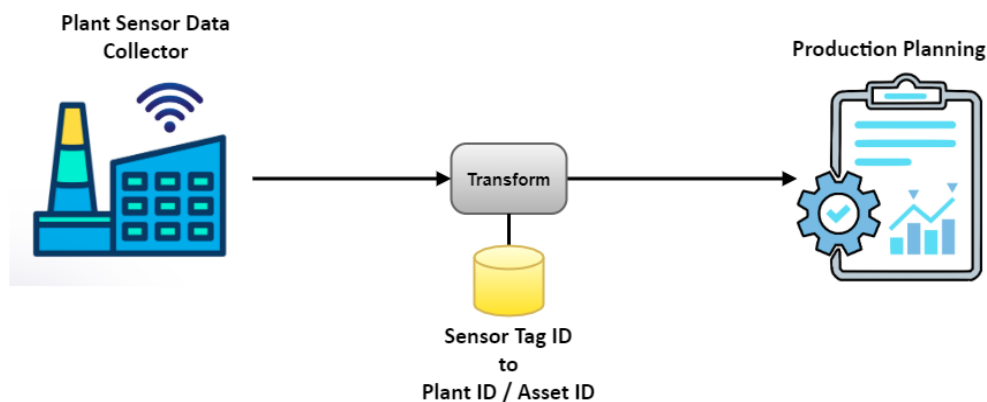


Figure 8. Event Streaming of Sensor Data.

In addition, the event needs to be enriched with additional data such as zone ID or asset ID which are not part of the original event data. Event processing requires an event broker that applies the publish and subscribe pattern. Apache Kafka [13] is an example of such a broker. The broker receives the event from the source and publishes the event to the respective subscriber queue. Events are immutable and therefore cannot be changed. This is a key difference between messages and events. To add additional data items to the event, the broker must create a new event from the

received event with the enriched data and publish the new event to a queue for the subscriber to consume.

With the popularity of IoT in the past two decades [43], event processing has become a critical part of an IT infrastructure. The described use case can be applied to other scenarios utilizing an event-driven architecture style [43]. Developed solutions for semantic transformation within the event-driven context should be generalized.

5 Discovery of Semantic Transformation Patterns

Vuković [10] presented a set of mapping functions that would aid in the semantic transformation process. Our approach borrows this concept by identifying four semantic traversal patterns applied to graphs, described below. These patterns were discovered in analyzing 250,000 transactions and their reoccurring transformation requirements from the described use cases.

5.1 One-To-One Pattern

The simplest and most common of all the patterns is the key-value lookup. For a specific key-value a given corresponding transformation value is provided.

Description

This pattern acts as a key-value lookup by providing a property value and the expected value of the property.

Requirement

Many times, semantic lookups require a quick determination of a value based on a key value. In this pattern the key is provided by the request node and property.

Problem

Given the value of a certain property of a node (key node), find the value of the property of a node in relationship to the key node.

Solution

The request node name and associated property value will be used as argument (key) to find the related response node and property value given a defined relationship between the request node and response node.

Application

The one-to-one pattern is commonly applied for single-argument lookups. Given a specific source value, the pattern will retrieve the respective associated value. An example would be finding the corporate product code given the product code supplied by a business partner.

Implementation

```
Match (RequestNode.Property.Value) -[]-> (ResponseNode.Property)  
Return ResponseNode.Property.Value
```

A relationship between nodes is presented as []

5.2 Enrichment Pattern

The Enrichment Pattern is used to add data elements that do not exist in the original message based on a key.

Description

This pattern provides additional property values based on a single key property.

Requirements

There is the requirement based on a key value to retrieve additional values to enrich the target data structure.

Problem

Given a source data structure that needs to be translated into a target data structure, the target data structure might require additional data elements that do not reside in the source data structure. These data elements must be retrieved from other sources and added to the transformation.

Solution

For each requested enrichment property, the pattern will use the provided key to locate the request node and traverse via the defined relationships between nodes to the different response nodes; retrieve their property; and return the respective values as the enrichment values to be added to the target structure.

Application

A good example is sensor tags from a plant. If the tag represents a valve, the source structure may include the flow rate and volume; but it will not include any reference to a plant ID or asset ID that the target system is expecting. Therefore, the data values must be enriched to satisfy the mapping requirements of the target application.

Implementation

```
Foreach(ResponseNode.Property.Value){  
  Match (RequestNode.Property.Value) -[>-> (ResponseNode.Property)  
  Return ResponseNode.Property.Value  
}  
A relationship between nodes is presented as []
```

5.3 Dependency Pattern

Description

This pattern provides a response property value based on a single key property. However, the retrieval of the response property value is dependent on intermediary nodes and properties. This means a navigation path needs to be established between request and response node.

Requirement

Semantic transformations often require determining a target value based on intermediary associated lookup conditions, starting with an initially provided key.

Problem

To derive a target value for a given semantic transformation, several intermediary lookup conditions must be exercised which can all influence the determination of the requested target value. This requires a multi-hop navigation logic to derive the final value that is required in the target data structure.

Solution

The pattern will use the initial key/value as starting node/property and chain the provided request nodes that make up the lookup chain to establish a navigation path to the response node and its property value. If multiple path options exist for a node, only the path that has the next provided node/property value will be taken. If multiple response nodes are available for a given request node, the relationship between the last request node and response node must contain the property value of the last request node-1 property value. Otherwise, it is impossible to find the correct response node.

Application

The application is to determine an asset ID based on a material code, a location code, a tankage, and a pipeline segment. The navigation path among nodes would be Material-> Location-> Tankage-> Pipeline Segment-> Asset. However, it is possible that a pipeline segment would not be provided, in which case the navigation path for the nodes would be Material-> Location-> Tankage-> Asset. In addition, there could be multiple asset IDs related to a tankage or pipeline segment code. The relationship between tankage and asset and pipeline segment and asset would need the location code to be attached as differentiating value.

Implementation

```
Loop {  
  Match (RequestNode.Property.Value) -[]-> (NextRequestNode.Property)  
  If NextRequestNode = Empty  
    LastRequestNode = RequestNode  
    Terminate Loop.  
}  
Match (LastRequestNode.Property.Value)-[RequestNode-1.Property.Value]-  
(ResponseNode.Property.Value)
```

A relationship between nodes is presented as [Contains the property value of the request node – 1 property value]

5.4 Granularity Pattern

Description

Based on a key/value, multiple options may exist for a target value.

Requirement

Semantic transformations can encounter the situation in which multiple target values may exist for a given key/value. Therefore, the requirement is to determine the right target value based on a selection criterion.

Problem

Given a key, multiple options exist that could be suitable for a correct target value. The problem is to determine the right target value from a list of options.

Solution

The request node (key) will be used as argument to find the related response node and property value from a list of possible options. This is accomplished by introducing conditions on the relationship between source and target that help determine the correct target value. Such conditions could be time windows and value ranges.

Application

A common scenario is products. A business partner integration provides the product identifier to the integration as part of the transaction. Internally, this product is further refined in various grades, specifications, or characteristics. However, the source message is missing these vital characteristics. A time window as part of the relationship between source and target value is provided to determine if the response should be a winter product or summer product.

Implementation

```
Match (RequestNode.Property.Value) –[Condition]-> (Response.Node.Property)  
Return ResponseNode.Property.Value
```

A relationship between nodes is presented as [Condition either Time window or Value Range]

6 Semantic Transformation Framework

6.1 Semantic Transformation Utilizing Graphs

As presented in the literature review, the semantic web provides significant benefits in representing entity relationships via the definition of RDF triplets, which in return can be presented as graphs. The scope of the graph structure will be the described use cases. This has several advantages. Firstly, it limits the initial effort in developing the graph compared to building a global ontology and then reversing it back to a graph structure. Secondly, a graph structure can easily be extended to another graph structure or by attaching new nodes. Therefore, this approach presents significant savings in development time and cost.

Graphs provide a network of nodes which are connected via relationships (also called edges). Each node can have various types of properties, and the network of nodes can easily be adjusted to meet new and changing lookup requirements. The search for a corresponding semantic value happens by traversing nodes. The entry point into the graph can be any node; by traversing the relationships, one can traverse from node to node until the final node with the respective property value is located.

Another advantage of graphs is the ease in changing the graph structure. New nodes and edges can be either added or removed without impacting the overall graph. This feature is particularly relevant in semantic transformations since new values and their relationships can be added or removed frequently. Compared to relational databases, graph structures do not require a schema change or an unload/reload of the database. The traversal algorithms will adjust to the changing graph structures.

Graphs also transform easily into an ontology and can be used to build a knowledge graph for an enterprise. Combining the reference data with master data, an organization can start developing

graphs into a sophisticated network of knowledge which can contribute significant value beyond semantic transformation for data integration.

For the described use case and demonstration purposes we will use the following graph structure depicted in Fig. 9. Because of their node connectivity, graph structures can be applied to any use case by extending the node and relationships. Therefore, the two use cases can be addressed with a single graph.

The material node of the graph contains material IDs from a shipper that need to be translated into the material IDs of the application that is to receive the transactions. There is also the requirement to determine an associated asset ID. The asset ID can be located either by traversing from material to location, to tankage and pipeline segment, or from the tankage directly. The graph allows a two-way path to locate an asset ID. Traversal algorithms must be able to consider such scenarios. If one path does not deliver a value, the traversal algorithm will attempt to use the next possible path alternative to find an asset ID.

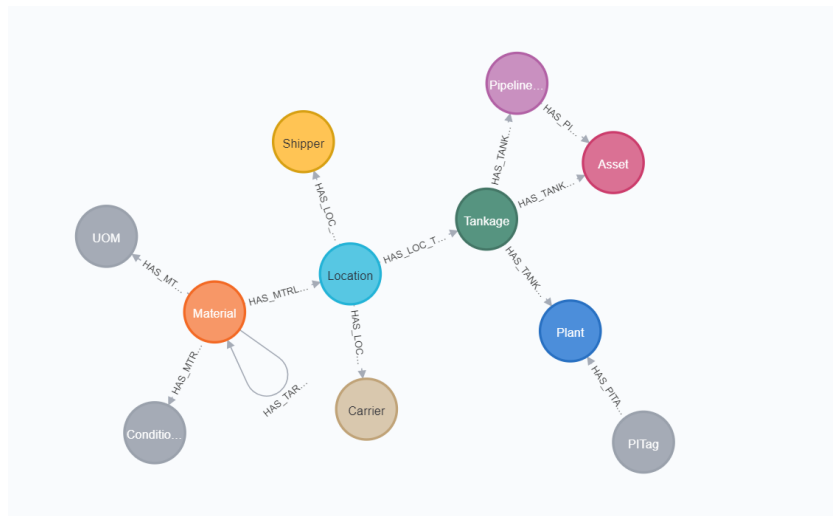


Figure 9. Base Graph for Use Cases.

Other transformation requirements include determining a shipper and finding a unit of measure. Unit of measure is a common value that needs translation since there is no standardization for the majority of units of measure. In this use case, for example, the unit of measure for a barrel of oil can be “BR” by the sending system and “bbl” for the receiving system. This scenario demonstrates that semantic transformations are most often code based and can be expressed through various code structures which in most cases have no relationship to each other or any grammatical rule base that can be applied.

For the second use case of translating a sensor tag ID to a plant ID, the plant ID can be attached to the tankage; and therefore the scenario is integrated into the overall graph structure. By designing an integrated graph and applying the traversal patterns, any combination of semantic transformation can be supported. A semantic transformation graph is the ideal data structure for these scenarios.

6.2 Microservices Architecture

Deviating from the semantic web services approach discussed in the literature review, this solution utilizes a microservices architecture. Microservices were first introduced in Lewis and Fowler’s original paper [44] as a new architecture style to implement more efficient service decoupling and advanced deployment strategies to meet the evolving demand for CI/CD. In essence, this architecture style is an evolution of service-based architecture [45], [46]. One of the differences lies in the segregation of databases, where each microservice will manage its own data source. Fundamentally, each microservice adopts the responsibility of a specific function within a bounded context [47]. The collaboration of these individual functions builds an overall software application within a domain. The management of such collaboration between microservices can be considered a workflow [48], [49].

Two scenarios exist [50]. The first scenario describes an orchestrated workflow, in which the invocation sequence is determined by a predefined rule set that can either be configured or is embedded in the code base and is initiated and managed through a microservice orchestrator. This scenario is easy to implement but causes a tight coupling between the microservice orchestrator and the actual microservices. The second scenario, called a choreographed workflow, proposes a looser coupling where microservices communicate amongst each other with the ultimate objective of providing an overall functional result. This scenario is much more complex to implement but achieves a greater rate of decoupling.

The microservice architectures provide a higher degree of agility in managing software functionality. However, they also present challenges in sustainability and data consistency that need to be considered when choosing this architecture pattern [51]. Prior to engaging in constructing a microservice solution, a sound architecture trade-off analysis should be done.

The four patterns that make up the core semantic transformations are functionally independent of each other; thus their implementation fits well with four distinct microservices (Fig. 10). Adopting a microservices architecture provides significant benefits in scalability, deployment, and maintainability, which were identified as requirements in the initial problem definition. Given the service independence, each service becomes its own architecture quantum. Each semantic transformation pattern presents a service; and these services can be combined into a service mesh. Changes to the code base will not impact other microservices, and for this reason, the microservices architecture provides a high degree of maintainability and deployability. Each architecture quantum can be deployed independently as part of a CI/CD pipeline.

Scalability is achieved in two ways. Firstly, each microservice can be instantiated multiple times to respond to its request load. Secondly, should new semantic transformation patterns be discovered, or unique functionality be required, additional microservices addressing those needs can be added and incorporated into the service mesh. Each microservice is its own deployment unit.

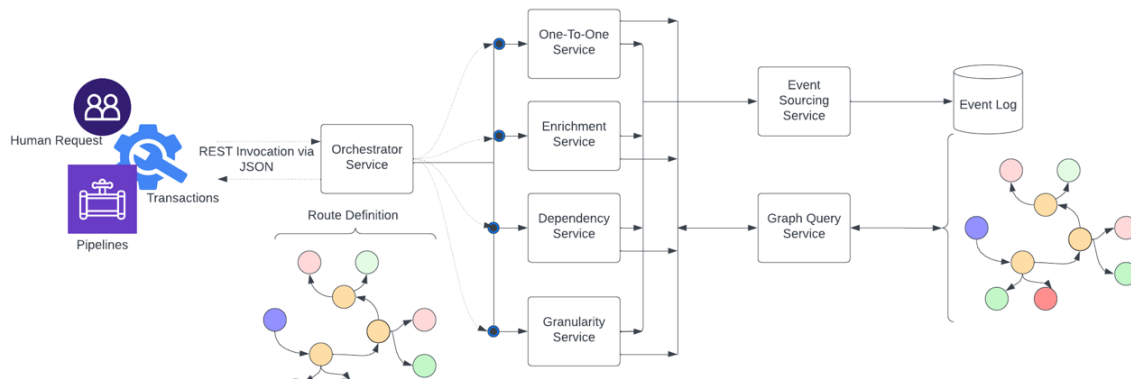


Figure 10. Orchestrator Architecture.

All services collaborate and communicate using protocol-aware heterogeneous interoperability, meaning that a common protocol is used; however, the form of the message structure may change depending on the number of key-value request parameters. The key-value parameters are packaged into a request object and are passed between the microservices. Semantic transformation results are placed in this request object. This way, one service can have access to results delivered by a previous service in the workflow if necessary.

Since each service performs a certain type of semantic transformation, an overall workflow capability is required that can manage the sequence of invocation and assemble the semantic transformation results to be passed back to the requester. In general, a microservices workflow is categorized into an orchestrated workflow and a choreographed workflow.

The development of the microservices architecture will adopt a test-driven development approach [52], [53]. Characteristic of this approach is that the test cases are designed prior to the actual construction of the functionality. The benefit to this is that the developed functionality incorporates the expected outcome from the start rather than building functionality first and adjusting test cases to those requirements. The quality of the solution therefore increases [52]. A test strategy for microservices can be structured in several steps [54]. The first step focuses on testing the functionality of the individual services. For the discussed semantic transformation solution, this means testing a single described pattern. In a classical definition this can also be called a unit test. The next step tests the integration and incorporation of two or more services working together to achieve the expected semantic transformation in an orchestrated fashion. This will ensure that a standard query can be executed. By tracking the traversals through log entries, a detailed evaluation of success and failure can be done. These log entries are then compared with the expected outcome identified in the respective test cases.

Given that the microservices each execute one of the described patterns, the testing is more focused on successful traversals rather than on individual value testing. The test cases will therefore increase in complexity not by changing test values but rather by increasing the number of node traversals to retrieve the expected semantic transformation. For example, the first test cycle would only test a simple one-to-one scenario. The second cycle would test a dependency but only with one or two node hops. The third cycle would test the combination of one-to-one with dependency, and so on. This ensures a gradual increase in complexity and a high degree of test coverage of the code. The contract testing phase will focus on the combination of the microservices within Kubernetes. In particular, the invocation or contract connectivity needs to be tested to ensure that the services can communicate with each other within this framework.

As discussed later on in this paper, the log tables play a significant role in validating that the traversals are performed correctly by the microservices and provide the bases for conflict resolution when nodes are missing.

Besides the actual testing of the microservices and their association with each other, the testing will also include the automation of the CI/CD pipeline workflow steps. These steps include the secure creation of container images for each microservice, deployment of these images to a central repository, and deployment to the actual Kubernetes environment. Each deployment workflow will require the scanning of the container for vulnerabilities such as secret scanning. In addition, these deployment workflows must incorporate static application security testing tools that will automatically scan images for code and dependency vulnerabilities.

6.3 Microservices Workflow Management

6.3.1 Orchestrator

In the orchestrated workflow scenario, the orchestrator receives the request message in JavaScript Object Notation (JSON) structure from various potential sources such as a pipeline, web form populated by a user, or general transaction processing. The JSON structure has some core elements such as message ID and source ID. However, the semantic terms are provided as a key-value term structure. One of the core elements that must be provided is a route identifier. The route describes the sequence of the microservice invocation. The orchestrator will access the routing graph based on the route ID provided and traverse the route definition in the graph to find the sequence in which the four microservices must be called. The route definition also provides the request and response parameters. The response parameter presents the expected result from the semantic transformation. The orchestrator then assembles the request object, which it passes to one of the four

microservices. This process continues until the orchestrator has traversed the whole route defined in the routing graph.

Each microservice generates an event log when performing a semantic transformation. This log entry will be passed to a special event log service that stores the log in an event log table. The event log structure adheres to the XES process log standard (www.XES-standard.org).

These event logs provide analytical insights into performance, error management, and predictions on missing nodes in the overall semantic transformation graph [9].

6.3.2 Choreographer with NLP

To test and demonstrate a choreographed approach for semantic transformation, we utilize natural language processing (NLP). In the context of this project, NLP is applied with two objectives in mind. The first objective is to use natural language to formulate a semantic transformation query within the scope of the back-end graph structure. The use of NLP provides an additional layer of abstraction from the front-end request systems and therefore further decouples the front-end applications from the actual semantic back-end transformation processes. The result is a wider range of application use than with a tight coupling approach. The second objective is the use of a choreographed workflow with microservices whereby the invocation of the semantic transformation microservices is based on the construct of sentences.

Sengloiluean et al. [55] present an approach that uses a generalized NLP query to retrieve information from two back-end knowledge bases. The algorithm described increases the accuracy of the result set and investigates similarities between text elements. In this context the query structure is more generic. Corporate semantic transformation is based on a distinct value set between a data source and its target. Therefore, the scope of the matching of terms is unique. The process of finding the right term may vary. The objective of the NLP query in semantic

transformation is not to retrieve explicit knowledge but to find the semantically related term that would most likely describe the entity in the target system. This term is then used in the transaction so that the receiving application can process the transaction.

The approach presented by Gupta et al. [56] shares some similarities with the approach presented in this paper. The objective of their research is to take a natural language query and transform it into an SQL query to query relational databases. The presented architecture provides the foundational components of query parsing by tokenizing the sentence structure. From the parsed token, an SQL query is then constructed that would be executed against a relational database management system (RDBMS). In its simplest form (key-value lookup), the described approach could be used for semantic transformation. However, the application of more sophisticated transformation requirements would result in more complex queries.

An approach to generate a knowledge graph from text using NLP is presented by Maksutov et al. [57]. The described algorithm parses the text into tokens and their inter-relationships. With the discovered word tokens and associated relationships, Maksutov et al. construct a knowledge graph. Once text is presented as a graph structure it provides new capabilities in analyzing relationships between nodes by performing, for example, community, similarity, and density analyses.

Within the orchestrator approach, the receiving request and the invocation sequence of the microservices are tightly coupled; and the invocation sequence is performed based on the definition of the routes in the routing graph. In contrast, the choreographed approach promotes a looser coupling (Fig. 11). The invocation sequence of the microservices is not predefined based

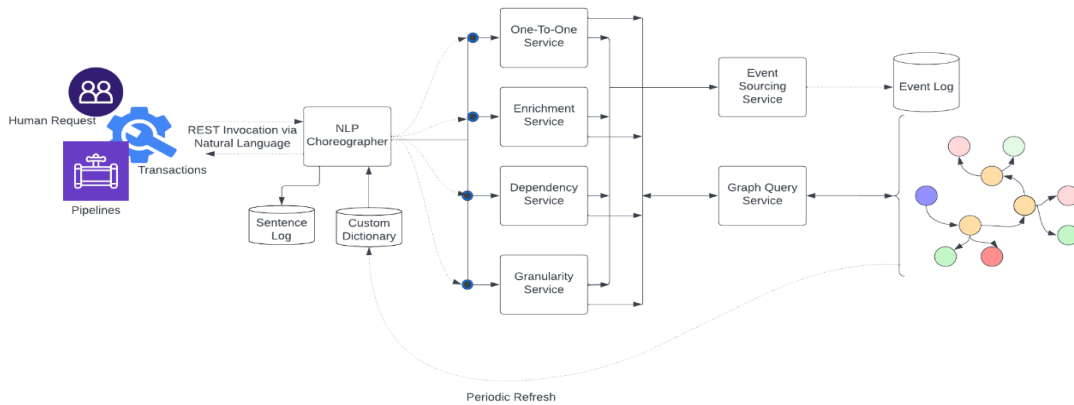


Figure 11. Choreographed Architecture with NLP.

on a route definition but is interpreted from the NLP choreographer of the sentence structure in the received request message. Each organization will have different ways of describing their business entities and attributes which in many cases differ from generally available vocabularies. The historical knowledge on language structure and vocabulary cannot be based on a general corpus of text but must be derived from the terms defined in the semantic graph. These entities with their attributes are represented as nodes and properties in the graph structure. Therefore, each node is made up of a label and its associated properties and their values.

6.3.2.1 Dictionary of Terms

Through the creation of a corporate dictionary that represents the label, property, and values, the NLP choreographer can interpret the different word tokens and discover their meanings related to the semantic graph.

The dictionary needs to be refreshed periodically when changes to the semantic graph structure occur. The dictionary also needs to accommodate scenarios of terms that are not represented in the back-end graph structure. For example, a label can be “Tankage” with assigned properties of tankage code and tankage description. An example query could be as follows: “Give me the tankage code for Location code KG.” The NLP choreographer will recognize “Tankage” as a label, i.e., node in the graph. However, the actual property name is “tankageCode” not “code”. To address these issues the dictionary allows for alias terms that can be applied on both label and properties. In this case an alias of “code” would be stored as an alias for “tankageCode”.

6.3.2.2 Grammatical Structure of the Natural Language Query

A natural language query must adhere to a few grammatical rules. As a first rule, a query must be structured into multiple query sentences. Each sentence represents a sub query that the NLP choreographer uses to determine which microservice pattern is to be invoked. This provides a solution to call multiple functional independent microservices within one query submission. The sentence parser analyzes the query, parses each sentence, and stores it in a sentence processing queue.

The second rule requires that the expected semantic response must be stated first in a query sentence, as in our previous example: “Give me the Tankage for tankageCode for Location locationCode KG.” It is not required that a label is provided as long as the label name is the prefix of a property that follows. For example, one can write “Tankage for tankageCode” as well as just “tankageCode” or “Tankage Code”. The language parser will recognize the different terms and break apart custom words like “tankageCode” to determine the required label.

The third rule requires that for value properties, the value must follow the property. Using the example sentence “Give me the Tankage for tankageCode for Location locationCode KG,” KG is the value of locationCode. Logical expressions among properties are currently not supported and are a topic for future research.

6.3.2.3 Language Parser

Given the grammatical rules described above, the language parser (Fig. 12) analyzes each sentence by performing a word tokenization over the sentence. Each sentence is mapped to only one invocation of microservices. To achieve a chain invocation as previously defined in the orchestrator scenario, multiple sentences must be provided. The language parser will specifically look for nouns (NOUN) and proper nouns (PROPN) as part of the word token. If it identifies either, it will search the dictionary for the term first as a label. If a label can’t be found, it will try to map the suffix of the word with aliases in the dictionary. If it finds one, it will parse the suffix from the word text and use the prefix for another attempt to find a label.

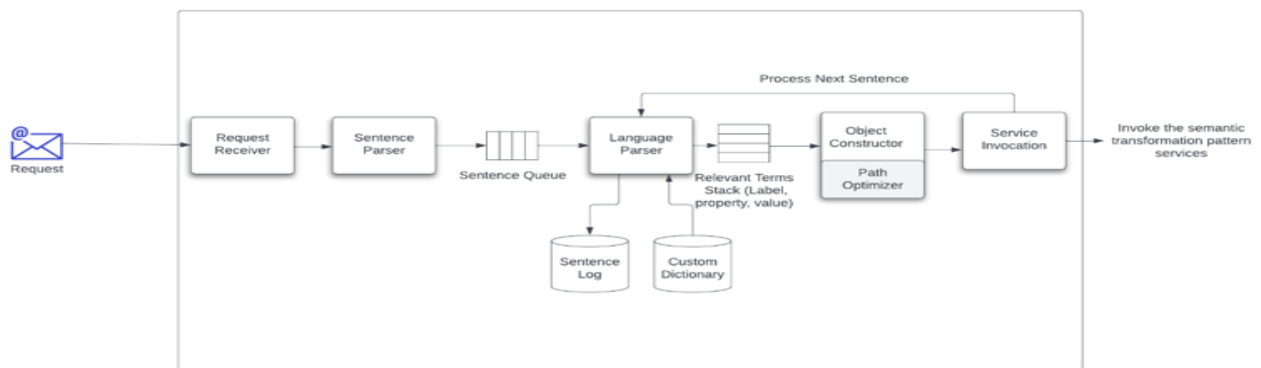


Figure 12. Language Parser.

For every word text found in the dictionary, the language parser will push the word into a stack. Once the sentence is parsed completely, the object constructor is invoked, which will process the stack and build the proper request key-value pairs with the final two entries being the response label and property as described in grammatical rule one.

The object constructor processes the stack and constructs the request object to be passed to the respective microservice. It will also determine through the path optimizer the optimal traversal path to find the correct semantic transformation value; and rearrange the labels and properties so that the microservice can follow the predefined traversal path. Once the object constructor has received a response, it will invoke the language parser again to process the next sentence.

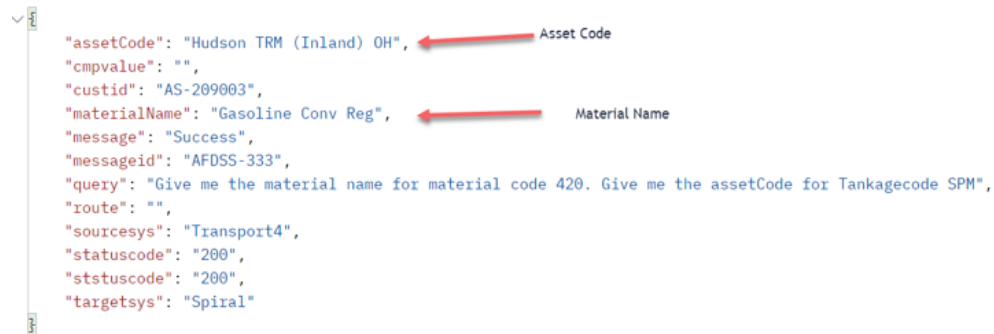
Each query sentence is stored in a sentence log file. The idea is that collecting various queries will eventually build a sufficient training set that can be used for parsing and improving parsing algorithms as well as for eventually generating dictionary aliases.

Fig. 13 demonstrates a sample request message of an NLP query with two sentences. The query is looking for the material name and the asset code for a respective material code and tankage code. Notice the different spelling of material code and tankage code. “Material code” is lower case and two words. In this case “Material” will be identified as label and code as property. Therefore, the language parser will look for an alias of “code” within the label “Material”. “Tankagecode” is one word and the first letter is upper case. The correct property spelling in the graph is “materialCode” and “tankageCode”. The language parser therefore has the capability to interpret the spelling to determine the right label and property name to find the correct result.

```
1 {
2   "custid": "AS-209003",
3   "route": "",
4   "messageid": "AFDSS-333",
5   "sourcesys": "Transport4",
6   "targetsys": "Spiral",
7   "cmpvalue": "",
8   "query": "Give me the material name for material code 420. Give me the assetCode for Tankagecode SPM"
```

Figure 13. Request Message.

Fig. 14 provides the response for the request query and shows the correct asset code and the material name of “Gasoline Conv Reg”.



```
{
  "assetCode": "Hudson TRM (Inland) OH",
  "cmpvalue": "",
  "custid": "AS-209003",
  "materialName": "Gasoline Conv Reg",
  "message": "Success",
  "messageid": "AFDSS-333",
  "query": "Give me the material name for material code 420. Give me the assetCode for Tankagecode SPM",
  "route": "",
  "sourcesys": "Transport4",
  "statuscode": "200",
  "ststatuscode": "200",
  "targetsys": "Spiral"
}
```

Figure 14. Response Message.

This query demonstrates the behavior of the NLP choreographer with multi-sentence structure to issue multiple queries. It is capable of deciphering the query sentence in such a way that a semantically correct query is constructed and passed to the correct back-end microservice to retrieve the results. In addition, the example demonstrates that companies can have their own unique code structure, and the language parser will be able to process these codes to correctly formulate a semantic request query for the microservices.

6.3.3 Evaluation of Workflow Approaches

The two workflow styles for microservices presented for the described use case of semantic transformation deliver various benefits as well as drawbacks. A comparison of the two methods is based on a set of selected architectural characteristics that can be considered most significant.

The choreographed approach provides an overall higher degree of adaptability to other environments and would accommodate a broader use case base within an organization. What is significant is that no structured format is required to invoke a semantic transformation compared

to semantic web services. If the grammatical rules are applied, the language parser can decipher the query and provide the transformation results. This feature is a significant benefit over the semantic web services where structured WSDL and SOAP definitions are necessary to invoke the web services. It therefore presents another level of abstraction where the user interface is decoupled from the underlying microservice orchestration. Therefore, the front-end (i.e., the query formulation part) can be replaced with different devices or even different APIs to invoke a semantic transformation on the microservices.

However, the complexity of parsing sentence structures and compiling back-end search queries offsets these benefits and therefore makes the orchestrated workflow an overall better solution. The implementation of an NLP choreographed invocation approach will therefore largely depend on the strategic direction that semantic transformation will take in an organization. If the intent is to build out the graph of terms to a corporate knowledge graph, then the choreographed approach would be a better choice over the orchestrated approach. For standard semantic transformation implementations, the orchestrated approach would be the best choice.

Because the request object acts as a contract between the microservices, it provides an architectural abstraction. Therefore, an initial implementation of the orchestrated approach is recommended with the plan to eventually migrate to an NLP choreographer to benefit from a higher degree of adaptability to various front-end applications. The existing microservices' functionality and the back-end graph would not have to change.

6.4 Semantic Transformation and Event Streaming

6.4.1 Event Streaming Concepts

The Internet of Things has penetrated every aspect of business and personal life. The collection and processing of this data occurs with data streams and requires different architectures compared to the classical batch and transaction processing. A major difference of processing event streams is that these streams are infinite and unbound in nature. Infinite describes the characteristic that data is continuously flowing, and unbound reflects that streams have no distinct data set limit. In addition, event streams are ordered based on event occurrence. This is important for events that require that their flow adheres to a defined sequence. Another characteristic of events is that they are immutable. Once an event is created it cannot be changed. Any changes required for an event requires the issuing of a new event with the desired changes to the stream. Immutable events therefore support the capability of replaying events so that they can be reprocessed. These characteristics of event streams therefore provide new challenges in managing optimal performance and reliability.

There are three fundamental concepts that describe stream processing [58]. First, event streams require the definition of a process topology that defines the sequence of the individual processing steps. These processing steps may consist of aggregations, branching, manipulation, and filtering.

Secondly, the processing of event streams may consider different time windows such as fixed, defined, or floating time windows. In addition, time events can be of different types to reflect either processing time, log append time, or ingestion times. The complexity of time management within event streams can be demonstrated when averages are to be calculated within a five-minute time window and a potential broker goes down for maintenance or other issues. Sheehy's discussion on time management in distributed systems [59] profoundly demonstrates this complexity.

Finally, stream processors need to consider stateful or stateless scenarios to accurately compute desired results on these event streams. Compared to stateful events, stateless events do not require context such as history or previous calculation results to compute a new result. Maintaining state will therefore require an intermediate storage capability in the event stream to ensure that the state context can be retrieved.

Given the streaming characteristics described above, it is evident that event streams require an architecture pattern that will support these characteristics. Streams are composed of events that are continuously flowing, thus forming an event stream. Events are records of something that happened at some point in time [60]. Events should not be confused with messages. The difference between events and messages is that messages indicate to the receiver an action to be taken, whereas events indicate that an action has occurred.

A common architecture pattern for event streams are event brokers [61]. Events arrive asynchronously and are published via an event producer to a topic for consumption by a topic consumer. These consumers can in return perform some computing on the received events and either persist them in a data store or publish the computed event back to a topic. Since the publish and subscribe pattern is implemented between producer and consumer, many consumers can retrieve the same event. A special consumer distribution logic is not required, and this is what makes this topology so attractive. Brokers are part of a cluster which provides significant scaling capabilities to process thousands of events. The following section will discuss two potential architectural scenarios of integrating semantic transformation using graphs and microservices with event streams.

6.4.2 Semantic Transformation with Pub/Sub

In the first scenario, the events that are created by sensors are sent to an event producer which publishes these events to a topic (Fig. 15). For every transformation pattern a consumer monitors the topic. When a new event arrives, the consumer will call the transformation microservice with the respective tag ID and the expected return value. The microservice will execute the necessary graph traversal to find the translated value and return this value back to the consumer.

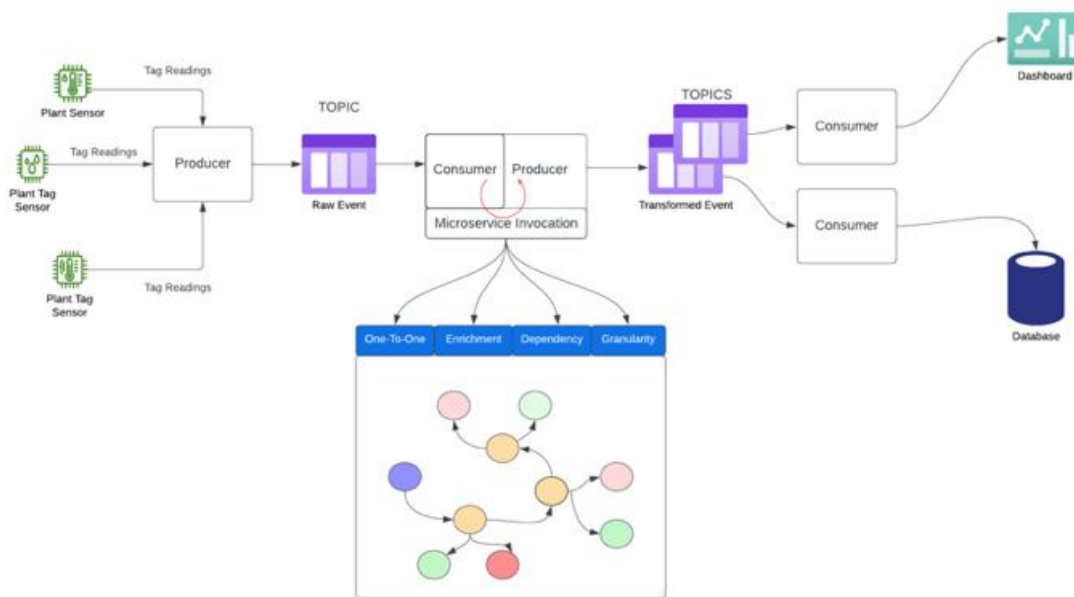


Figure 15. Event Hub.

The consumer in return will create a new event with the returned translated values; and will publish this new event as producer to various topics depending on downstream event processing requirements. This allows different “end” consumers to listen to different topics and use the events retrieved (which are now semantically translated and enriched) to further aggregate, store, or otherwise process the events. It is important that the original tag ID is passed through the process. This way, data lineage of the event is preserved and a full audit trail of the event stream is

maintained. The semantic transformation consumer will communicate with the respective microservices via a request object. The consumer therefore acts in the architecture as orchestrator of the four pattern microservices. This introduces abstraction into the architecture. Loose coupling allows the underlying microservice to change by introducing potential new transformation functionality and therefore the capability of horizontal scaling.

6.4.3 Semantic Transformation with Stream Processing

Compared to the traditional pub/sub scenario discussed in the previous section, stream processors execute a topology of APIs to perform event manipulation through aggregations or value mapping [61]. State is maintained by storing event values in special stream tables. In its simplest form, value mapping could be performed with the provided APIs. As previously discussed, however, semantic transformation requires more sophisticated mapping than simple key-value lookups. This is the justification to extend the functionality of the stream processor by introducing the four patterns as additional API functions. These new API functions establish a direct connection with the graph to perform the required traversal to find the correct semantic values.

Fig. 16 depicts the architecture of a stream processor with integrated semantic transformation. As in the previous scenario, the sensor data is published to a topic of raw events. However, now the stream processor retrieves these events with a source process.

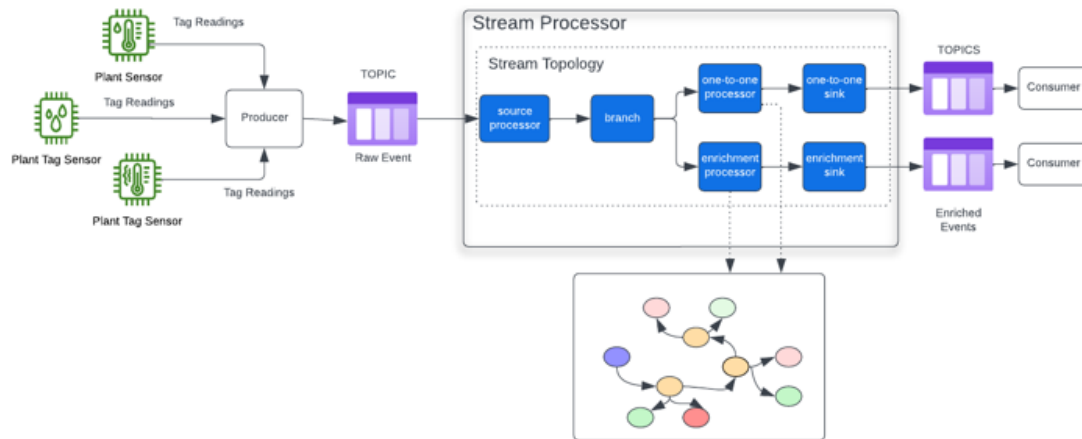


Figure 16. Event Hub with Streams.

A branch step determines which of the patterns to invoke. This could be determined based on the source topic or on other characteristics embedded in the event stream itself. The appropriate semantic pattern API is invoked, which initiates the traversal in the graph to find the correct translated results. After the semantic translation step, the stream processor invokes a sink step that publishes the new event with the translated data element to a new topic. Consumers monitor these topics, retrieve the events, and handle any requirements for downstream processes.

Compared to the pub/sub scenario, the integration to the graph in this scenario requires a tighter coupling with APIs rather than the invocation of microservices. The reason is the latency that is involved in communicating with the microservice. In this scenario, a microservice invocation would cause backpressure to develop which would have to be managed. Backpressure in streaming processes occurs when arriving events queue up from the source systems because of latency within the stream process [58].

Fig. 17 provides a demonstration of the described use case. The right side shows the streaming events as they are being sent by the different sensors, and their temperature readings. The events

are published as raw events to a topic. The left side shows the transformation from a tag ID to the associated plant ID which is added by creating a new event for further processing of downstream applications.

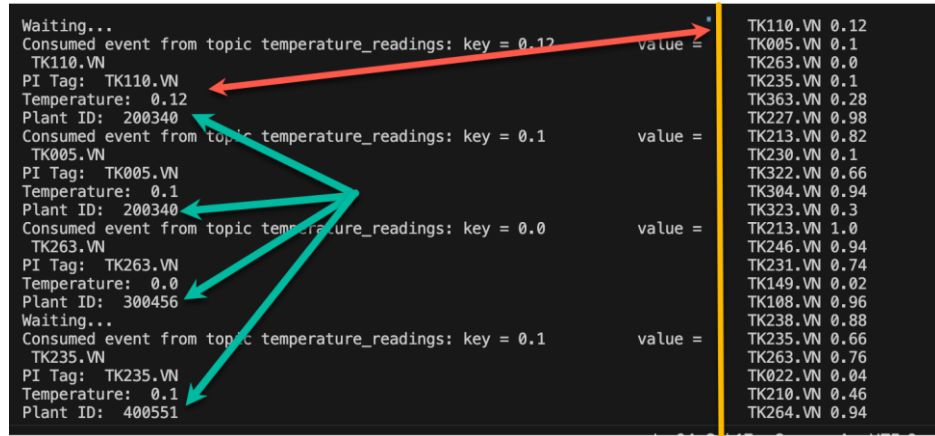


Figure 17. Sample Sensor Output from the Event Hub.

To note here is that multiple tag IDs may belong to one plant. Since the tag ID is preserved, a downstream dashboard can now show the temperature fluctuations within a plant with the capability to further drill down to the actual sensor. This case can be extended to include other nodes such as asset ID and zone ID.

6.4.4 Event Stream Architecture Comparison

The primary objective of this study is to demonstrate the integration of semantic transformation with event streams. The evaluation of the solution therefore focuses on the three primarily dynamic coupling forces [62] between the event streams, the semantic transformation patterns, and the graphs. These coupling forces consist of communication, coordination, and consistency. Communication describes the interaction between the event stream and the respective pattern to provide the right transformation. Coordination describes the interaction and collaboration between

the streaming services and the semantic transformation patterns. Consistency evaluates how well state is maintained throughout the semantic transformation process. Besides the three coupling forces, consideration should be given to scalability (both horizontal and vertical). Horizontal scalability describes the capability of adding new functionality to the integration. Vertical scalability considers the increase of processing instances to improve performance.

With the pub/sub scenario, a consumer acts as an orchestrator and communicates with the pattern microservice via REST communication. The communication is loosely coupled between the consumer and the respective microservice. The benefits of loose coupling are offset by the attempt to integrate asynchronous event streams with synchronous request and response patterns, which can cause performance issues. Another advantage of loose coupling is that added functionality via additional microservices can be easily integrated into the scenario. Consistency of events is maintained during the process via creating new events and publishing them to new topics. The coordination is managed through the consumer who has embedded logic for sequencing the execution of the various microservices. From a vertical scaling perspective, it's easy to increase the instantiation of each microservice to address communication performance issues.

With the event streaming scenario, a custom API must be developed to integrate the transformation graph with the stream process. The communication is tightly coupled with the graph to achieve high performance. Consistency is managed through internal event tables that the stream provides. The management of the API invocation is achieved through the design of the processing topology, which again requires a tight coupling approach. Scalability is more challenging since new translation functionality would require the development of a new API.

The semantic transformation process always occurs against a single event in the stream and therefore supports task and data execution independence [63]. Parallel processing of these transformations to increase throughput is viable because the same process can be executed on different event sources.

Both the pub/sub and the stream processing scenarios are viable architectural options. The decision of which one to use will largely depend on business objectives. If performance and throughput are the primary concerns, then stream processing is the right choice. One should consider that a custom API is required to support the semantic transformation patterns, which is not a trivial task to take on. However, if a looser coupling approach is desired to support the implementation of enhanced functionality, then the traditional pub/sub scenario would be a better alternative to address semantic transformation requirements. The advantage here is that the same microservice pattern can be used in other use cases such as data analytics pipelines and enterprise service busses; and therefore, provides a high degree of utilization compared to event stream processing APIs.

7 Conflict Resolution with Machine Learning Scenarios

A key characteristic of our semantic transformation is the adoption of reactive architecture [64]. This architecture style utilizes the concept of application behavior tracking. In traditional architecture, logs are used to track the state of a given object. The objective of behavior tracking is to record any changes to an object at any point in time. The advantage of this approach is that logged changes to an object can be used for analytical purposes. In our case, the traversals performed through the four patterns will be recorded in an event log that will be used for missing node predictions.

The example in Fig. 18 shows the traversal path for a successful node search and the same for an unsuccessful node search. For the unsuccessful node searches, an approach is required to predict the missing nodes. We will present two different methodologies to determine missing node predictions.

LogKey	Case	Process	Activity	ActivityInstance	UpdateDateTime	Position	TransType	Resource	ReqValue	Message
28	2249840Imp6909-926	LocTrnPit	Req	Location	locationCode	2020-10-25 19:28:05.027	Spiral	Dependency	Transport4	DUBL
29	2249840Imp6909-926	LocTrnPit	Req	Tankage	tankage	2020-10-25 19:28:05.030	Spiral	Dependency	Transport4	CIT
30	2249840Imp6909-926	LocTrnPit	Rsp	Plant	plantCode	2020-10-25 19:28:05.037	Spiral	Dependency	Transport4	600881
31	2249840Imp6909-1134	LocTrnPit	Req	Location	locationCode	2020-10-25 19:28:05.190	Spiral	Dependency	Transport4	DUBL
32	2249840Imp6909-1134	LocTrnPit	Req	Tankage	tankage	2020-10-25 19:28:05.190	Spiral	Dependency	Transport4	CIT
33	2249840Imp6909-1134	LocTrnPit	Rsp	Plant	plantCode	2020-10-25 19:28:05.200	Spiral	Dependency	Transport4	600881
34	2249840Imp6909-8069	LocTrnPit	Req	Location	locationCode	2020-10-25 19:28:05.377	Spiral	Dependency	Transport4	DUBL
35	2249840Imp6909-8069	LocTrnPit	Req	Tankage	tankage	2020-10-25 19:28:05.377	Spiral	Dependency	Transport4	CIT
36	2249840Imp6909-8069	LocTrnPit	Rsp	Plant	plantCode	2020-10-25 19:28:05.383	Spiral	Dependency	Transport4	600881
37	2249840Imp6909-3053	LocTrnPit	Req	Location	locationCode	2020-10-25 19:28:05.530	Spiral	Dependency	Transport4	DUBL
38	2249840Imp6909-3053	LocTrnPit	Req	Tankage	tankage	2020-10-25 19:28:05.530	Spiral	Dependency	Transport4	CIT
39	2249840Imp6909-3053	LocTrnPit	Rsp	Plant	plantCode	2020-10-25 19:28:05.540	Spiral	Dependency	Transport4	600881
40	2249840Imp6909-1625	LocTrnPit	Req	Location	locationCode	2020-10-25 19:28:05.693	Spiral	Dependency	Transport4	DUBL
41	2249840Imp6909-1625	LocTrnPit	Req	Tankage	tankage	2020-10-25 19:28:05.693	Spiral	Dependency	Transport4	CIT
42	2249840Imp6909-1625	LocTrnPit	Rsp	Plant	plantCode	2020-10-25 19:28:05.703	Spiral	Dependency	Transport4	600881
43	7934931Imp8251-196	LocTrnPit	Req	Location	locationCode	2020-10-25 19:28:05.867	Spiral	Dependency	Transport4	PATK
44	7934931Imp8251-196	LocTrnPit	Req	Tankage	tankage	2020-10-25 19:28:05.867	Spiral	Dependency	Transport4	MTA
45	7934931Imp8251-196	LocTrnPit	Rsp	Plant	plantCode	2020-10-25 19:28:05.880	Spiral	Dependency	Transport4	***plantCode For locationCode PATK Not Found
46	6375188Exp5044-8768	LocTrnPit	Req	Location	locationCode	2020-10-25 19:28:06.040	Spiral	Dependency	Transport4	LI07
47	6375188Exp5044-8768	LocTrnPit	Req	Tankage	tankage	2020-10-25 19:28:06.040	Spiral	Dependency	Transport4	HUS
48	6375188Exp5044-8768	LocTrnPit	Rsp	Plant	plantCode	2020-10-25 19:28:06.050	Spiral	Dependency	Transport4	400551
49	6375195Imp7219-9847	LocTrnPit	Req	Location	locationCode	2020-10-25 19:28:06.133	Spiral	Dependency	Transport4	AKRO

Figure 18. Event Log.

7.1 Classical Machine Learning Approach

The previously presented event log is the historical data set and provides the feature set for the classification algorithms. Each traversed node in the log represents a feature. Since for classifications the feature set needs to be consistent, identical node traversals are grouped together as a feature set, and predictions can only be made for that distinct feature pattern. In the example, we used a grouping within the dependency pattern that provided a feature set of four nodes. Three classification algorithms were applied with the results listed in Table 3.

Table 3. Machine Learning Approaches Cross Validation.

	LogReg	SVM	Knn
Cross Validation	94.8333%	94.2083%	88.9167%

Each result set was then compared (shown in Table 4) to determine the similarity of the predictions between the three methods presented above.

Table 4. Machine Learning Approaches Similarity Ratio.

	Similarity Ratio
LogReg – SVM	85.54%
LogReg – Knn	87.95%
SVM – Knn	84.33%

Even though the cross-validation scores are relatively high, the classification methods present significant drawbacks since the feature set must be consistent for the traversal scenario being examined. This adds significant overhead in selecting and grouping the traversal scenarios. Comparing the similarity of the result sets of the three methods shows that LogReg to Knn has the highest similarity rate, but Knn itself has the lowest cross-validation ratio. LogReg has the highest cross-validation ratio and should therefore be considered the algorithm of choice.

One significant problem with the classification method is that it is based on historical feature occurrences and does not consider a blank node in its community. Therefore, predictions are

possible of nodes that are outside the direct community for which a replacement value is being determined; and it is thus possible to get predictions of totally unrelated nodes in graph areas that are also unrelated to the current node location. This error is not reflected in the cross-validation score and contributes the overall wrong predictions. Therefore, a community detection approach within a graph is more suitable for node predictions in semantic transformations.

7.2 Node Predictions using Graph Methods

The use of graph algorithms requires a homogenous graph that is derived from the event log. Each log entry represents a node. The value for the node can be considered as a word, where the edge between two nodes becomes the rules on building sentences. Taking the node values as words across all nodes makes each node the same with a different value. This allows for the generation of a homogenous graph from the event log. In cases where a result node was not defined (error case), a blank node is created as an intermittent place holder.

The task now (shown in Fig. 19) is to eliminate these blank nodes within the graph and find suitable replacement nodes.

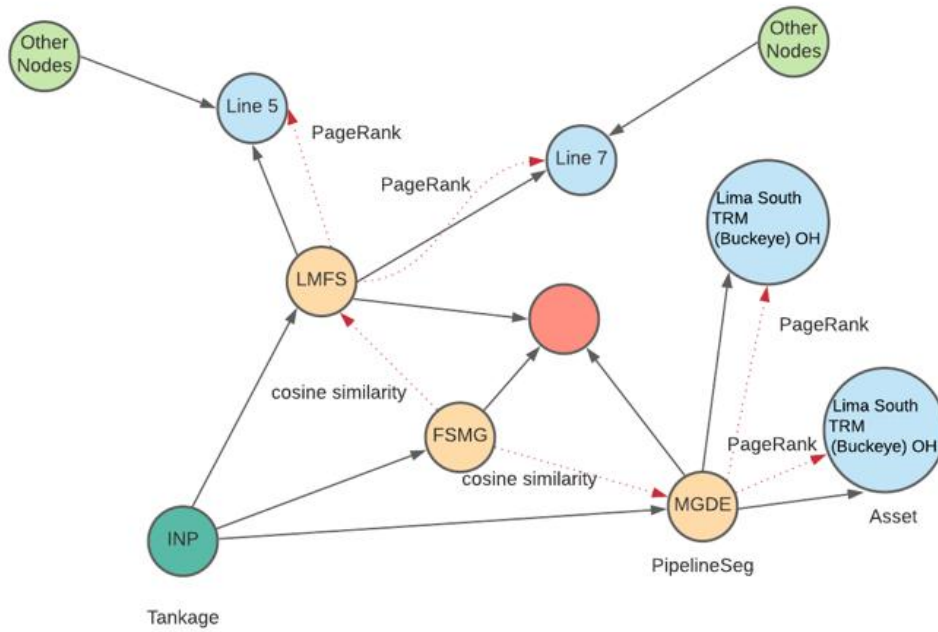


Figure 19. Missing Node Resolution.

There are two scenarios that need to be considered:

1. A parent node has multiple child nodes, of which one is a blank node.
2. A parent node has one child node, which is a blank node.

To address the first scenario we consider the PageRank algorithm. The PageRank algorithm [65] considers the ranking of the node within the community based on the number of incoming relationships and the importance of the corresponding source nodes. The underlying assumption is that a node is only as important as the nodes that link to it.

$$PR(A) = (1 - d) + d\left(\frac{PR(T1)}{C(T1)} + \dots + \frac{PR(Tn)}{C(Tn)}\right)$$

Where:

- We assume that node A has nodes T1 and Tn pointing to it.

- d is a damping factor which can be set between 0 (exclusive) and 1 (inclusive). It is usually set to 0.85.
- $C(A)$ is the number of links going out of node A .

Based on the PageRank value of all the child nodes, the node with the highest PageRank value will be chosen as a replacement node.

In the second scenario (shown in Fig. 20) a parent has only one child node, which is a blank node. In this case our process retracts to the grandparent node of the blank node, for which the parent node of the blank nodes is a child with sibling nodes. We determine which of the sibling nodes is the most like the node for which a replacement is required.

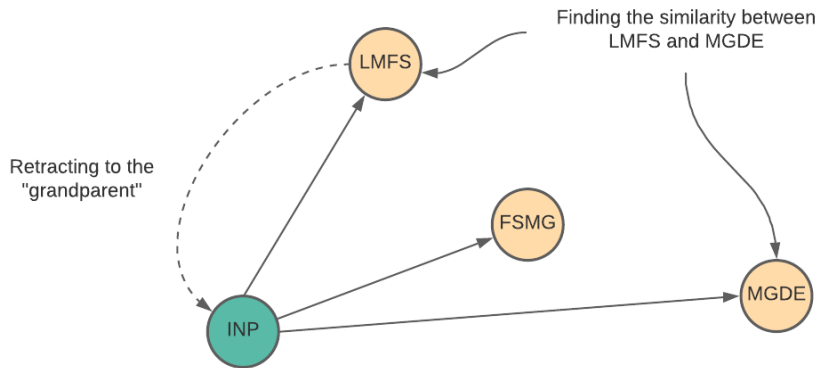


Figure 20. Missing Node Retraction.

To accomplish this, we use node embeddings to transform the graph into a simpler vector-based representation. In our second scenario we use the Node2Vec algorithm [66] to achieve this node embedding. Next, we apply the cosine similarity function [16] among the child nodes to determine the node with the highest similarity.

$$similarity(A,B) = \frac{A \cdot B}{\|A\| \times \|B\|} = \frac{\sum_{i=1}^n A_i \times B_i}{\sqrt{\sum_{i=1}^n A_i^2} \times \sqrt{\sum_{i=1}^n B_i^2}}$$

Once that node is determined, the process returns to the first scenario and applies a PageRank on the child nodes of the newfound parent. The node with the highest page rank will then provide the replacement value for the blank node.

7.3 Evaluation and Comparison

The following (Table 5) provides a comparison between the classification method and the graph method.

Table 5. ML and Graph Algorithm Comparison.

CLASSIFICATION METHOD	GRAPH METHOD
• Code predictions are based on history data. • Code predictions do not reflect code dependencies. • Amount of historical data will increase over time and process will slow down. • Not suited for real-time assessments.	• Node relationships within the network drive predictions. • The neighborhood of the node will influence the prediction of nodes. • Only immediate neighbors are considered for node predictions. • Real-time predictions are possible.

The graph algorithmic approach that we presented represents a viable option for performing semantic transformations; but the solution will still need additional functionality to become a viable production version. One feature that would support this is an easy-to-manage node and relationship management tool that allows users to adjust and confirm predictions. A second consideration is that event log entries occur continuously, and the graph would need to have the ability to update itself every time a new log entry was made. This would also require a recalculation

of the node-embedding algorithm to derive the latest node-embedding values. Solutions for these shortcomings are part of future research. In addition, the base graph would need to be updated with the new node predictions to ensure that missing nodes in traversals are constantly eliminated.

8 Realization and Value Contribution

After a detailed technical discussion on semantic transformation, an assessment of value contribution to an organization is presented. The cost of integration is a significant part of an overall IT budget. Vuković et al. [10] in 2017 reference a Forrester Research study of Fortune 1000 companies where 35% of their software maintenance budget was spent on application integration. A more recent survey by Digibee [67] from 2023 states that 71% of the surveyed 1000 companies are planning to migrate to an iPaaS environment in 2023. The survey also states that 16% to 25% of the IT budget is spent on integration. Both references from different years demonstrate that integration costs are still a significant portion of any company's IT budget. Therefore, solutions such as are presented in this study can reduce integration complexity as well as increase functionality; and therefore, be a critical contributor to reducing funding requirements for integration solutions for IT organizations.

Since application integration is a cost, benefits must be measured in reduction of cost. In Fig. 21 the top graph shows the number of transaction errors due to missing search values without a recommender system. If search values cannot be found, the transaction transformation will fail and therefore be considered a transaction error. If a transaction fails it requires a manual intervention to resolve the issue so that the transaction can be reprocessed. This can take between hours and days. The bottom graph shows the number of barrels of oil that thus cannot be shipped and are waiting for reprocessing. The following example demonstrates a scenario of cost for failed transactions:

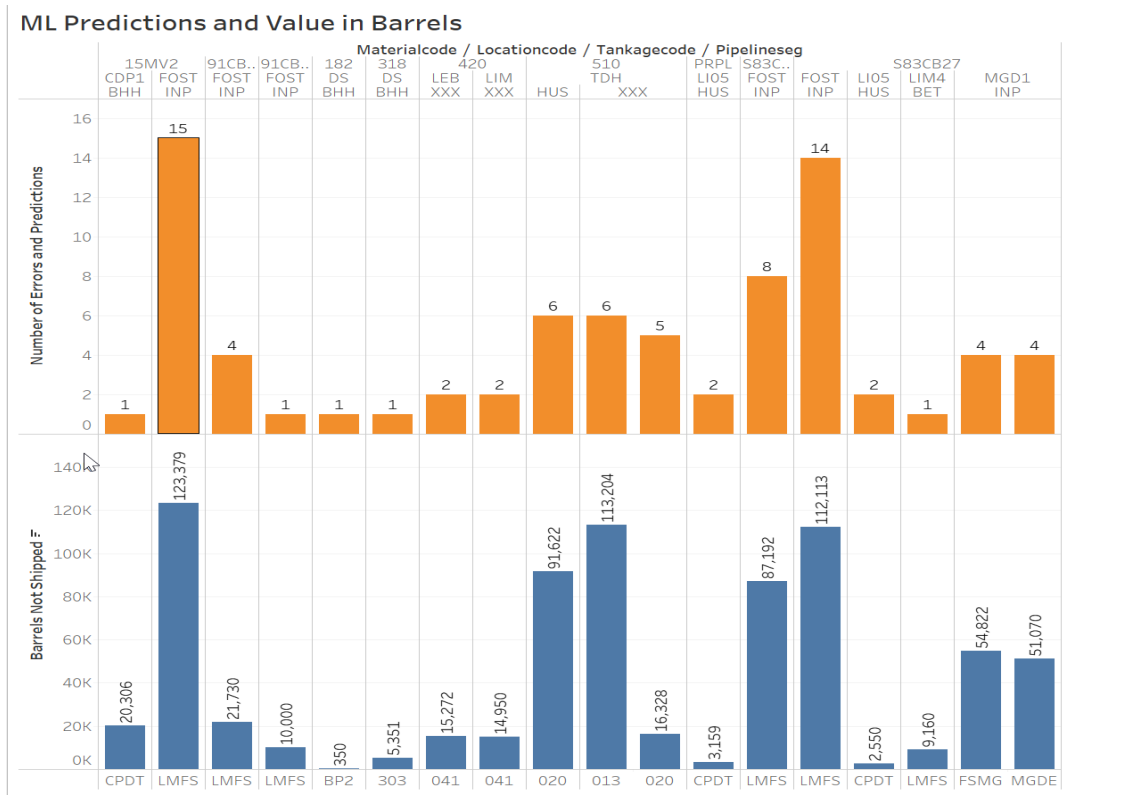


Figure 21. Error Messages with Associated Barrels of Oil Not Shipped.

- Based on the graphs, the total number of barrels for all transactions is 752,599 BR
- Assuming a price per barrel of \$50.00
- Assuming a capital cost of 5%
- Assuming a reprocessing time of 3 days
- These failed transaction amount to \$15,463 $[(752599 * \$50 * 0.05)/(365*3)]$
- Assuming 30 days in a month $(\$15,462 * 10) = \$154,640$

Therefore, the cost of these failed transactions is \$15,462. If one could reduce the reprocessing time by two days by using the semantic transformation framework, the savings would be \$10,309. Considering that this example only represents one integration, it should be evident that the cost

savings are significant when the presented technology is applied to several integration scenarios within an organization.

Semantic transformation can therefore have a significant economic impact on an organization, which in many cases is not measured. Streamlining this process with the framework presented in this paper will speed up the overall transaction process and contribute significantly to the bottom line.

9 Conclusion

9.1 Solution Comparison

The research presented in this thesis evaluated the complexities and difficulties involved with semantic transformation in an enterprise setting. It introduced and tested a new and innovative approach by identifying and applying a set of semantic transformation patterns. These patterns were then used to traverse a graph structure to find the correct transformation value. The graph structure provided new opportunities in capturing node traversal in an event log. This event log became the basis for the prediction of missing nodes in the graph by applying graph embeddings.

A microservice architecture was used to demonstrate how various request message types could be processed through the semantic transformation engine. Two scenarios were presented. The first applied an orchestrated approach, and the second used NLP techniques to search for the transformed code value.

The developed architecture was then integrated into event streams, since the popularity of using event streams has increased and semantic transformations are increasingly required to develop analytics and predictions on sensor data.

In the literature review we presented several research projects and provided a comparison on their major characteristics. The following is a comparison, summarized in Table 6, between the semantic enterprise transformation framework presented and the solutions discussed in the literature review. The comparison is not conducted on each of the nine presented solutions, but rather in summary. Each of the evaluation criteria are discussed with differences to the presented solution.

Service Type

Most solutions presented in the literature review are web service based with the specific standard web service semantic extension of WSMO/WSML. The primary reason for the use of web services is their extensions to ontologies. However, web services require a large footprint both from a communication and message structure standpoint. In comparison, the semantic enterprise transformation framework presented uses microservices. Microservices are small independent services that run as containers, which allow for flexible deployments. In addition, microservices are easy scalable compared to web services; and therefore, the architecture of semantic enterprise transformation framework is nimbler compared to any web services-based solution.

Ontology

Most of the semantic transformation solutions in the literature review require the development of an ontology prior to use of the integration. The web service extension requires that either a local, domain, or global ontology exists for the semantic transformation to work. The development of ontologies is expensive and time consuming. The enterprise transformation framework takes a different approach. It requires the definition of terms and their relationships for a single integration scenario as a graph. The development of an ontology is initially not required; therefore, the presented solution can be immediately utilized by an organization without having to first conduct a lengthy ontology development exercise. The semantic transformation approach is built bottom up by developing the relationships for an integration first, from which an ontology can later be derived. In the presented use case, the relationships for the pipeline shipping process were developed first, followed by the plant sensor data. This approach shows the agility of graph

structures in starting from a limited scope and continuously extending, compared to a top-down approach where an ontology has to first be defined so that the whole integration can be utilized.

Functional Extension

Any solution will eventually require some type of functional extension, if only for the fact that different data structures can present new processing requirements. The enterprise semantic transformation solution addresses this through the application of microservices. Each transformation pattern discussed is implemented as a microservice. If additional functional extensions are required, the existing microservices are not impacted since new microservices can be developed and integrated into the service mesh to be managed by the cluster. Through its more rigid implementation approach by using WSDL, SOAP, and HTML and their semantic extensions, web services are tightly coupled, and any changes to their logic will require significant testing. Therefore, the microservice architecture and its loose coupling presents a benefit over the use of web services.

Data Structures

Since most of the projects presented in the literature review are based on web services that use SOAP and WSDL, it requires that the message structure be XML. The presented solution, however, uses JSON. There are several benefits to using JSON over XML. Firstly, JSON is a simpler and more lightweight structure than XML. XML requires extensive tagging (opening and closing tags) to describe data elements, whereas in JSON, data elements are presented as key/value pairs. JSON is widely supported in modern languages and APIs and thus provides higher

interoperability than XML; it is also a better choice for loose coupling of applications. In addition, JSON requires lower overhead than XML in parsing and computing of messages.

Table 6. Solution Comparison.

Category	Literature Review Proposals	Proposed Solution
Service Type	Web service	Microservice
Ontology	OWL/DAML	RDF/OWL
Functional Extension	Through new web services	Through new microservices
Data Structures	XML/XSD	JSON
Data Domain	Specific	Universal
Connection to Existing Integration Platforms	Standalone	REST call
Communication	SOAP	REST
Transformation	Syntactic and semantic	Semantic
Presentation	OWL tools	Graph
Mediation	Ontology interpretation	Graph traversals
Mediation Resolution	Approximation	Graph algorithms
Dynamic Update	Only manual ontology updates	Automatic through community detection

Data Domain

The data domain criteria describe which specific data transaction can be used for the semantic transformation. Most of the projects discussed use specific transaction types (e.g., ROSETTANET). This requires that the integration be adjusted to handle different transaction formats. The semantic enterprise transformation framework uses a generic interface object. The interface object allows for different search terms regardless of transaction type as long as the search terms are defined in the graph. This interface object therefore presents an abstraction between the microservices and a calling client.

Connection to Existing Integration Platforms

Not one of the solutions presented in the literature review section addresses the interoperability with existing integration platforms. This is a critical aspect since most organizations already have one or more solutions in production that will handle syntactic transformation of messages. The semantic enterprise transformation framework is designed to coexist with existing integration solutions. Through its REST API it seamlessly integrates semantic transformation functionality into an existing enterprise application integration ecosystem without the need for adopting new integration solutions or building custom add-ons.

Communication

As a communication protocol, the semantic enterprise transformation framework uses REST protocol versus SOAP. REST uses the four methods (GET, POST, PUT, and DELETE) embedded in HTML to communicate; whereas SOAP is its own standard, and a SOAP message is encapsulated into the HTML body. REST is therefore more lightweight and is supported by many APIs.

Transformation

All projects discussed in the literature review require syntactic transformation in addition to semantic transformation. The semantic enterprise transformation framework is only concerned with semantic transformation since the syntactic transformation is being handled by existing integration platforms. The invocation for a semantic transformation request happens via a REST service call in providing a request object as a JSON structure. This reduces the scope and complexity of the framework compared to solutions presented in the literature review.

Presentation

The presentation category assesses how semantic data is being displayed. Many of the discussed projects use custom-built Windows applications that are used to present semantic context and relationships. Since the semantic enterprise transformation framework is based on graph structure, all relationships are presented as a graph. Given this graph structure it is easy to trace the relationships of entities by traversing from node to node.

Mediation

The mediation category addresses how a semantic transformation finds the requested outcome of the transformation. Most projects require some reasoner to determine a transformation result based on the actual semantic query. The accuracy of the result is based on the capability of the reasoner. In the solution presented, the mediation is performed through graph traversals which are driven by the respective transformation patterns. The outcome is that for any combination of transformation values, a resulting value must exist, otherwise a mediation resolution is required.

Mediation Resolution

In case of missing values, the result can only be predicted. In the literature review projects this is performed through the ontology reasoner; whereas the semantic enterprise transformation framework uses graph algorithms for predictions.

Dynamic Updates

When ontologies are the primary building blocks for semantic transformations, it is important that they are continuously updated to ensure they stay current. This will mostly happen through a manual process. However, in the presented solution, missing values are predicted through graph algorithms and the predicted result can be updated back to the graph in real time.

9.2 Future Research

In conclusion, some future research opportunities are identified which would further contribute to achieving either new architecture approaches or solutions that refine and provide better results.

With the emergence of Large Language Models (LLM), the question arises of how semantic transformation in the context described in this paper would contribute value. LLM are powerful tools in delivering responses to queries, largely due to the large text corpora that have been used to train these models. A combination of a custom LLM with a public LLM would need further investigation. To incorporate LLMs would be a change to the architecture, and the graph structure would be an input to the model. Based on Raschka's presentation [68], an LLM architecture could be presented as described in Fig. 22. The graph nodes and the meaning of the edges would produce statements as input. For example, Material Code 2343 requires Location Code 23 with Tankage 51 and Pipeline Segment 33 to get the Asset Code 54. Every permutation and combination of nodes and edges would have to be extracted from the graph in such a statement format. This set of statements would then provide the statement base for tokenization. Once the set of statements have been tokenized, one could use OpenAI embeddings to load the resulting vectors into a vector database. This vector dataset would be the training model.

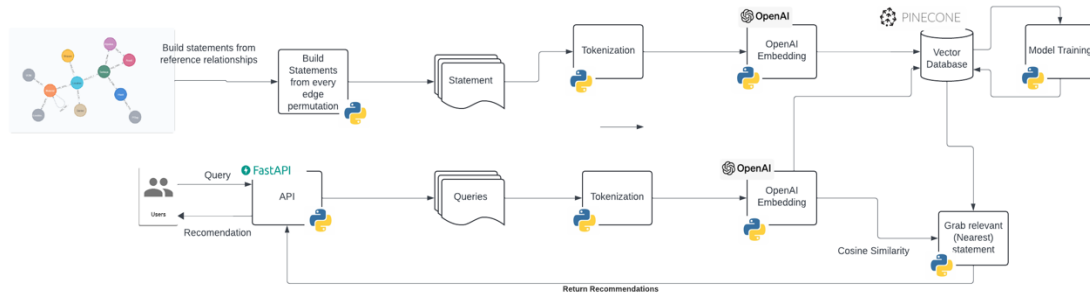


Figure 22. LLM Architecture.

From a query perspective, the same approach that was presented in this research could be used. The user or any other application would make a REST API call and provide a query. This query would be tokenized and use OpenAI embeddings to be stored in the vector database. Using cosine similarity algorithm, for example, the closest matching result would be extracted and returned to the query initiator. In a transaction processing scenario where the transformation of a certain code expects the exact result, the question would be, could such an LLM model deliver such accuracy or would it be an approximate prediction? The exact match for existing code results is a requirement not only from a business perspective but also from a legal standpoint.

In the presented solution, if a node exists, the result is always the exact match based on the definition within the graph structure. However, approximations occur when there are missing nodes, in which case the described mediation strategy would be executed. Building the architecture (Fig. 22) and running sample tests and comparing them to the solution presented in this paper would definitely be of value.

Other future research opportunities would be the development of a full cloud-based testing environment to further test semantic transformation scenarios. This would require close collaboration with various companies to ensure that the solution meets their needs from a performance, reliability, and scalability perspective. By applying and developing various types of

fitness functions, better insight could be gained in graph behavior and performance given live integration scenarios.

Once this test environment exists, further research could be conducted to extend capabilities of missing code predictions using various potential AI approaches. However, to be able to do that, much larger data sets would be required than were used in this research. Such data sets are only available through collaboration with business partners that are willing to sponsor such a project.

Finally, the unique requirements of the NLP processes could be further investigated and extended. One area would be the automatic generation of the dictionary based on previous queries so that syllables and phrases can be stored to enhance the parsing algorithms and develop a higher query accuracy. In addition, the language capabilities could be extended to include nested phrases, comparisons, and potentially filters.

In summary, the semantic enterprise transformation framework provides an architecture that is characterized by its flexibility in adapting to changing requirements, its use of a bottom-up approach to expand to full knowledge graphs without the need for large investments, and its ability to provide immediate cost savings for organizations.

References

- [1] N. Banerjee, A. Chordia, and P. Rajib, “Seamless Enterprise Computing Using Enterprise Application Integration (EAI),” *Journal of Services Research*, vol. 5, no. 1, 2005, pp. 171–196.
- [2] Online Browsing Platform (OBP): Information technology – Vocabulary. Interoperability. Website. <https://www.iso.org/obp/ui/en/#iso:std:iso-iec:2382:ed-1:v2:en::~text=2121317-interoperability,-%3Cfundamental%20terms%3E%20capability>.
- [3] V. Bush, “As We May Think,” *Atlantic Monthly*, vol. 176, July 1945, pp. 101–108.
- [4] G. Wiederhold, “Mediators in the Architecture of Future Information Systems,” *Computer*, vol. 25, no. 3, Mar. 1992, pp. 38–49, doi:10.1109/2.121508.
- [5] C. H. Goh, “Representing and Reasoning about Semantic Conflicts in Heterogeneous Information Systems,” Ph.D. thesis, MIT, 1997.
- [6] D. S. Linthicum, *Enterprise Application Integration*. Addison-Wesley Professional, 2000.
- [7] D. Trastour, C. Preist, and D. Coleman, “Using Semantic Web Technology to Enhance Current Business-to-Business Integration Approaches,” *Proc. Seventh IEEE International Enterprise Distributed Object Computing Conference (EDOC)*, IEEE, Sep. 2003, pp. 222–230, <https://doi.org/10.1109/EDOC.2003.1233851>.
- [8] S. Izza, L. Vincent, and P. Burlat, “A Framework for Semantic Enterprise Integration,” in *Interoperability of Enterprise Software and Applications*, N. Ivezic, A. Jones, D. Konstantas, J.P. Bourrières, M. Léonard, and N. Boudjlida, Eds. London: Springer, 2006, pp. 75–86, https://doi.org/10.1007/1-84628-152-0_8.
- [9] T. Bouras, P. Gouvas, and G. Mentzas, “Dynamic Data Mediation in Enterprise Application Integration,” in *Collaboration and the Knowledge Economy: Issues, Applications, Case Studies*, P. Cunningham and M. Cunningham, Eds. IOS Press, 2009, pp. 917–924.
- [10] Ž. Vuković, N. Milanović, R. Vadera, I. Dejanović, G. Milosavljević, and V. Malbaša, “Semantic-Aided Automation of Interface Mapping in Enterprise Integration with Conflict Detection,” *Information Systems and e-Business Management [Online]*, vol. 15, 2016, pp. 305–22, <https://doi.org/10.1007/s10257-016-0326-7>.
- [11] N. Ebert, K. Weber, and S. Koruna, “Integration Platform as a Service,” *Business & Information Systems Engineering*, vol. 59, 2017, pp. 375–379, <https://doi.org/10.1007/s12599-017-0486-0>.
- [12] Apache Spark. Website. <https://spark.apache.org>.
- [13] Apache Kafka. Website. <https://kafka.apache.org>.

- [14] P. Muschamp, “An introduction to Web Services,” *BT Technology Journal*, vol. 22, 2004, pp. 9–18, <https://doi.org/10.1023/B:BTTJ.0000015537.71918.c2>.
- [15] G. Alonso, F. Casati, H. Kuno, and V. Machiraju, “Web Services,” in *Web Services*. Springer Berlin, Heidelberg, 2004, pp. 123–149.
- [16] F. Curbera, M. Duftler, R. Khalaf, W. Nagy, N. Mukhi, and S. Weerawarana, “Unraveling the Web services web: an introduction to SOAP, WSDL, and UDDI,” *IEEE Internet Computing*, vol. 6, 2002, pp. 86–93.
- [17] R. T. Fielding, “Architectural Styles and the Design of Network-based Software Architectures,” Ph.D. thesis, University of California, Irvine, 2000.
- [18] M. Garriga, C. Mateos, A. Flores, A. Cechich, and A. Zunino, “RESTful service composition at a glance: A survey,” *Journal of Network and Computer Applications*, vol. 60, 2016, pp. 32–53.
- [19] T. R. Gruber, “Toward principles for the design of ontologies used for knowledge sharing?,” *International Journal of Human-Computer Studies*, vol. 43, 1995, pp. 907–928, <https://doi.org/10.1006/ijhc.1995.1081>.
- [20] J. de Bruijn, C. Bussler, J. Domingue, D. Fensel, M. Hepp, U. Keller, M. Kifer, B. König-Ries, J. Kopecky, R. Lara, H. Lausen, E. Oren, A. Polleres, D. Roman, J. Scicluna, and M. Stollberg, “Web Service Modeling Ontology (WSMO),” Eds. H. Lausen, A. Polleres, and D. Roman, W3C Member Submission, 2005, <http://23.105.212.211/submissions/WSMO/>.
- [21] A. Haller, J. M. Gómez Berbis, and C. Bussler, “Exposing Semantic Web Service principles in SOA to solve EAI scenarios,” [Online], 2005.
- [22] J. de Bruijn, D. Fensel, U. Keller, M. Kifer, H. Lausen, R. Krummenacher, A. Polleres, and L. Predoiu, “Web Service Modeling Language (WSML),” eds J. de Bruijn and H. Lausen, W3C Member Submission, 2005, <https://www.w3.org/submissions/WSML/>.
- [23] A. Haller, E. Cimpian, A. Mocan, E. Oren, and C. Bussler, “WSMX – a semantic service-oriented architecture,” *Proc. IEEE International Conference on Web Services (ICWS’05)*, IEEE, July 2005, pp. 321–328, <https://doi.org/10.1109/ICWS.2005.139>.
- [24] D. Allemang and J. Hendler, *Semantic Web for the Working Ontologist: Effective Modeling in RDFS and OWL*, 1st ed. Amsterdam: Morgan Kauffman, 2008.
- [25] W3C Semantic Web Standards: Resource Description Framework (RDF). Website. <https://www.w3.org/RDF/>.
- [26] W3C: RDF Schema 1.1. Website. <https://www.w3.org/TR/rdf-schema/>.

- [27] L. Yu, Introduction to the Semantic Web and Semantic Web Services. Chapman and Hall/CRC, 2007.
- [28] W3C Semantic Web Standards: Web Ontology Language (OWL). Website. <https://www.w3.org/OWL/>.
- [29] S. Izza, L. Vincent, and P. Burlat, "Ontology urbanization for semantic integration: dealing with semantics within large and dynamic enterprises," Proc. Ninth IEEE International EDOC Enterprise Computing Conference (EDOC'05), IEEE, Sep. 2005, pp. 83–94, doi:10.1109/EDOC.2005.27.
- [30] S. Izza, L. Vincent, P. Burlat, and P. Lebrun, "An Ontology-Driven Data Mediation Framework for Enterprise Application Integration," [Online], 2006.
- [31] V. Roberto, M. Rowlatt, R. Davies, A. Gugliotta, L. Cabral, and J. Domingue, "A semantic web service-based architecture for the interoperability of e-Government services," [Online], 2005.
- [32] A. Leben and M. Bohanec, "Architecture of an Active Life-Event Portal: A Knowledge-Based Approach," in Knowledge Management in Electronic Government: 5th IFIP International Working Conference, KMGov 2004, Krems, Austria, May 2004, Proceedings, M. A. Wimmer, Ed. Springer, 2004, pp. 147–156.
- [33] Ž. Vuković, N. Milanović, R. Vadera, I. Dejanović, and G. Milosavljević, "SAIL: A Domain-Specific Language for Semantic-Aided Automation of Interface Mapping in Enterprise Integration," in On the Move to Meaningful Internet Systems: OTM 2015 Workshops: Confederated International Workshops: OTM Academy, OTM Industry Case Studies Program, EI2N, FBM, INBAST, ISDE, META4eS, and MSC 2015 Rhodes Greece, October 26–30, 2015, Proceedings, I. Ciuciu, H. Panetto, C. Debruyne, A. Aubry, P. Bollen, R. Valencia-García, A. Mishra, A. Fensel, and F. Ferri, Eds. Springer, 2015, pp. 97–106.
- [34] N. Anicic, N. Ivezic, and A. Jones, "An Architecture for Semantic Enterprise Application Integration Standards," in Interoperability of Enterprise Software and Applications, D. Konstantas, J.P. Bourrières, M. Léonard, N. Boudjlida, Eds. Springer, 2006, pp. 25–34, https://doi.org/10.1007/1-84628-152-0_3.
- [35] K. Shi, F. Gao, Q. Xu, and G. Xu, "Integration framework with semantic aspect of heterogeneous system based on ontology and ESB," Proc. The 26th Chinese Control and Decision Conference (2014 CCDC), IEEE, 2014, pp. 4143–4148, <https://doi.org/10.1109/CCDC.2014.6852907>.
- [36] K. Mecheri and L. Souici-Meslati, "Semantic interoperability of Web services: A survey," Proc. 2010 International Conference on Machine and Web Intelligence (ICMWI), IEEE, Oct. 3–5, 2010, pp. 82–87, <https://doi.org/10.1109/ICMWI.2010.5647928>.

- [37] J. F. Nunamaker Jr., M. Chen, and T. D. M. Purdin, "Systems Development in Information Systems Research," *Journal of Management Information Systems*, vol. 7, 1990, pp. 89–106.
- [38] J. R. Venable, J. Pries-Heje, and R. Baskerville, "Choosing a Design Science Research Methodology," *Proc. ACIS 28th Australasian Conference on Information Systems (ACIS2017)*, AISel, Dec. 2017, pp. 1–11, <https://aisel.aisnet.org/acis2017/112>.
- [39] C. Baham and R. Hirschheim, "Issues, challenges, and a proposed theoretical core of agile software development research," *Information Systems Journal*, vol. 32, Jan. 2022, pp. 103–129, doi.org/10.1111/isj.12336.
- [40] R. Knaster and D. Leffingwell, *SAFe Distilled: Applying the Scaled Agile Framework for Lean Enterprises*. Boston, MA: Addison-Wesley, 2019.
- [41] PIDX International: Standards. Website. <https://pidx.org/standards/>.
- [42] ByteByteGo Newsletter: API Gateway. Website. https://blog.bytebytego.com/p/api-gateway?utm_source=substack&publication_id=817132&post_id=149640765&utm_medium=email&utm_content=share&utm_campaign=email-share&triggerShare=true&isFreemail=false&r=24uc4r&triedRedirect=true
- [43] P. Suresh, J. Vijay Daniel, V. Parthasarathy, and R. H. Aswathy, "A state of the art review on the Internet of Things (IoT) history, technology and fields of deployment," *Proc. 2014 International Conference on Science Engineering and Management Research (ICSEMR)*, IEEE, Nov. 27–29, 2014, pp. 1–8, <https://doi.org/10.1109/ICSEMR.2014.7043637>.
- [44] J. Lewis and M. Fowler, "Microservices: a definition of this new architectural term." Website. <https://www.martinfowler.com/articles/microservices.html>.
- [45] N. Dragoni, S. Giallorenzo, A. L. Lafuente, M. Mazzara, F. Montesi, R. Mustafin, and L. Safina, "Microservices: Yesterday, Today, and Tomorrow," in *Present and Ulterior Software Engineering*, M. Mazzara and B. Meyer, Eds. Springer Cham, 2017, pp. 195–216.
- [46] T. Engel, M. Langermeier, B. Bauer, and A. Hofmann, "Evaluation of Microservice Architectures: A Metric and Tool-Based Approach," in *Information Systems in the Big Data Era: CAiSE Forum 2018*, Tallinn, Estonia, June 11–15, 2018, *Proceedings*, J. Mendling and H. Mouratidis, Eds. Springer Cham, 2018, pp. 74–89.
- [47] I. Karabey Aksakalli, T. Çelik, A. Burak Can, and B. Tekinerdoğan, "Deployment and communication patterns in microservice architectures: A systematic literature review," *Journal of Systems and Software*, vol. 180, Oct. 2021, pp. 1–25, [doi:10.1016/j.jss.2021.111014](https://doi.org/10.1016/j.jss.2021.111014).
- [48] A. Megargel, C. M. Poskitt, and V. Shankararaman, "Microservices Orchestration vs. Choreography: A Decision Framework," *Proc. 2021 IEEE 25th International Enterprise Distributed Object Computing Conference (EDOC)*, IEEE, Oct. 2021, pp. 134–141, [doi:10.1109/EDOC52215.2021.00024](https://doi.org/10.1109/EDOC52215.2021.00024).

- [49] M. Richards and N. Ford, *Fundamentals of Software Architecture: An Engineering Approach*. Sebastopol, CA: O'Reilly, 2020.
- [50] N. Ford, M. Richards, P. Sadalage, and Z. Dehghani, *Software Architecture: The Hard Parts: Modern Trade-Off Analyses for Distributed Architectures*. O'Reilly, 2021.
- [51] C. T. Joseph and K. Chandrasekaran, "Straddling the crevasse: A review of microservice software architecture foundations and recent advancements," *Software: Practice and Experience*, vol. 49, July 2019, pp. 1448–1484, doi:10.1002/spe.2729.
- [52] L. Crispin, "Driving Software Quality: How Test-Driven Development Impacts Software Quality," *IEEE Software*, vol. 23, 2006, pp. 70–71, doi: 10.1109/MS.2006.157.
- [53] K. Beck, *Test-Driven Development: By Example*, 1st ed. Addison-Wesley, 2002.
- [54] C. Daase, D. Staegemann, M. Volk, and K. Turowski, "Creation of a Framework and a Corresponding Tool Enabling the Test-Driven Development of Microservices," *Proc. 2022 2nd International Conference on Advanced Enterprise Information System (AEIS 2022)*, Dec. 2022.
- [55] K. Sengloiluean, N. Arch-int, S. Arch-int, and T. Thongkrau, "A semantic approach for question answering using DBpedia and WordNet," *Proc. 2017 14th International Joint Conference on Computer Science and Software Engineering (JCSSE)*, IEEE, July 2017, pp.1–6, doi:10.1109/JCSSE.2017.8025918.
- [56] P. Gupta, A. Goswami, S. Koul, and K. Sartape, "IQS-intelligent querying system using natural language processing," *Proc. 2017 International conference of Electronics, Communication and Aerospace Technology (ICECA)*, IEEE, April 2017, pp. 410–413, doi:10.1109/ICECA.2017.8212846.
- [57] A. A. Maksutov, V. I. Zamyatovskiy, V. N. Vyunnikov, and A. V. Kutuzov, "Knowledge Base Collecting Using Natural Language Processing Algorithms," *Proc. 2020 IEEE Conference of Russian Young Researchers in Electrical and Electronic Engineering (EIconRus)*, IEEE, Jan. 2020, pp. 405–407, doi: 10.1109/EIconRus49466.2020.9039303.
- [58] G. Shapira, T. Palino, R. Sivaram, and K. Petty, *Kafka: The Definitive Guide: Real-Time Data and Stream Processing at Scale*, 2nd ed. Sebastopol, CA: O'Reilly, 2021.
- [59] J. Sheehy, "There is No Now: Problems with simultaneity in distributed systems," *Queue*, vol. 13, Mar. 2015, pp. 20–27, doi:10.1145/2742694.2745385.
- [60] M. Kleppmann, *Designing Data-Intensive Applications: The Big Ideas Behind Reliable, Scalable, and Maintainable Systems*. Sebastopol, CA: O'Reilly, 2017.

- [61] W. P. Bejeck, Jr., *Kafka Streams in Action: Real-time apps and microservices with the Kafka Streams API*. Shelter Island, NY: Manning, 2018.
- [62] M. Richards and N. Ford, “Building Distributed Systems Using Sagas,” Online Presentation: O’Reilly, 2023.
- [63] J. Fischer and N. Wang, *Grokking Streaming Systems: Real-time event processing*. Shelter Island, NY: Manning, 2022.
- [64] D. DeVore, S. Walsh, and B. Hanafée, *Reactive Application Development*. Manning, 2018.
- [65] S. Brin and L. Page, “The Anatomy of a Large-Scale Hypertextual Web Search Engine.” Website. <http://infolab.stanford.edu/~backrub/google.html>.
- [66] A. Grover and J. Leskovec, “node2vec: Scalable Feature Learning for Networks,” *Proc. 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, August 2016, pp. 855–864.
- [67] Digibee: The adoption of enterprise system integration is accelerating. Website. https://marketing.digibee.com/state-of-enterprise-integration-report-2023?_gl=1*1yvh7v6*_gcl_au*MTI4MDkwNi4xNzI2NTg1Njk5*_ga*MjUyMTc5Mjk0LjE3MjY1ODU2OTk.*_ga_7XPE175ETS*MTcyNjU4NTY5OS4xLjEuMTcyNjU4NTcxMy40Ni4wLjA.
- [68] S. Raschka, *Build a Large Language Model (From Scratch)*. Shelter Island: Manning, 2024.

Appendix A

T. Reimer, “Semantic Transformation Utilizing Graphs,” Proc. 2022 2nd International Conference on Advanced Enterprise Information System (AEIS 2022), Dec. 2022, pp. 138–145, doi:10.1109/AEIS59450.2022.00026.

Semantic Transformation Utilizing Graphs

Tim Reimer

Department of Electrical and Software Engineering
University of Calgary
Calgary, Canada
email: timothy.reimer@ucalgary.ca

Abstract—Semantic transformation for data in motion has been a continuous problem, and various approaches have been proposed to solve it. This paper proposes a graph approach to handle semantic data transformations. By using a set of traversal patterns, an event log will provide the feature set for both machine-learning algorithms and graph algorithms to make missing node predictions. The paper compares the two approaches and presents significant differences in results.

Keywords—semantic transformation, graph databases, know algorithms, traversal, patterns, data transformation, homogenous graphs, node predictions.

I. INTRODUCTION

Any organization is faced with the continuous challenge of utilizing data assets to maximize value. There are two fundamental aspects that need to be considered in addressing this challenge. Firstly, data needs to be trusted. This is achieved by developing sound corporate data governance and a data quality strategy that ensures a maximum level of validity of the data. Secondly, data needs to be in motion to ensure it is available to the business. In considering a joint effort of data governance, data quality, and data integration, the maximum contribution to business value can be achieved.

Data integration and interoperability are defined as “managing the movement and consolidation of data within and between applications and organizations” [1]. The movement of

data manifests in a variety of ways by utilizing different architectures and technologies. There are the classical enterprise application integration (EAI) patterns such as publish and subscribe, broker, and enterprise service bus (ESB) which address message-based integrations. There are the extract, transform, and load (ETL) technologies to load large data sets into data warehouses and data lakes. With the advent of Internet of Things (IoT), there is streaming data that is received in a continuous flow and routed to large data stores in the cloud. Then there are the new application programming interface (API) architectures that send and receive data within their universe of microservice applications. Finally, there is the ingestion of unstructured data and the association of that data with structured data.

As data travels through the enterprise, it will undergo transformations at various points, from both a syntactic and semantic perspective. This enables the receiving application to generate the information needed by the business. Syntactic transformations are required when data structure hierarchies change as well as when field definitions change, e.g., strings, integers, decimals, and dates. The semantic transformation is concerned with the translation of meaning, in that during the transformation process, the meaning is maintained if not enhanced. Many integration tools have ample support for syntactic data transformation. However, the challenge remains with the semantic transformations, since maintaining meaning cannot be achieved based solely on a grammatical, ontological set of rules. In many cases, maintaining meaning for data structure transformations is arbitrary and is driven by configurations between source and target applications.

II. PROBLEM SPACE

There is no clear line that defines where syntactic mapping merges into semantic mapping. The disparities on the semantic level are more significant since the result can alter the meaning of the intended target information. This is particularly the case with reference data. The source application understanding of a unit of measure is represented via a proprietary character string. The same unit of measure is represented differently in the target system. In most cases, such differences can only be resolved with internal value tables (hard coded) or lookups in some form of database table. However, as Bean [2] correctly identifies, the mapping combination between values can adopt various levels of complexity.

As soon as external tables are involved in resolving semantic differences, additional challenges arise in maintaining these tables. The first challenge is entropy. Data quality degrades over time if not governed. Entropy states that unless energy is injected from the outside, the amount of disorder in an enclosed system will increase over time if not explicitly controlled [3]. These tables must thus be maintained, which in turn incurs challenges in determining who will maintain them. Most often the tables contain business data and thus it seems obvious that maintenance should be performed by a business user. This then raises the question of access and authorization. Today, most cross-referencing is done using various forms of relational database management system (RDBMS) technologies, which provide a very rigid environment as a solution to this problem. Cross-referencing data values requires more agility, though, since relationships can change or be expanded.

Failed or inaccurate maintenance of these cross-reference tables incurs direct cost to the organization, in that integration scenarios will fail and anticipated messages will not be processed in time, thereby causing delays. The cost is twofold: First, missing references will cause transactions to fail; and second, manual labor will be required to maintain and keep this reference data current so that integration scenarios will not fail. Data quality is directly affected which in turn causes data trust issues as described in the introduction.

The fundamental problem can be represented in two essential questions:

- Are there other architectures beyond the traditional cross-reference lookup approach that would improve the overall semantic transformation process?
- How should this architecture handle semantic conflicts?

This paper is structured as follows: After defining the problem space, we review the evolution of related research in using ontologies and semantic transformations in heterogeneous environments. Next follows a discussion of common semantic transformation patterns identified in our research, which provide the building blocks to the proposed architecture. We

will then apply reactive architecture design concepts to capture semantic transformation behavior. Then we will discuss two different conflict mediation approaches using machine learning algorithms and graph embeddings. Finally, we will present a value proposition to support our approach with opportunities for further research.

III. RELATED RESEARCH

The need for interoperability of data to derive information from dissimilar sources has been part of the evolution of information technology. A first visionary reference was made by Vannevar Bush in his description of the Memex machine [4], which was a device that would be able to join various forms of information through indexing. His vision was centered around a desktop computing device that would provide new capabilities to process, combine, and assimilate information from different sources. As technology continued to evolve, databases emerged as a new form of data storage. Wiederhold core have the lo[5] recognized the need to integrate these new data store technologies to provide decision support systems with better information capabilities. He proposed a mediator application that would at its gic to synthesize the data sources by transforming, merging, and storing derived data and delivering it to a workstation application. Given the heterogeneous nature of the data stores, the mediator would also need to be able to resolve data conflicts between them. This type of conflict resolution approach for heterogeneous databases was further researched by Goh [6]. He developed the context interchange framework (COIN) that would not only resolve syntactic differences between databases but also address semantic differences. Semantic conflicts consisted of three categories: scaling conflicts, which addressed differences in measurements and units of measure; naming conflicts, which addressed the differences of synonyms and homonyms of attribute values in tables; and finally, confounding conflicts, which are temporal in nature to describe the latest derived value for an attribute. The classification of these semantic differences provides the base from which solution patterns can be developed for performing semantic transformations. The evolving research on semantic transformation recognized that an abstraction layer was needed to provide the necessary logic in joining a disparate application landscape. This abstraction layer would utilize an ontology as a common definition to provide the basis for semantic transformations.

With the emergence of web service standards and technologies, the interoperability of data has moved from direct database interchange to the application-level interchange and provided new possibilities for enterprise-wide integration architectures. To address semantic differences of data exchange between applications, ontologies at various abstraction levels have been proposed. One such framework is the Ontology-Driven Service-Oriented Integration (ODSOI) [7]. The framework presents a layered architecture style that provides a set of data, functional, and business services with respective ontologies. Given that the development and maintenance of an enterprise ontology is nearly impossible, the abstraction to

local, domain, and global ontologies delivers a better solution. The framework supports a semantic transformation process through the exposed service portfolio. These services will also provide the necessary semantic conflict mediation. Given the nature of web service and applied XML structure, the standard message structure can be enhanced to aid in the semantic transformation process by adding semantic definitions and meaning [8]. Other case studies followed that adopted these architecture styles, one of which was the Semantic Annotation and Profiling (SEAP) tool [9]. The concept was to take an XML-based message and map this to an ontology which would improve conflict resolution of certain data elements. The result would then be converted back to an XML structure to be received by the target application.

Conflict mediation required the use of mappings that would provide the transformation rules to be executed by the middleware. Further development introduced a dynamic mapping capability through a set of mapping functions [10]. These mapping functions would be invoked for certain types of semantic transformations that would need to be performed. Standardization of these functions would improve the overall transformation process between source and target.

The semantic web introduced the standardization of the Resource Description Framework (RDF) and the Web Ontology Language (OWL) [11] to describe semantic relationships in machine-readable form. These relationships are based on triple stores and can be represented in graphical form. By utilizing these concepts, semantic interoperability can be researched from a new perspective. Similarly, the emergence of graph databases allows for storing these relationships as graphs instead of traditional relational structures [12]. Graph databases provide the opportunity to implement different levels of organizational principles: from a plain conceptual relationship graph of classes, to a taxonomy and a description of these classes with their object properties as relationship definitions. Utilizing graph structures moves away from a mapping approach to resolve data interoperability towards a navigation and traversal approach to find matching semantic values. In addition, the transformation process becomes dynamic since it eliminates the need for predefined relationship maps between source application, ontology, and target application.

Using a graph traversal approach will also reduce conflict mediation in cases where a node cannot be located since it may not exist. These scenarios can be resolved by applying reactive application design concepts to track application behavior rather than state. Application behavior tracking is performed by logging the graph traversals and using these logs in machine-learning algorithms to predict missing links.

IV. LIMITATIONS OF RELATIONAL APPROACHES

The scope of semantic transformation can be as simple as translating one unit of measure into another unit of measure, or deriving new values based on a combination of conditions, search results, and a provided key value. Traditionally, these

types of transformations have utilized a relational database with a predefined schema. Relational databases are widely available and represent the logical choice. However, as these search queries grow in complexity, particularly when crossing domains, the relational schema will quickly become complex.

The objectives of schema design are to minimize data redundancy and provide optimal query access by minimizing join operations and nested queries, both of which contribute to performance degradation. A further consideration in schema design is that cross-reference relationships require agility to change quickly. These adaptations primarily result from changing business requirements such as adding new cross-reference conditions or integrating new business partners or application domains.

A third consideration in schema design is that the queries supporting semantic lookup searches are usually static and cannot be changed easily, since they are part of the mapping logic within the middleware and are run in controlled test or production environments. Consequently, the access path of the associated query is static as well. Any change will require adherence to the organizational change and deployment process.

Another limitation of relational databases is that once a base access path is defined, any alternative access path can only be established through joins and/or indexes. It is not so much that a cross-reference database grows in volume of data, but rather that it grows in expansion of its relationships between entities. Since the fundamental design is set, the only way to accommodate this relationship expansion is through join operations or a complete restructure of the underlying relational database. The impact is both costly and performance degrading.

Finally, relational databases are not designed to be integrated into semantic interpretations such as ontologies. There must always be an interpretation layer between the definition of the ontology and the underlying database schema. The rigid schema design of a relational database does not lend itself well to changes that are introduced by an overwriting ontology. However, the enhancement of semantic interpretation between two or more applications requires an ontology that both defines the relationship between the classes and subclasses through its definition language and enables the query of the associated data items. Achieving this with relational databases is complex.

V. SEMANTIC TRANSFORMATION USING GRAPHS

Given these characteristics of relational databases in the context of semantic transformation, our design evaluated NoSQL (not only SQL) databases, particularly graph databases. Graphs provide a network of nodes which are connected via relationships (also called edges). Each node can have various types of properties, and the network of nodes can easily be adjusted to meet new and changing lookup requirements. The search for a corresponding semantic value happens by

traversing nodes. The entry point into the graph can be any node; by traversing the relationships, one can traverse from node to node until the final node with the respective property value is located.

Graphs also transform easily into an ontology and can be used to build a knowledge graph for an enterprise. Combining the reference data with master data, an organization can start developing graphs into a sophisticated network of knowledge which can contribute significant value beyond semantic transformation for data integration.

A. Use Case Description

The applied use case for testing the approaches presented is based on pipeline transactions received from a business partner. These transactions require the semantic transformation of various values in order for the target application to be able to process the transactions. Therefore, a base graph is required that represents the relationships of the different entities and their values. The following (Fig. 1) is the base graph that will be used to perform the traversal to locate the value being searched for. The traversal of the nodes follows a set of patterns described in the next section.

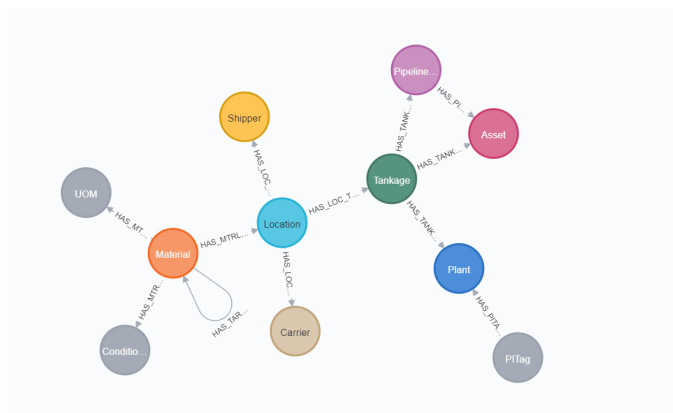


Figure 1. Base graph for use case.

B. Traversal Patterns

Vuković [10] presented a set of mapping functions that would aid in the semantic transformation process. Our approach borrows this concept by identifying four semantic traversal patterns applied to graphs, shown below in the following tables.

TABLE I. ONE-TO-ONE PATTERN

One-to-One

Description

This pattern acts as a key-value lookup by providing the name of the node and property value and the expected value of the response node and response property.

Requirement

Many times, semantic lookups require a quick determination of a value based on a key value. In this pattern the key is provided by the request node and property.

Problem

Given the value of a certain property of a node (key node), find the value of the property of a node in relationship to the key node.

Solution

The request node name and associated property value will be used as argument (key) to find the related response node and property value given a defined relationship between the request node and response node.

Application

The one-to-one pattern is commonly applied for single-argument lookups. Given a specific source value, the pattern will retrieve the respective associated value. An example would be finding the corporate product code given the product code supplied by a business partner.

Implementation

```
Match (RequestNode.Property.Value) -[]-> (ResponseNode.Property)
Return ResponseNode.Property.Value
```

A relationship between nodes is presented as []

TABLE II. ENRICHMENT PATTERN

Enrichment

Description

This pattern provides additional property values based on a single key property.

Requirement

There is the requirement based on a key value to retrieve additional values to enrich the target data structure.

Problem

Given a source data structure that needs to be translated into a target data structure, the target data structure might require additional data elements that do not reside in the source data structure. These data elements must be retrieved from other sources and added to the transformation.

Solution

For each requested enrichment property, the pattern will use the provided key to locate the request node and traverse via the defined relationships between nodes to the different response nodes; retrieve their property; and return the respective values as the enrichment values to be added to the target structure.

Application

A good example is sensor tags from a plant. If the tag represents a valve, the source structure may include the flow rate and volume; but it will not include any reference to a plant ID or asset ID that the target system is expecting. Therefore, the data values must be enriched to satisfy the mapping requirements of the target application.

Implementation

```
Foreach(ResponseNode.Property.Value){
Match (RequestNode.Property.Value) -> (ResponseNode.Property)
Return ResponseNode.Property.Value
}
```

A relationship between nodes is presented as []

TABLE III. DEPENDENCY PATTERN

Dependency

Description

This pattern provides a response property value based on a single key property. However, the retrieval of the response property value is dependent on intermediary nodes and properties. This means a navigation path needs to be established between request and response node.

Requirement

Semantic transformations often require determining a target value based on intermediary associated lookup conditions, starting with an initially provided key.

Problem

To derive a target value for a given semantic transformation, several intermediary lookup conditions must be exercised which can all influence the determination of the requested target value. This requires a multi-hop navigation logic to derive the final value that is required in the target data structure.

Solution

The pattern will use the initial key/value as starting node/property and chain the provided request nodes that make up the lookup chain to establish a navigation path to the response node and its property value. If multiple path options exist for a node, only the path that has the next provided node/property value will be taken. If multiple response nodes are available for a given request node, the relationship between the last request node and

response node must contain the property value of the last request node–1 property value. Otherwise, it is impossible to find the correct response node.

Application

The application is to determine an asset ID based on a material code, a location code, a tankage, and a pipeline segment. The navigation path among nodes would be Material-> Location-> Tankage-> Pipeline Segment-> Asset. However, it is possible that a pipeline segment would not be provided, in which case the navigation path for the nodes would be Material-> Location-> Tankage-> Asset. In addition, there could be multiple asset IDs related to a tankage or pipeline segment code. The relationship between tankage and asset and pipeline segment and asset would need the location code to be attached as differentiating value.

Implementation

```

Loop {
  Match (RequestNode.Property.Value) -[> (NextRequestNode.Property)
  If NextRequestNode = Empty
    LastRequestNode = RequestNode
    Terminate Loop.
}
Match (LastRequestNode.Property.Value)-[RequestNode-
1.Property.Value]-(ResponseNode.Property.Value)

```

A relationship between nodes is presented as [Contains the property value of the request node – 1 property value]

TABLE IV. GRANULARITY PATTERN

Granularity

Description

Based on a key/value, multiple options may exist for a target value.

Requirement

Semantic transformations can encounter the situation in which multiple target values may exist for a given key/value. Therefore, the requirement is to determine the right target value based on a selection criterion.

Problem

Given a key, multiple options exist that could be suitable for a correct target value. The problem is to determine the right target value from a list of options.

Solution

The request node (key) will be used as argument to find the related response node and property value from a list of possible options. This is accomplished by introducing conditions on the relationship between source and target that help determine the correct target value. Such conditions could be time windows and value ranges.

Application

A common scenario is products. A business partner integration provides the product identifier to the integration as part of the transaction. Internally, this product is further refined in various grades, specifications, or characteristics. However, the source message is missing these vital characteristics. A time window as part of the relationship between source and target value is provided to determine if the response should be a winter product or summer product.

Implementation

```

Match (RequestNode.Property.Value) -[Condition]->
(ResponseNode.Property)
Return ResponseNode.Property.Value
A relationship between nodes is presented as [Condition either Time
window or Value Range]

```

C. Tracking Traversal Behavior

A key characteristic of our semantic transformation is the adoption of reactive architecture [13]. This architecture style utilizes the concept of application behavior tracking. In traditional architecture, logs are used to track the state of a

given object. The objective of behavior tracking is to record any changes to an object at any point in time. The advantage of this approach is that logged changes to an object can be used for analytical purposes. In our case, the traversals performed through the four patterns will be recorded in an event log that will be used for missing node predictions.

The example in Fig. 2 shows the traversal path for a successful node search and the same for an unsuccessful node search. For the unsuccessful node searches, an approach is required to predict the missing nodes. We will present two different methodologies to determine missing node predictions.

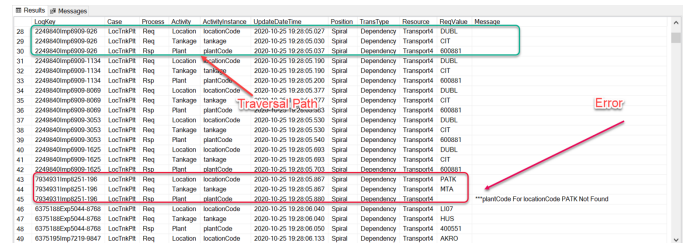


Figure 2. Event log.

VI. NODE PREDICTIONS USING CLASSIFICATION METHODS

The previously presented event log is the historical data set and provides the feature set for the classification algorithms. Each traversed node in the log represents a feature. Since for classifications the feature set needs to be consistent, identical node traversals are grouped together as a feature set and predictions can only be made for that distinct feature pattern. In the example, we used a grouping within the dependency pattern that provided a feature set of four nodes. Three classification algorithms were applied with the results listed in Fig. 3.

	LogReg	SVM	Knn
Cross Validation	94.8333%	94.2083%	88.9167%

Figure 3. Cross-validation results of classification algorithms.

Each result set was then compared (shown in Fig. 4) to determine the similarity of the predictions between the three methods presented above.

	Similarity Ratio
LogReg – SVM	85.54%
LogReg – Knn	87.95%
SVM – Knn	84.33%

Figure 4. Similarity result ratios.

Even though the cross-validation scores are relatively high, the classification methods present significant drawbacks since the feature set must be consistent for the traversal scenario being examined. This adds significant overhead in selecting and grouping the traversal scenarios. Comparing the similarity of the result sets of the three methods shows that LogReg to Knn has the highest similarity rate, but Knn itself has the lowest cross-validation ratio. LogReg has the highest cross-validation

ratio and should therefore be considered the algorithm of choice.

One significant problem with the classification method is that it is based on historical feature occurrences and does not consider a blank node in its community. Therefore, predictions are possible of nodes that are outside the direct community for which a replacement value is being determined; and it is thus possible to get predictions of totally unrelated nodes in graph areas that are also unrelated to the current node location. This error is not reflected in the cross-validation score and contributes the overall wrong predictions. Therefore, a community detection approach within a graph is more suitable for node predictions in semantic transformations.

VII. NODE PREDICTIONS USING GRAPH METHODS

The use of graph algorithms requires a homogenous graph that is derived from the event log. Each log entry represents a node. The value for the node can be considered as a word; where the edge between two nodes becomes the rules on building sentences. Taking the node values as words across all nodes makes each node the same with a different value. This allows for the generation of a homogenous graph from the event log. In cases where a result node was not defined (error case), a blank node is created as an intermittent place holder.

The task now (shown in Fig. 5) is to eliminate these blank nodes within the graph and find suitable replacement nodes.

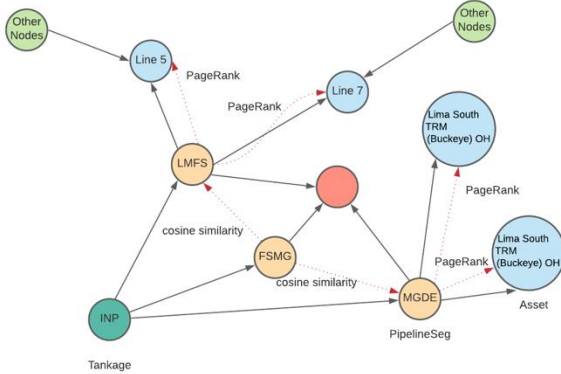


Figure 5. Finding replacements for blank nodes.

There are two scenarios that need to be considered:

8. A parent node has multiple child nodes, of which one is a blank node.
9. A parent node has one child node, which is a blank node.

To address the first scenario (1), we consider the PageRank algorithm. The PageRank algorithm [14] considers the ranking of the node within the community based on the number of incoming relationships and the importance of the corresponding

source nodes. The underlying assumption is that a node is only as important as the nodes that link to it.

$$PR(A) = (1 - d) + d \left(\frac{PR(T1)}{C(T1)} + \dots + \frac{PR(Tn)}{C(Tn)} \right) \quad (1)$$

Where:

- We assume that node A has nodes T1 and Tn pointing to it.
- d is a damping factor which can be set between 0 (exclusive) and 1 (inclusive). It is usually set to 0.85.
- C(A) is the number of links going out of node A.

Based on the PageRank value of all the child nodes, the node with the highest PageRank value will be chosen as a replacement node.

In the second scenario (shown in Fig. 6) a parent has only one child node, which is a blank node. In this case our process retracts to the grandparent node of the blank node, for which the parent node of the blank nodes is a child with sibling nodes. We determine which of the sibling nodes is the most like the node for which a replacement is required.

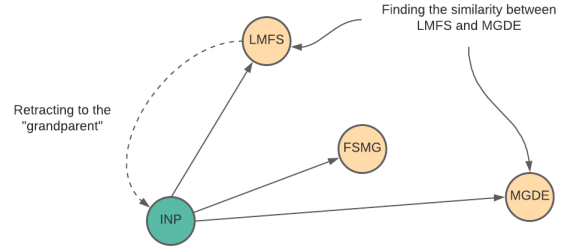


Figure 6. Finding similarity.

To accomplish this, we use node embeddings to transform the graph into a simpler vector-based representation. In our scenario (2) we use the Node2Vec algorithm [15] to achieve this node embedding. Next, we apply the cosine similarity function [16] among the child nodes to determine the node with the highest similarity.

$$similarity(A, B) = \frac{A \cdot B}{\|A\| \times \|B\|} = \frac{\sum_{i=1}^n A_i \times B_i}{\sqrt{\sum_{i=1}^n A_i^2} \times \sqrt{\sum_{i=1}^n B_i^2}} \quad (2)$$

Once that node is determined, the process returns to scenario 1 and applies a PageRank on the child nodes of the newfound parent. The node with the highest page rank will then provide the replacement value for the blank node.

VIII. EVALUATION OF GRAPH SOLUTION AND FUTURE WORK

The following (Fig. 7) provides a comparison between the classification method and the graph method.

CLASSIFICATION METHOD	GRAPH METHOD
• Code predictions are based on history data. • Code predictions do not reflect code dependencies. • Amount of historical data will increase over time and process will slow down. • Not suited for real-time assessments.	• Node relationships within the network drive predictions. • The neighborhood of the node will influence the prediction of nodes. • Only immediate neighbors are considered for node predictions. • Real-time predictions are possible.

Figure 7. Method comparison.

The graph algorithmic approach that we presented represents a viable option for performing semantic transformations; but the solution will still need additional functionality to become a viable production version. One feature that would support this is an easy-to-manage node and relationship management tool that allows users to adjust and confirm predictions. A second consideration is that event log entries occur continuously, and the graph would need to have the capability to update itself every time a new log entry was made. This would also require a recalculation of the node-embedding algorithm to derive the latest node-embedding values. Solutions for these shortcomings are part of future research. In addition, the base graph would need to be updated with the new node predictions to ensure that missing nodes in traversals are constantly eliminated.

IX. VALUE PROPOSITION FOR SEMANTIC TRANSFORMATIONS

Any technical solution should have some way of tracking its value contribution to the organization. The dashboard below (Fig. 8) shows transactions for which a semantic transformation failed and our node predictions were performed. These transactions would normally result in errors, and the impact would be that the volume of barrels shown would not be shipped. The value of these barrels can be associated with a \$ value for which a capital cost can be calculated. The cost savings occurring by being able to ship the barrels of oil is the value contribution our solution would generate.

Semantic transformations can therefore have a significant economic impact on an organization, which in many cases is not measured. Streamlining this process with methodologies presented in this paper will contribute significantly to the bottom line and speed up the overall transaction process.

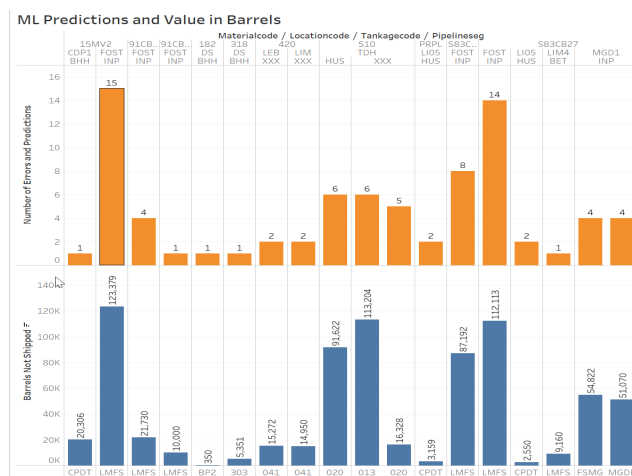


Figure 8. Cost incurred for failed semantic transformations.

REFERENCES

- [1] D. Henderson and S. Earley, *DAMA-DMBOK: Data Management Body of Knowledge*, 2nd ed. Basking Ridge, NJ: Technics Publications, 2017.
- [2] J. Bean, "Enterprise application integration," in *Engineering Global E-Commerce Sites: A Guide to Data Capture, Content and Transactions*. San Francisco: Morgan Kaufmann Publishers, 2003, pp. 271–306.
- [3] P. H. Southekal, *Data for Business Performance: The Goal Question-Metric (GQM) Model to Transform Business Data into an Enterprise Asset*. Basking Ridge, NJ: Technics Publications, 2017.
- [4] V. Bush (1945, July). As we may think. *Atlantic Monthly*, vol. 176, pp. 101–108.
- [5] Wiederhold, G. (1992, March). Mediators in the architecture of future information systems. *Computer*, vol. 25, no. 3, pp. 38–49, doi:10.1109/2.121508.
- [6] C. H. Goh, "Representing and reasoning about semantic conflicts in heterogeneous information systems," Ph.D. thesis, MIT, 1997.
- [7] S. Izza, L. Vincent, and P. Burlat, "A framework for semantic enterprise integration," in *Interoperability of Enterprise Software and Applications*, N. Ivezic, A. Jones, D. Konstantas, J. P. Bourrières, M. Léonard, and N. Boudjlida, Eds. London : Springer, 2006, pp. 75–86.
- [8] S. Izza, L. Vincent, P. Burlat, P. Lebrun, and H. Solignac, "An ontology-driven data mediation framework for enterprise application integration," in *Information Control Problems in Manufacturing*, May 2006, Saint-Etienne, France, pp. 641–646.
- [9] T. Bouras, P. Gouvas, and G. Mentzas, "Dynamic data mediation in enterprise application integration," in *Collaboration and the Knowledge Economy: Issues, Applications, Case Studies*, P. Cunningham and M. Cunningham, Eds. IOS Press, 2008, pp. 917–924.
- [10] Ž. Vuković, N. Milanović, R. Vadera, I. Dejanović, G. Milosavljević, and V. Malbaša (2016). Semantic-aided automation of interface mapping in enterprise integration with conflict detection. *Information Systems and e-Business Management* [Online], vol. 15, pp. 305–322.
- [11] M. Smith, C. Welty, and D. McGuinness, Eds., *Owl Web Ontology Language Guide* [Online]. Available: <https://www.w3.org/TR/owl-guide/>
- [12] J. Barrasa, A. E. Hodler, and J. Webber, *Knowledge Graphs*. O'Reilly Media, Inc., 2021.
- [13] D. DeVore, S. Walsh, and B. Hanafee, *Reactive Application Development*. Manning Publications, 2018.
- [14] S. Brin and L. Page, *The Anatomy of a Large-Scale Hypertextual Web Search Engine*. Available:

<http://infolab.stanford.edu/~backrub/google.html>

- [15] A. Grover and J. Leskovec, “node2vec: Scalable feature learning for networks,” in Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining, August 2016, pp. 855–864.
- [16] F. Rahutomo, T. Kitasuka, and M. Aritsugi, “Semantic cosine similarity,” in The 7th International Student Conference on Advanced Science and Technology ICAST, October 2012.

Appendix B

T. Reimer, “Architecture to Productize Semantic Transformation by Utilizing Natural Language Processing with Microservices,” Proc. 2023 International Conference on Advanced Enterprise Information System (AEIS 2023), Dec. 2023, pp. 35–41, doi:10.1109/AEIS61544.2023.00013.

Architecture to Productize Semantic Transformation by Utilizing Natural Language Processing with Microservices

Tim Reimer

Department of Electrical and Software Engineering
University of Calgary
Calgary, Canada
Email: timothy.reimer@ucalgary.ca

Abstract—Semantic transformation is a necessary requirement in translating message values between various applications so that these messages can be processed. The transformation utilizes several patterns that can be implemented using a microservices architecture. The collaboration of these microservices needs to be managed. This paper discusses the orchestrated and choreographed approaches for this scenario. To demonstrate the choreographed approach, the paper presents a natural language query processing solution that can use custom vocabularies for microservice invocations to achieve semantic message transformations.

Keywords—microservices; semantic transformation; natural language processing; microservice orchestration; natural language queries; choreographed microservices; orchestrated microservices; graph databases; traversal patterns

I. INTRODUCTION

Semantic transformation in the context of this paper is concerned with the transformation of data elements between two or more applications whereby the meaning is maintained throughout the transformation process. This transformation process occurs most often with data in motion. Such use cases could be the exchange of transactional data between applications or the ingestion of data through event or streaming processes for data analytics. Currently, these transformations are performed by applying cross-reference tables structured as key-value pairs. The downside of this

approach is the manual maintenance of such tables with no mechanism for automation or automatic error correction.

A new proposed approach [1] utilizes the concept of graphs whereby the reference data are connected within a graph and the determination of a translated data value is achieved through graph traversals. Each data reference element is a node in the graph connected via its edges, i.e., its relationship to other nodes. A decisive factor for such traversals is the following four transformation patterns:

One-To-One

This pattern acts as a key-value lookup by providing the name of the node and property value and the expected value of the response node and response property.

Dependency

This pattern provides a response property value based on a single key property. However, the retrieval of the response property value is dependent on intermediary nodes and properties. This means a navigation path must be established between request and response node.

Enrichment

This pattern provides additional property values based on a single key property.

Granularity

Based on a key/value, multiple options may exist for a target value.

II. PROBLEM SPACE

Given the described context, a software architecture needs to be developed to implement a suitable solution that incorporates new pattern structures, establishes the bases for a corporate semantic layer, and delivers transformations agnostic of any specific application integration requirements. The problem space can therefore be framed by the following three research questions:

RQ1: What is a suitable architecture for semantic transformation?

RQ2: Which functional collaboration approach would best support semantic transformations and the described patterns?

RQ3: How can natural language processing (NLP) be used to perform a collaboration of services to achieve semantic transformations and provide a layer of abstraction to such architecture?

III. METHODOLOGY

There are numerous research methodologies that are suitable for and specific to information systems research [2]. The main differentiator of these methodologies is the proposed process of execution through various phases to deliver the research results as artifact. Most of the methodologies are presented as a sequential approach with minimal direct feedback or continuous improvement loops. Based on our research questions we opted for the Systems Development Research Methodology (SDRM) [3]. SDRM provided the best fit to deliver the research life cycle from framing the research question to creating a workable solution or product. The research methodology is a phased approach structured as follows: defining the research question; establishing the associated requirements; designing a conceptual framework; defining a proposed system architecture; designing and constructing the solution; and finally, testing, observing, and evaluating the constructed research product.

SDRM is a sequential approach that evaluates research results in the final phase of observation. Since there is no endorsed official agile research approach [4], we opted to incorporate a frequent feedback cycle and incremental development in our research. New research discoveries were tested and integrated into the code base if these new increments contributed to the overall solution in addressing the defined research question. This iterative approach contributed to publishing a minimum viable product (MVP) early in the research cycle to validate research progress. We identified several epics that would encompass a feature set for research. These epics included the discovery of common semantic transformation patterns, research on conflict resolution within semantic transformation, and collaboration of services. Each of these epics would contribute to delivering an MVP that could be presented, reviewed, and observed. Such an MVP for pattern discovery and conflict resolution was presented in our publication [1]. The subsequent step, accomplished by the findings and discoveries in this paper, is to refine the research from the MVP into minimal marketable features (MMF) [5]. Part of the delivered solution requires the identification of enablers that allow for the construction

of the MMF as a solution. One such enabler is a suitable architecture with a supporting technology stack. It is therefore a requirement to define an architecture on which the MMF can be delivered. This process involves refining the requirements from the MVP and mapping them to architectural characteristics, which drives the selection of an appropriate architecture pattern. Along with the selected architecture pattern, a service catalogue needs to be defined that lays out the right service decomposition and communication strategy for the solution. Throughout this process a continuous improvement cycle is adopted and implemented that uses the research findings to make necessary adjustments both in design and construction of functionality. An example of such an iterated research approach is the application of NLP concepts in analyzing choreographed service workflows, which was not considered in the initial design iterations. Our solution therefore supports an incremental research approach that is based on continuous validation of findings while making necessary adjustments to the final MMF.

IV. LITERATURE RESEARCH

We modelled our process of discovery and evaluation of related research after Kitchenham and Charters' [6] recommended guidelines in performing a systematic literature review in software engineering. Based on the research questions, we developed a literature review strategy from Daase et al. [7] that consisted of defining the research terms, which in this context comprised microservices and NLP. We also defined the research scope as the University of Calgary Library (Q1) which includes all significant databases such as IEEE Xplore, SpringerLink, ScienceDirect, and many more. In addition, we included Google Scholar (Q2). For each of the terms we defined three criteria for selection: general criteria, inclusion criteria, and exclusion criteria.

For the term microservices, we specifically focused on recent papers; more specifically, on architecture patterns and collaboration frameworks. These two subcategories provided substantial insight into applying microservice concepts in developing a semantic transformation solution. However, we excluded papers that focused on conceptual architecture designs and future trends since they seemed too general for our intended research scope. Table I provides a summary of the evaluation criteria applied for microservices.

TABLE I. EVALUATION CRITERIA FOR TERM MICROSERVICES

Queries	General Criteria	Inclusion Criteria	Exclusion Criteria
Q1/Q2 Microservices architecture, patterns, and collaboration	- Published between 2017 to current - Written in English - Conference paper or journal article - No duplicates	- Main topic microservice architecture and patterns - Use case scenario for microservices - Microservice collaboration examples and differentiation	- Conceptual architecture designs - Future trends and evolution - Duplicates

For the second term, NLP, our focus was not on text mining or text processing but on query processing based on a custom set of vocabularies. The intended research focuses on demonstrating a choreographed microservice interaction using NLP to provide a solution to semantic transformation in the context of custom term definitions. As with the previous microservice selection criteria, we defined a general, inclusion, and exclusion criteria for both search sources Q1 and Q2 (Table II). Even though the general selection criteria matched the microservices search term, the inclusion criteria focused on NLP query processing in conjunction with microservice collaboration scenarios. We excluded text mining scenarios and general sentence analysis and other NLP frameworks.

TABLE II. EVALUATION CRITERIA FOR TERM NLP

Queries	General Criteria	Inclusion Criteria	Exclusion Criteria
Q1/Q2 NLP query processing	- Published between 2017 and 2021 - Written in English - Conference paper or journal article - No duplicates	- Main topic NLP query processing - Microservice collaboration examples and differentiation	- NLP applications of text mining, translations, sentence analysis etc. - NLP frameworks - Future trends and evolution - Duplicates

The search results proved to be very limited given the criteria of microservice collaboration with NLP queries. Sufficient publications on NLP query processing exist but with different underlying technological implementations. We therefore engaged in a distinct field of research, which is examining the use of microservices and using custom dictionaries for semantic transformations by applying NLP. Semantic transformation use case scenarios in corporations are mostly based on custom terms describing various forms of material, equipment, and other entities. A custom semantic term layer is a compiled dictionary that can be used in queries utilizing NLP.

Microservices were first introduced in Lewis and Fowler's original paper [8], as a new architecture style to implement more efficient service decoupling and advanced deployment strategies to meet the evolving demand for CI/CD. In essence, this architecture style is an evolution of service-based architecture [9], [10]. One of the differences lies in the segregation of databases, where each microservice will manage its own data source. Fundamentally, each microservice adopts the responsibility of a specific function within a bounded context [11]. The collaboration of these individual functions builds an overall software application within a domain. The management of such collaboration between microservices can be considered a workflow [12], [13].

Two scenarios exist [14]. The first scenario describes an orchestrated workflow, in which the invocation sequence is

determined by a predefined rule set that either can be configured or is embedded in the code base and is initiated and managed through a microservice orchestrator. This scenario is easy to implement but causes a tight coupling between the microservice orchestrator and the actual microservices. The second scenario, called a choreographed workflow, proposes a looser coupling where microservices communicate amongst each other with the ultimate objective of providing an overall functional result. This scenario is much more complex to implement but achieves a greater rate of decoupling.

The microservice architectures provide a higher degree of agility in managing software functionality. However, they also present challenges in sustainability and data consistency that need to be considered when choosing this architecture pattern [15]. Prior to engaging in constructing a microservice solution, a sound architecture trade-off analysis should be done.

In the context of this paper, NLP is applied with two objectives in mind. The first objective is to use natural language to formulate a semantic transformation query within the scope of the back-end graph structure. The use of NLP provides an additional layer of abstraction from the front-end request systems and therefore further decouples the front-end applications from the actual semantic back-end transformation processes. The result is a wider range of application use than with a tight coupling approach. The second objective is the use of a choreographed workflow with microservices whereby the invocation of the semantic transformation microservices is based on the construct of sentences.

Sengloiluean et al. [16] present an approach that uses a generalized NLP query to retrieve information from two back-end knowledge bases. The algorithm described increases the accuracy of the result set and investigates similarities between text elements. In this context the query structure is more generic. Corporate semantic transformation is based on a distinct value set between a data source and its target. Therefore, the scope of the matching of terms is unique. The process of finding the right term may vary. The objective of the NLP query in semantic transformation is not to retrieve explicit knowledge but to find the semantically related term that would most likely describe the entity in the target system. This term is then used in the transaction so that the receiving application can process the transaction.

The approach presented by Gupta et al. [17] shares some similarities with the approach presented in this paper. The objective of his research is to take a natural language query and transform it into an SQL query to query relational databases. The presented architecture provides the foundational components of query parsing by tokenizing the sentence structure. From the parsed token an SQL query is then constructed that would be executed against an RDBMS. In its simplest form (key-value lookup), the described approach could be used for semantic transformation. However, the application of more sophisticated transformation requirements would result in more complex queries.

An approach to generate a knowledge graph from text using NLP is presented by Maksutov et al. [18]. The described algorithm parses the text into tokens and their inter-relationships. With the discovered word tokens and associated relationships, Maksutov constructs a knowledge graph. Once text is presented as a graph structure it provides new capabilities in analyzing relationships between nodes by performing, for example, community, similarity, and density analyses.

V. SOLUTION ARCHITECTURE

The four patterns that make up the core semantic transformations are functionally independent from each other; thus their implementation fits well with four distinct microservices (Fig. 1). Adopting a microservices architecture provides significant benefits in scalability, deployment, and maintainability, which were identified as requirements in the initial problem definition. Given the service independence, each service becomes its own architecture quantum. Each semantic transformation pattern presents a service; and these services can be combined into a service mesh. Changes to the

code base will not impact other microservices, and for this reason, the microservices architecture provides a high degree of maintainability and deployability. Each architecture quantum can be deployed independently as part of a CI/CD pipeline.

Scalability is achieved in two ways. First, each microservice can be instantiated multiple times to respond to its request load. Secondly, should new semantic transformation patterns be discovered, or unique functionality be required, additional microservices addressing those needs can be added and incorporated into the service mesh. Each microservice is its own deployment unit.

All services collaborate and communicate using protocol-aware heterogeneous interoperability, meaning that a common protocol is used; however, the form of the message structure may change depending on the number of key-value request parameters. The key-value parameters are packaged into a request object and are passed between the microservices. Semantic transformation results are placed in this request

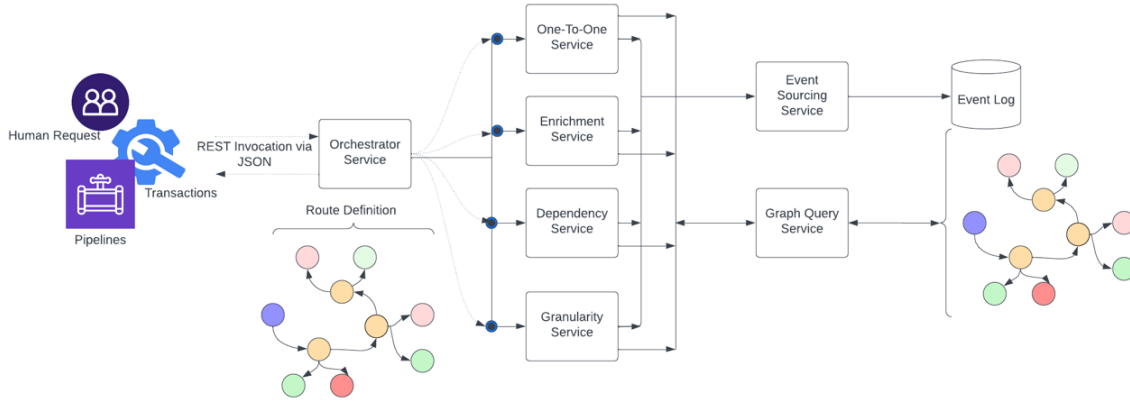


Figure 1: Orchestrated Workflow

object. This way, one service can have access to results delivered by a previous service in the workflow if necessary.

Since each service performs a certain type of semantic transformation, an overall workflow capability is required that can manage the sequence of invocation and assemble the semantic transformation results to be passed back to the requester. In general, a microservices workflow is categorized into an orchestrated workflow and a choreographed workflow.

A. Orchestrated Workflow

In the orchestrated workflow scenario, the orchestrator receives the request message in JSON structure from various potential sources such as a pipeline, web form populated by a user, or general transaction processing. The JSON structure has some core elements such as message ID and source ID. However, the semantic terms are provided as a key-value term structure. One of the core elements that must be provided is a route identifier. The route describes the sequence of the microservice invocation. The orchestrator

will access the routing graph based on the route ID provided and traverse the route definition in the graph to find the sequence in which the four microservices must be called. The route definition also provides the request and response parameters. The response parameter presents the expected result from the semantic transformation. The orchestrator then assembles the request object, which it passes to one of the four microservices. This process continues until the orchestrator has traversed the whole route defined in the routing graph.

Each microservice generates an event log when performing a semantic transformation. This log entry will be passed to a special event log service that stores the log in an event log table. The event log structure adheres to the XES process log standard (www.XES-standard.org).

These event logs provide analytical insights into performance, error management, and predictions on missing nodes in the overall semantic transformation graph [1].

B. Choreographed Workflow

Within the orchestrator approach, the receiving request and the invocation sequence of the microservices are tightly coupled; and the invocation sequence is performed based on the definition of the routes in the routing graph. In contrast, the choreographed approach promotes a looser coupling (Fig. 2). The invocation sequence of the microservices is not predefined based on a route definition but is interpreted from the NLP choreographer of the sentence structure in the received request message. Each organization will have different ways of describing their business entities and attributes which in many cases differ from generally available vocabularies. The historical knowledge on language structure and vocabulary cannot be based on a general corpus of text but must be derived from the terms defined in the semantic graph. These entities with their attributes are represented as nodes and properties in the graph structure. Therefore, each node is made up of a label and its associated properties and their values.

1. Dictionary of Terms

Through the creation of a corporate dictionary that represents the label, property, and values, the NLP choreographer can interpret the different word tokens and discover their meanings related to the semantic graph.

The dictionary needs to be refreshed periodically when changes to the semantic graph structure occur. The dictionary also needs to accommodate scenarios of terms that are not represented in the back-end graph structure. For example, a label can be “Tankage” with assigned properties of tankage code and tankage description. An example query could be as follows: “Give me the tankage code for Location code KG.” The NLP choreographer will recognize “Tankage” as a label, i.e., node in the graph. However, the actual property name is “tankageCode” not “code”. To address these issues the dictionary allows for alias terms that can be applied on both label and properties. In this case an alias of “code” would be stored as an alias for “tankageCode”.

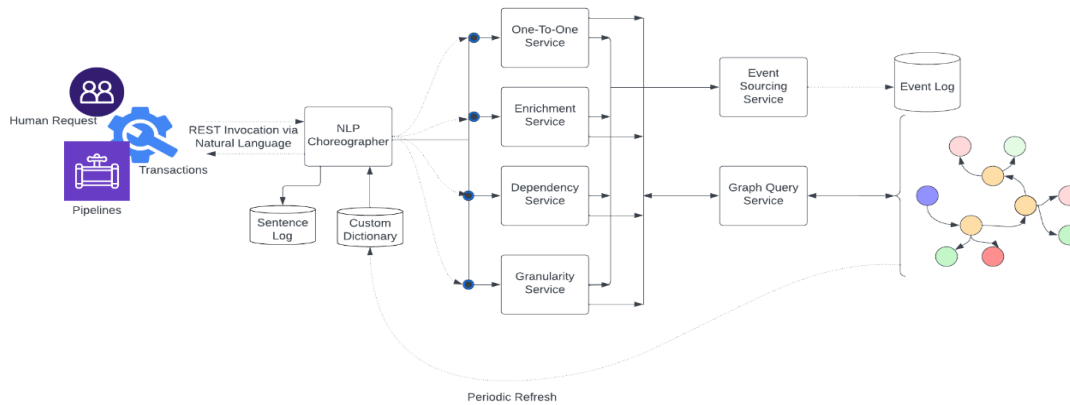
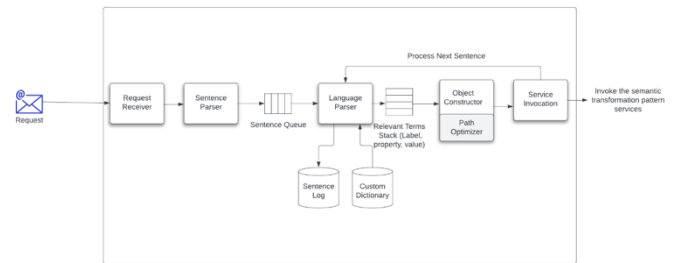


Figure 2: Choreographed Workflow

2. Grammatical Structure of the Natural Language Query

A natural language query must adhere to a few grammatical rules. As a first rule, a query must be structured into multiple query sentences. Each sentence represents a sub query that the NLP choreographer uses to determine which microservice pattern is to be invoked. This provides a solution to call multiple functional independent microservices within one query submission. The sentence parser analyzes the query and parses each sentence and stores it in a sentence processing queue.

The second rule requires that the expected semantic response must be stated first in a query sentence, as in our previous example: “Give me the Tankage for tankageCode for Location locationCode KG.” It is not required that a label is provided as long as the label name is the prefix of a property that follows. For example, one can write “Tankage for tankageCode” as well as just “tankageCode” or “Tankage



Code”. The language parser will recognize the different terms and break apart custom words like “tankageCode” to determine the required label.

The third rule requires that for value properties, the value must follow the property. Using the example sentence “Give me the Tankage for tankageCode for Location locationCode KG,” KG is the value of locationCode. Logical expressions

Figure 3: Language Parser

among properties are currently not supported and are a topic for future research.

3. The Language Parser

Given the grammatical rules described above, the language parser (Fig. 3) analyzes each sentence by performing a word tokenization over the sentence. Each sentence is mapped to only one invocation of microservices. To achieve a chain invocation as previously defined in the orchestrator scenario, multiple sentences must be provided. The language parser will specifically look for nouns (NOUN) and proper nouns (PROPN) as part of the word token. If it identifies either, it will search the dictionary for the term first as a label. If a label can't be found, it will try to map the suffix of the word with aliases in the dictionary. If it finds one, it will parse the suffix from the word text and use the prefix for another attempt to find a label.

For every word text found in the dictionary, the language parser will push the word into a stack. Once the sentence is parsed completely, the object constructor is invoked, which will process the stack and build the proper request key-value pairs with the final two entries being the response label and property as described in grammatical rule one.

The object constructor processes the stack and constructs the request object to be passed to the respective microservice. It will also determine through the path optimizer the optimal traversal path to find the correct semantic transformation value; and rearrange the labels and properties so that the microservice can follow the predefined traversal path. Once the object constructor has received a response, it will invoke the language parser again to process the next sentence.

Each query sentence is stored in a sentence log file. The idea is that collecting various queries will eventually build a sufficient training set that can be used for parsing and improving parsing algorithms as well as for eventually generating dictionary aliases.

```
1 {
2   "custid": "AS-209003",
3   "route": "",
4   "messageid": "AFDSS-333",
5   "sourcesys": "Transport4",
6   "targetsys": "Spiral",
7   "cmpvalue": "",
8   "query": "Give me the material name for material code 420. Give me the assetCode for Tankagecode SPM"
9 }
```

Fig. 4 demonstrates a sample request message of an NLP query with two sentences. The query is looking for the material name and the asset code for a respective material code and tankage code. Notice the different spelling of material code and tankage code. “Material code” is lower case and two words. In this case “Material” will be identified as label and code as property. Therefore, the language parser will look for an alias of “code” within the label “Material”. “Tankagecode” is one word and the first letter is upper case. The correct property spelling in the graph is “materialCode” and “tankageCode”. The language parser therefore has the capability to interpret the spelling to determine the right label and property name to find the correct result.

```
{
  "assetCode": "Hudson TRM (Inland) OH",
  "cmpvalue": "",
  "custid": "AS-209003",
  "materialName": "Gasoline Conv Reg",
  "message": "Success",
  "messageid": "AFDSS-333",
  "query": "Give me the material name for material code 420. Give me the assetCode for Tankagecode SPM",
  "route": "",
  "sourcesys": "Transport4",
  "statuscode": "200",
  "statusCode": "200",
  "targetsys": "Spiral"
}
```

Fig. 5 provides the response for the request query and shows the correct asset code and the material name of “Gasoline Conv Reg”.

This query demonstrates the behavior of the NLP choreographer with multi-sentence structure to issue multiple queries; it is capable of deciphering the query sentence in such a way that a semantically correct query is constructed and passed to the correct back-end microservice to retrieve the results. In addition, the example demonstrates that companies can have their own unique code structure, and the language parser will be able to process these codes to correctly formulate a semantic request query for the microservices.

VI. EVALUATION

The two workflow styles for microservices presented for the described use case of semantic transformation deliver various benefits as well as drawbacks. A comparison of the two methods is based on a set of selected architectural characteristics that can be considered most significant. The evaluation shown in Table III is a qualitative comparison of these characteristics for the two options.

TABLE III. ARCHITECTURAL CHARACTERISTIC EVALUATION

Architectural Characteristics	Orchestrated	Choreographed	Comment
Adaptability	**	*****	The ability to adapt to new front-end applications
Correctness	*****	**	Consistent semantic transformation results delivered
Maintainability	***	**	Maintainability of code base for fixes and new functions
Testability	*****	**	Complexity in applying various test cases
Operability	***	**	Ease of operation of the workflow from a sustainability perspective

The choreographed approach provides an overall higher degree of adaptability to other environments and would accommodate a broader use case base within an organization. However, the complexity of parsing sentence structures and compiling back-end search queries offsets these benefits and

gives the orchestrated workflow an overall better rating even though any new microservice invocation scenario would have to be manually maintained and tested. The implementation of an NLP choreographed invocation approach will therefore largely depend on the strategic direction that semantic transformation will take in an organization. If the intent is to build out the graph of terms to a corporate knowledge graph, then the choreographed approach would be a better choice over the orchestrated approach. For standard semantic transformation implementations, the orchestrated approach would be the best choice.

Because the request object acts as a contract between the microservices, it provides an architectural abstraction. Therefore, an initial implementation of the orchestrated approach is recommended with the plan to eventually migrate to an NLP choreographer to benefit from a higher degree of adaptability to various front-end applications. The existing microservices' functionality and the back-end graph would not have to change.

VII. FUTURE RESEARCH AND CONCLUSION

Given the potential of these two workflow scenarios, various future research initiatives could be explored. For one, the design and implementation of a central service registry would be a feature that would allow for the publication of services to potential consumers. In such a case, the back-end graph would become a data product that would be consumable by a wider number of applications. It would also contribute to integrating new services in the overall route definition for either the orchestrator or the NLP choreographer.

In addition, the generic creation of alias terms based on the semantic log tables should be further researched so that the creation of the term dictionary can be automated to a higher degree. Finally, the grammatical complexity of the natural language query should be pursued by adding new logical selection and comparison capabilities.

In conclusion, this paper demonstrates two distinct microservice workflow scenarios using two completely different approaches to achieve the same results. Each demonstrated approach has trade-offs that need to be considered in making a final implementation decision. Even though both presented solutions vary significantly, the abstraction through the definition of a request object allows for the use of a common back-end with a significantly different microservice management approach. It demonstrates that decoupling of components allows for the change of components and therefore creates an overall more versatile and agile software architecture.

REFERENCES

- [1] T. Reimer, "Semantic Transformation Utilizing Graphs," Proc. 2022 2nd International Conference on Advanced Enterprise Information System (AEIS 2022), Dec. 2022, pp. 138–145, doi:10.1109/AEIS59450.2022.00026.
- [2] J. F. Nunamaker Jr., M. Chen, and T. D. M. Purdin, "Systems Development in Information Systems Research," *Journal of Management Information Systems*, vol. 7, 1990, pp. 89–106.
- [3] J. R. Venable, J. Pries-Heje, and R. Baskerville, "Choosing a Design Science Research Methodology," Proc. ACIS 28th Australasian Conference on Information Systems (ACIS2017), AISel, Dec. 2017, pp. 1–11, <https://aisel.aisnet.org/acis2017/112>.
- [4] C. Baham and R. Hirschheim, "Issues, Challenges, and a Proposed Theoretical Core of Agile Software Development Research," *Information Systems Journal*, vol. 32, Jan. 2022, pp. 103–129, doi.org/10.1111/isj.12336.
- [5] R. Knaster and D. Leffingwell, *SAFe Distilled: Applying the Scaled Agile Framework for Lean Enterprises*. Boston, MA: Addison-Wesley, 2019.
- [6] B. Kitchenham and S. Charters, "Guidelines for Performing Systematic Literature Reviews in Software Engineering: Version 2.3," EBSE Technical Report, Jan. 2007.
- [7] C. Daase, D. S., M. Volk, and K. Turowski, "Creation of a Framework and a Corresponding Tool Enabling the Test-Driven Development of Microservices," Proc. 2022 2nd International Conference on Advanced Enterprise Information System (AEIS 2022), Dec. 2022.
- [8] J. Lewis and M. Fowler, *Microservices: A Definition of This New Architectural Term* [Online]. Available <https://www.martinfowler.com/articles/microservices.html>.
- [9] N. Dragoni, S. Giallorenzo, A. L. Lafuente, M. Mazzara, F. Montesi, R. Mustafin, and L. Safina, "Microservices: Yesterday, today, and tomorrow," in *Present and Ulterior Software Engineering*, M. Mazzara and B. Meyer, Eds. Cham: Springer, 2017, pp. 195–216.
- [10] T. Engel, M. Langermeier, B. Bauer, and A. Hofmann, "Evaluation of microservice architectures: A metric and tool-based approach," in *Information Systems in the Big Data Era: CAiSE Forum 2018*, Tallinn, Estonia, June 11–15, 2018, Proceedings, J. Mendling and H. Mouratidis, Eds. Cham: Springer, 2018, pp. 74–89.
- [11] I. Karabey Aksakalli, T. Çelik, A. Burak Can, and B. Tekinerdoğan, "Deployment and Communication Patterns in Microservice Architectures: A Systematic Literature Review," *Journal of Systems and Software*, vol. 180, Oct. 2021, pp. 1–25, doi:10.1016/j.jss.2021.111014.
- [12] A. Megargel, C. M. Poskitt, and V. Shankararaman, "Microservices Orchestration vs. Choreography: A Decision Framework," Proc. 2021 IEEE 25th International Enterprise Distributed Object Computing Conference (EDOC), IEEE, Oct. 2021, pp. 134–141, doi:10.1109/EDOC52215.2021.00024.
- [13] M. Richards and N. Ford, *Fundamentals of Software Architecture*. O'Reilly, 2020.
- [14] N. Ford, M. Richards, P. Sadalage, and Z. Dehghani, *Software Architecture: The Hard Parts*. O'Reilly, 2021.
- [15] C. T. Joseph and K. Chandrasekaran, "Straddling the Crevasse: A Review of Microservice Software Architecture Foundations and Recent Advancements," *Software: Practice and Experience*, vol. 49, July 2019, pp. 1448–1484, doi:10.1002/spe.2729.

- [16] K. Sengloiluean, N. Arch-int, S Arch-int, and T. Thongkrau, "A Semantic Approach for Question Answering using DBpedia and WordNet," Proc. 2017 14th International Joint Conference on Computer Science and Software Engineering (JCSSE), IEEE, Sept. 2017, pp.1–6, doi:10.1109/JCSSE.2017.8025918.
- [17] P. Gupta, A. Goswami, S. Koul, and K. Sartape, "IQS-Intelligent Querying System Using Natural Language Processing," Proc. 2017 International Conference of Electronics, Communication and Aerospace Technology (ICECA), IEEE, April 2017, pp. 410–413, doi:10.1109/ICECA.2017.8212846.
- [18] A. A. Maksutov, V. I. Zamyatovskiy, V. N. Vyunnikov, and A. V. Kutuzov, "Knowledge Base Collecting Using Natural Language Processing Algorithms," Proc. 2020 IEEE Conference of Russian Young Researchers in Electrical and Electronic Engineering (EIConRus), IEEE, Jan. 2020, pp. 405–407, doi: 10.1109/EIConRus49466.2020.9039303

Appendix C

T. Reimer, “Utilizing Semantic Transformations in Event Streams,” Proc. 2023 International Conference on Advanced Enterprise Information System (AEIS 2023), Dec. 2023, pp. 47–52, doi:10.1109/AEIS61544.2023.

Utilizing Semantic Transformation in Event Streams

Tim Reimer

Department of Electrical and Software Engineering

University of Calgary

Calgary, Canada

Email: timothy.reimer@ucalgary.ca

Abstract—Semantic transformation is a necessary requirement in the exchange of messages between heterogenous applications. With the increase in applied sensor technologies, stream processing architectures have emerged as a critical processing capability for enterprises. This paper provides a discussion on two different stream architecture styles. The first is the publish and subscribe pattern and the second is the stream process. In both cases, a solution is presented and evaluated to integrate semantic data transformation so that downstream applications can provide analytics beyond the original sensor IDs and their measurements.

Keywords—event streaming, event processing, semantic transformation, pub/sub pattern, IoT streaming, data pipelines, semantic routing, event brokers, stream processors, sensors, semantic transformation patterns

I. ARCHITECTURE CONCEPTS

A. Event Streaming Concepts

The Internet of Things has penetrated every aspect of business and personal life. The collection and processing of this data occurs with data streams and requires different architectures compared to the classical batch and transaction processing. A major difference of processing event streams is that these streams are infinite and unbound in nature. *Infinite* describes the characteristic that data is continuously flowing, and *unbound* reflects that streams have no distinct data set limit. In addition, event streams are ordered based on event occurrence. This is important for events that require that their flow adheres to a defined sequence. Another

characteristic of events is that they are immutable. Once an event is created it cannot be changed. Any changes required for an event requires the issuing of a new event with the desired changes to the stream. Immutable events therefore support the capability of replaying events so that they can be reprocessed. These characteristics of event streams therefore provide new challenges in managing optimal performance and reliability.

There are three fundamental concepts that describe stream processing [1]. First, event streams require the definition of a process topology that defines the sequence of the individual processing steps. These processing steps may consist of aggregations, branching, manipulation, and filtering.

Secondly, the processing of event streams may consider different time windows such as fixed, defined, or floating time windows. In addition, time events can be of different types to reflect either processing time, log append time, or ingestion times. The complexity of time management within event streams can be demonstrated when averages are to be calculated within a five-minute time window and a potential broker goes down for maintenance or other issues. Sheehy’s discussion on time management in distributed systems [2] profoundly demonstrates this complexity.

Finally, stream processors need to consider stateful or stateless scenarios to accurately compute desired results on these event streams. Compared to stateful events, stateless events do not require context such as history or previous calculation results to compute a new result. Maintaining state will therefore require an intermediate storage capability in the event stream to ensure that the state context can be retrieved.

Given the streaming characteristics described above, it is evident that event streams require an architecture pattern that

will support these characteristics. Streams are composed of events that are continuously flowing, thus forming an event stream. Events are records of something that happened at some point in time [3]. Events should not be confused with messages. The difference between events and messages is that messages indicate to the receiver an action to be taken, whereas events indicate that an action has occurred.

A common architecture pattern for event streams are event brokers [4]. Events arrive asynchronously and are published via an event producer to a topic for consumption by a topic consumer. These consumers can in return perform some computing on the received events and either persist them in a data store or publish the computed event back to a topic. Since the publish and subscribe pattern is implemented between producer and consumer, many consumers can retrieve the same event. A special consumer distribution logic is not required, and this is what makes this topology so attractive. Brokers are part of a cluster which provides significant scaling capabilities to process thousands of events.

B. Semantic Transformation Concepts

The sharing of data through message exchange among dissimilar computing applications has been an ongoing challenge. A first visionary reference was made by Vannevar Bush in his description of the Memex machine [5]. As technology continued to evolve, databases emerged as a new form of data storage. Wiederhold [6] recognized the need to integrate these new data store technologies to provide decision support systems with better information capabilities. The exchange of syntactical transformation is achieved through specific mapping functions that allow for transformation from one format to another. However, the semantic transformation is still a challenge since the meaning of the data element in the source application needs to be maintained in the target environment.

Different approaches have been proposed. Some use ontologies and taxonomies to transform the receiving values and structures into a global canonical structure. Once such a transformation is achieved, the ontology is used to find the related meaning. One such framework is the Ontology-Driven Service-Oriented Integration (ODSOI) [7]. The framework presents a layered architecture style that provides a set of data, functional, and business services with respective ontologies to semantically translate values through the data exchange; and provides mediation capabilities in case of missing values.

The challenge compared to normal language translations is that semantic transformation must address (in most cases) the transformation of various codes. These codes are not standardized and will take on different values within organizations but also between different applications. For example, a material code in one application may differ from the same material code in another application. The same can

be seen in units of measure. For example, the unit of measure for a barrel of oil may be “BR” in one application but “bbl” in another.

In its simplest form, a key-value lookup is executed using a key-value store. The source system provides a certain code for an entity which needs to be translated into the code of the target application for the same entity. However, semantic transformations can take on more complex forms [8]. In some cases, a source value code with several supporting codes may be provided to determine the respective target code value. In this case there will be dependencies between the different source codes that will determine how the target code should be derived. For example, a material code combined with a location code for the material will determine an associated asset code. This scenario requires a complex search to be able to dynamically derive the correct target value. Another scenario is the enrichment of certain values with additional data elements that do not exist in the original message but are required for the receiving application. In this case, these new data values need to be found based on data elements in the source message and added to the target message. For example, the asset code provided does not have any asset description, but the target application requires the description to be consumed and stored. Finally, the scenario exists where a source application provides a certain code, but the target application has multiple code options for the single source code. This is often the case when product codes in the receiving system describe seasonal product variations. To determine the correct target code, a time window can be used as a condition.

In summary, semantic transformation must consider various transformation scenarios that go above and beyond the simple key-value lookup. The previously described translation scenarios are not only executed as a single execution but can be sequenced to establish a chain of transformations to achieve a list with multiple semantic results. The individual use case of the transformation requirement will determine the correct pattern execution sequence.

II. PROBLEM SPACE

Event streams provide a different processing challenge compared to transactional and batch processing scenarios. In both transactional and batch, integrating semantic transformation logic would use a request and response pattern. The source application would make a request to the semantic transformation logic for value transformation to a semantic transformation orchestrator. This orchestrator would manage the execution sequence of the described transformation patterns based on a predefined execution sequence. In contrast, event flows use a publish and subscribe integration pattern where the event producer publishes the event to a topic. The consumers listen on the registered topic and retrieve the event for further processing. The consumer can then either forward the event to a data store or republish the event to another topic. In addition, the use of stream

processing logic provides another method of event processing. Stream processing occurs between the event being published and the consumer retrieving the event. Each stream processor follows a predefined topology of processing steps which can be functions such as filter, branching, grouping, and aggregations.

The purpose of this paper is to demonstrate how semantic transformation logic can be architecturally integrated into event streams, since there are many use cases where such transformations are of value. Therefore, the research questions can be stated as follows:

RQ1: How can semantic transformations be integrated into event streams?

RQ2: What architectures would support semantic transformation in event streams?

The paper is structured as follows. First, we introduce a common use case in collecting sensor data from IoT devices as events and streaming them to an event broker. Then we discuss two proposed architecture solution scenarios for integrating semantic transformations into event processing. Finally, we compare these two proposed solutions.

III. SOLUTION ARCHITECTURE

A. Use Case

For demonstration purposes, the following use case is applied. A production facility has numerous sensors on the production floor collecting various types of measurements. Each of these sensors has a tag ID for identification purposes. As a measurement is taken, the sensor transmits this measurement as an event to an event broker. The event broker in return publishes the event to various topics which could be based on different criteria such as plant and location. Consumers are monitoring these topics and retrieve the events for further downstream processing (Fig. 1).

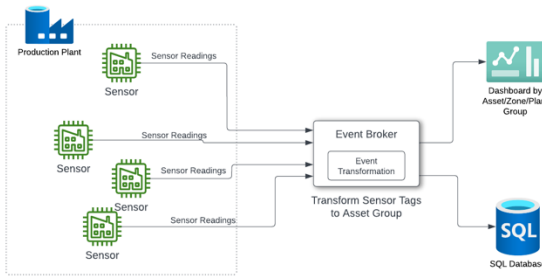


Figure 1. Sensor Data Collection Stream Use Case.

The objective is for the consumer to present the event not based on tag ID but rather in the form of asset ID or equipment ID. Therefore, the tag ID needs to be translated to the proper asset ID/equipment ID. This would follow a one-

to-one mapping pattern in which the key (tag ID) is translated to the asset ID.

The scenario described above demonstrates only one production plant. However, the scenario could be extended to multiple production plants, and each plant could be subdivided into production zones. In this case the tag ID semantic transformation would need to be extended to also enrich the event for the consumer to include the plant ID and the zone ID. This demonstrates how the semantic transformation quickly increases in complexity from a simple key-value lookup to adding additional data elements to supplement the measurement.

Once the enrichment has occurred, the consuming application (e.g., a dashboard) can now present the performance of various sensor groups by asset ID/equipment ID, by zone, and by production plant. The enrichment pattern also provides better data for business analytics scenarios such as preventive maintenance and production forecasting. The use case demonstrates the requirements for semantic transformations where events require either the enrichment of additional data elements or the transformation of a tag identifier to an associated category for aggregation purposes.

B. Semantic Transformation Architecture

Most applications use the traditional key-value lookup approach with database tables. However, this approach is very simplistic and does not address the previously described transformation scenarios. It also contributes to a fragmented distribution of these database tables, since in most cases each use case will have its own supporting translation database. A more effective method is to utilize graph structures [8]. Graph structures connect data entities with their attributes through graph relationships, also called edges. Compared to relational database stores, graph structures can be consistently enhanced without the need for a reload. Database reloads are most often required when the database schema changes. The graph structures provide significant advantages in centralizing the node relationships by allowing for easy modification of nodes and edges. New nodes with their properties can be added, and nodes can be linked with new relationships at any time. There is no requirement for a database load. Via the relationships or edges, multiple traversal paths exist. This allows the traversal algorithm to search for a respective node using different paths.

Each transformation pattern utilizes a unique traversal algorithm to satisfy its unique purpose. Table I provides a synopsis of each of the four patterns and their traversal approaches.

TABLE I. SEMANTIC TRANSFORMATION PATTERNS

Pattern	Description
One-To-One	This pattern acts as a key-value lookup by providing the name of the node and property value and the expected value of the response node and response property.

Dependency	This pattern provides a response property value based on a single key property. However, the retrieval of the response property value is dependent on intermediary nodes and properties. This means a navigation path must be established between request and response node.
Enrichment	This pattern provides additional property values based on a single key property.
Granularity	Based on a key/value, multiple options may exist for a target value.

Fig. 2 depicts a sample graph for the described use case. The tag is connected to an asset ID, zone ID, and plant ID. Assets are in return connected to zones. Using the described patterns for semantic transformation, one can traverse the graph to find the associated zones, assets, or plants.

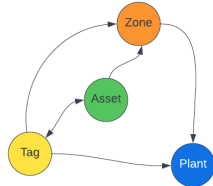


Figure 2. Graph Structure.

Each of the patterns is implemented via microservices since its bounded context is the scope of the pattern. An overall orchestrator manages the invocation of these microservices to ensure that the correct traversal approach is taken and to get the correct translation results. Failed attempts at locating correct transformation results are handled via community detection algorithms [8]. The following section will discuss two potential architectural scenarios of integrating semantic transformation using graphs and microservices with event streams.

C. Semantic Transformation with Pub/Sub

In the first scenario, the events that are created by sensors are sent to an event producer which publishes these events to a topic (Fig. 3). For every transformation pattern a consumer monitors the topic. When a new event arrives, the consumer will call the transformation microservice with the respective tag ID and the expected return value. The microservice will execute the necessary graph traversal to find the translated value and return this value back to the consumer.

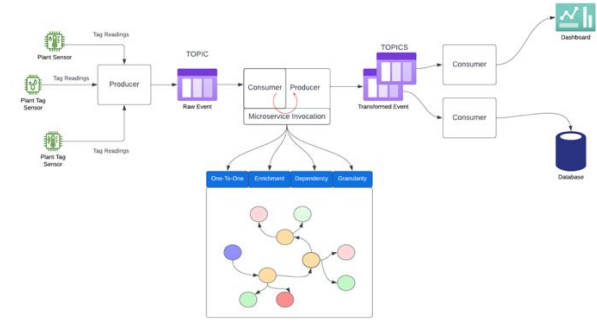


Figure 3. Semantic Transformation with Pub/Sub.

The consumer in return will create a new event with the returned translated values; and will publish this new event as producer to various topics depending on downstream event processing requirements. This allows different “end” consumers to listen to different topics and use the events retrieved (which are now semantically translated and enriched) to further aggregate, store, or otherwise process the events. It is important that the original tag ID is passed through the process. This way, data lineage of the event stream is preserved, and a full audit trail of the event stream is maintained. The semantic transformation consumer will communicate with the respective microservices via a request object. The consumer therefore acts in the architecture as orchestrator of the four pattern microservices. This introduces abstraction into the architecture. Loose coupling allows the underlying microservice to change by introducing potential new transformation functionality and therefore the capability of horizontal scaling.

D. Semantic Transformation with Stream Processing

Compared to the traditional pub/sub scenario discussed in the previous section, stream processors execute a topology of APIs to perform event manipulation through aggregations or value mapping [9]. State is maintained by storing event values in special stream tables. In its simplest form, value mapping could be performed with the provided APIs. As previously discussed, however, semantic transformation requires more sophisticated mapping than simple key-value lookups. This is the justification to extend the functionality of the stream processor by introducing the four patterns as additional API functions. These new API functions establish a direct connection with the graph to perform the required traversal to find the correct semantic values.

Fig. 4 depicts the architecture of a stream processor with integrated semantic transformation. As in the previous scenario, the sensor data is published to a topic of raw events. However, now the stream processor retrieves these events with a source process.

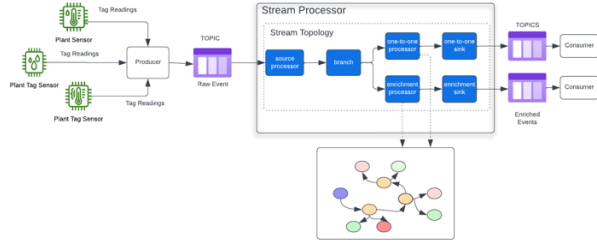


Figure 4. Streaming Process Architecture.

A branch step determines which of the patterns to invoke. This could be determined based on the source topic or on other characteristics embedded in the event stream itself. The appropriate semantic pattern API is invoked, which initiates the traversal in the graph to find the correct translated results. After the semantic translation step, the stream processor invokes a sink step that publishes the new event with the translated data element to a new topic. Consumers monitor these topics, retrieve the events, and handle any requirements for downstream processes.

Compared to the pub/sub scenario, the integration to the graph in this scenario requires a tighter coupling with APIs rather than the invocation of microservices. The reason is the latency that is involved in communicating with the microservice. In this scenario, a microservice invocation would cause backpressure to develop which would have to be managed. Backpressure in streaming processes occurs when arriving events queue up from the source systems because of latency within the stream process [1].

Fig. 5 provides a demonstration of the described use case. The right side shows the streaming events as they are being sent by the different sensors, and their temperature readings. The events are published as raw events to a topic. The left side shows the transformation from a tag ID to the associated plant ID which is added by creating a new event for further processing of downstream applications.



Figure 5. Event Flow of Temperature Readings.

To note here is that multiple tag IDs may belong to one plant. Since the tag ID is preserved, a downstream dashboard can now show the temperature fluctuations within a plant with the capability to further drill down to the actual sensor.

This case can be extended to include other nodes such as asset ID and zone ID.

E. Solution Evaluation

The prime objective of this study is to demonstrate the integration of semantic transformation with event streams. The evaluation of the solution therefore focuses on the three primarily dynamic coupling forces [10] between the event streams, the semantic transformation patterns, and the graphs. These coupling forces consist of communication, coordination, and consistency. Communication describes the interaction between the event stream and the respective pattern to provide the right transformation. Coordination describes the interaction and collaboration between the streaming services and the semantic transformation patterns. Consistency evaluates how well state is maintained throughout the semantic transformation process. Besides the three coupling forces, consideration should be given to scalability (both horizontal and vertical). Horizontal scalability describes the capability of adding new functionality to the integration. Vertical scalability considers the increase of processing instances to improve performance.

With the pub/sub scenario, a consumer acts as an orchestrator and communicates with the pattern microservice via REST communication. The communication is loosely coupled between the consumer and the respective microservice. The benefits of loose coupling are offset by the attempt to integrate asynchronous event streams with synchronous request and response patterns, which can cause performance issues. Another advantage of loose coupling is that added functionality via additional microservices can be easily integrated into the scenario. Consistency of events is maintained during the process via creating new events and publishing them to new topics. The coordination is managed through the consumer who has embedded logic for sequencing the execution of the various microservices. From a vertical scaling perspective, it's easy to increase the instantiation of each microservice to address communication performance issues. A comparison rating of the scenario is presented in Table II.

With the event streaming scenario, a custom API has to be developed to integrate the transformation graph with the stream process. The communication is tightly coupled with the graph to achieve high performance. Consistency is managed through internal event tables that the stream provides. The management of the API invocation is achieved through the design of the processing topology, which again requires a tight coupling approach. Scalability is more challenging since new translation functionality would require the development of a new API.

TABLE II. ARCHITECTURE EVALUATION

	Pub/Sub Scenario	Stream Processing
Communication	***	****
Consistency	***	****
Coordination	****	**
Scalability	****	**

The semantic transformation process always occurs against a single event in the stream and therefore supports task and data execution independence [11]. Parallel processing of these transformations to increase throughput is viable because the same process can be executed on different event sources.

Both the pub/sub and the stream processing scenarios are viable architectural options. The decision on which one to use will largely depend on business objectives. If performance and throughput are the primary concerns, then stream processing is the right choice. One needs to consider that a custom API is required to support the semantic transformation patterns, which is not a trivial task to take on. However, if a looser coupling approach is desired to support the implementation of enhanced functionality, then the traditional pub/sub scenario would be a better alternative to address semantic transformation requirements. The advantage here is that the same microservice pattern can be used in other use cases such as data analytics pipelines and enterprise service busses; and therefore, provides a high degree of utilization compared to event stream processing APIs.

IV. CONCLUSION

Semantic transformation requirements appear in many different enterprise information architecture scenarios. In this paper, the discussion focused on event streams and the design of such architectures. A comparison of two solutions was performed. However, the semantic transformation patterns

described can have a wide range of implementations and integrations. Other scenarios could include choreographed microservice invocations with natural language processing or the development of a complete semantic data layer. With a data layer, organizations could offer semantic transformations as a service rather than a single business implementation. Such a semantic data layer could then merge into a knowledge graph and act as a binding element between various data domains to form a data mesh.

REFERENCES

- [1] G. Shapira, T. Palino, R. Sivaram, and K. Petty, *Kafka: The Definitive Guide*, 2nd ed. Sebastopol, CA: O'Reilly, 2021.
- [2] J. Sheehy, "There is No Now: Problems with Simultaneity in Distributed Systems," *ACM Queue*, vol. 13, no. 3, Mar. 2015, pp. 20–27, doi:10.1145/2742694.2745385.
- [3] M. Kleppmann, *Designing Data-Intensive Applications: The Big Ideas Behind Reliable, Scalable, and Maintainable Systems*. Sebastopol, CA: O'Reilly, 2017.
- [4] M. Richards and N. Ford, *Fundamentals of Software Architecture: An Engineering Approach*. Sebastopol, CA: O'Reilly, 2020.
- [5] V. Bush, "As We May Think," *Atlantic Monthly*, vol. 176, July 1945, pp. 101–108.
- [6] G. Wiederhold, "Mediators in the Architecture of Future Information Systems," *Computer*, vol. 25, no. 3, Mar. 1992, pp. 38–49, doi:10.1109/2.121508.
- [7] S. Izza, L. Vincent, and P. Burlat, "A framework for semantic enterprise integration," in *Interoperability of Enterprise Software and Applications*, N. Ivezic, A. Jones, D. Konstantas, J. P. Bourrières, M. Léonard, and N. Boudjlida, Eds. London: Springer, 2006, pp. 75–86.
- [8] T. Reimer, "Semantic Transformation Utilizing Graphs," *Proc. 2022 2nd International Conference on Advanced Enterprise Information System (AEIS 2022)*, Dec. 2022, pp. 138–145, doi:10.1109/AEIS59450.2022.00026.
- [9] W. P. Bejeck, Jr., *Kafka Streams in Action: Real-time Apps and Microservices with the Kafka Streams API*. Shelter Island, NY: Manning, 2018.
- [10] M. Richards and N. Ford, "Building Distributed Systems Using Sagas," *Online Presentation*, 2023.
- [11] J. Fischer and N. Wang, *Grokking Streaming Systems: Real-time Event Processing*. Shelter Island, NY: Manning, 2022.

Appendix D

T. Reimer, “Knowledge Management: The Influencer to Business Model Innovation,” Proc. 2024 4th International Conference on Advanced Enterprise Information System (AEIS 2024), accepted for publication.

Knowledge Management: The Influencer to Business Model Innovation

Tim Reimer
Department of Electrical and Software Engineering
University of Calgary
Calgary, Canada
Email: timothy.reimer@ucalgary.ca

Abstract—Given today’s global business environment, companies are wrestling with how best to confront and adapt to ever-changing conditions as part of their overall business strategy. The success of business model innovation as part of defining and developing new market opportunities requires the foundation of a sound knowledge management strategy. The focus of such a strategy needs to be based on value creation and value contribution within the firm. Value creation can only be achieved by identifying strong knowledge management capabilities to act as underlying building blocks. These capabilities are supported further by knowledge enablers. A well-choreographed knowledge management strategy will consider all three elements. The use of a knowledge management framework called the knowledge house will help provide context for the discussion of value contribution, capabilities, and enablers. By applying an overall strategic road map, a path to value is presented that contributes to unlocking new markets and product offerings and therefore helps the firm adapt to constantly changing market and economic conditions.

Keywords—*knowledge management, knowledge frameworks, business innovation, knowledge value, knowledge roadmaps*

I. INTRODUCTION

A. Business Model Innovation Context

The global economic environment has never been as challenging and uncertain as it is today; but it has also never been so rich in opportunities to succeed.

One of the drivers for these opportunities are the new technology platforms that are disconnected from space and time. In many cases, realization of new ideas does not require significant investment power and can be achieved simply by establishing an account to a cloud infrastructure and utilizing the vast technologies available. The realization of such ideas can instigate the disruption of entire industries, as has been witnessed many times in the past.

What is an opportunity for one is a threat to another. Being blindsided by these new disruptive initiatives is one of the greatest threats for established business environments. The development of corporate blind spots is a natural occurrence which is frequently stimulated through steady financial performance, lack of innovative thinking and culture, and a corporate attitude of overconfidence.

To counter the development of corporate blind spots, it is necessary to develop a set of warning signals and techniques that provide timely alerts indicating when current business models need to be re-evaluated. Every business will experience the life cycle of launch phase, growth, maturity, and decline [1]. Throughout this life cycle, evaluation indicators can be deployed to assess the business’s progression. The ability to determine this will highlight the need for change in order to ensure sustainability of the corporation.

One of these techniques is the use of the business model canvas [2] that helps to not only validate current

business models but also search for new opportunities. The business model canvas provides a structured technique to analyze the current business environment and develop new opportunities which mostly result in the adoption of innovations. A business model can be structured in the value proposition which contains the determination of the target segments, the product or service offering, and the expected revenue model. The focus of the operating model is on value creation, associated costs, and organization structure. The transition from a pure business model to business model innovation happens when two more of these components are reinvented and redesigned [3]. The outcome of business model innovation is a new strategic direction. Strategic change requires organizational buy-in or adoption, motivation, and capability to change.

The challenge lies not in formulating a strategy but in discovering the right innovation mix of value proposition and operating model that will ensure the desired market impact. Recognizing opportunities for new product or service offerings will require the adjustment of the supporting operating models in order to be able to transition to a new business environment with the objective of generating increased value. This innovation interaction between value proposition and operating model can only be achieved by implementing practices of continuous improvement into the DNA of the corporation. Many executives are under the illusion that an annual strategic planning session will generate sufficient innovation to formulate the strategic plan for the coming business period. The discovery of innovative possibilities and opportunities cannot be produced, only nurtured and facilitated. For innovation to blossom it needs fertile ground, which is created through a culture of continuous improvement.

By adopting a culture of continuous improvement, the threat of inhibiting forces that can counter business model innovation can be neutralized. Recognizing the existence of these forces is critical in understanding how to counteract them. One of these such negative barriers is the limited scope of thinking about solutions. When one is preconditioned to look only in a certain direction, most often induced by long-time proven practices, it can be challenging to break out of this mode to view things from a different perspective. This can also be called limited situational awareness. One way to counter this phenomenon is to forcefully transition to a different line of sight on the issue. Another barrier can be system-related—evidenced when processes are overburdened by bureaucracy. Innovation might be possible in simplifying and streamlining these overburdened processes, which will

generate new efficiencies [4]. Organizational culture of course contributes significantly to either the successes or failures of innovation. Nurturing a culture of innovation by recognizing not only big discoveries but also small contributions will motivate everyone in the organization to look for areas of improvement. Finally, searching in the wrong places for new solutions by not adopting a holistic perspective will not produce the innovation results that could drive a new innovative business model.

For business model innovation to see success it needs to be based on a foundation of organizational capabilities, one of which is the management of knowledge within the corporation. Knowledge is derived from information and drives ability in decision making. Capturing knowledge from all relevant sources, assembling and consolidating it, and making it available at the right time in the right format is the task and challenge of knowledge management. It is therefore the most important ingredient to drive successful business model innovation.

B. Knowledge Management Context

Developing a definition for knowledge has been an age-old undertaking; and a detailed discussion on this topic would be beyond the scope of this paper. In our context we will focus on the IT perspective and a more general understanding. Within the context of IT, knowledge is the result of data and information [5]. Data can be considered the raw material that is being collected, stored, and assimilated. Attaching context to this data results in information. Data could be temperature measurements taken over time. These measurements alone do not provide much meaning unless they are wrapped in some context. By adding that these measurements reflect the daily weather temperature at a given location, these measurements become meaningful and interpretive.

Subsequently, knowledge is derived by interpreting trends and relationships and making assumptions from the information provided. In other words, knowledge can be interpreted as information in context. Bellinger [5] adds a further category of understanding. One can acquire and learn knowledge of a certain domain; however, understanding will ensure that one can apply this knowledge to various situations or circumstances.

A common knowledge perspective is the interchange between explicit and tacit knowledge. Explicit knowledge is formal and systematic in nature and is therefore easily shared and communicated [6]. Tacit knowledge, on the other hand, is personally acquired and applied through learning and experience. The interaction between explicit knowledge and tacit

knowledge presents a spiral form of knowledge creation [7]. An example of the relationship between n explicit and tacit knowledge is in building a birdhouse: Tacit knowledge is being applied in building a birdhouse from scratch. This experience is being stored and can be applied in developing different designs. However, documenting the process and providing informative drawings results in explicit knowledge. Once the tacit knowledge is transformed into explicit knowledge it can be shared and can create new tacit knowledge by others attempting to build the birdhouse using the provided documentation. Understanding the interaction between explicit and tacit knowledge is important in understanding the knowledge creation process in an organization.

The perspectives described above are only two in a vast number of views that have been developed. Another way to describe knowledge management concepts is through the use of a framework. The design of a framework helps to provide better insight into a complex domain that comprises many different perspectives. Frameworks also present reference architectures that can be used to derive custom versions to address specific architectural needs.

Many attempts have been made to develop an overall knowledge management framework. A good analysis was done by Shongwe [8] with the aim of providing a unified view. Many knowledge management frameworks are very process oriented. However, given the complexity of managing knowledge, this seems too simplistic of an approach. Knowledge management is composed of different views providing insight into a certain perspective. The presentation of these views can be achieved by utilizing notations from business process design, data modelling, and organizational structure design.

Architecture in general uses a multi-view perspective to describe the features and functionality of a model. In building architecture, we usually present a side, top, and front view of a building. The same goes for software architecture where multiple perspectives contribute to an overall understanding of the end solution. By applying the same angles to knowledge management, it is possible to gain insight into various perspectives while still maintaining an overall harmonized model of how the different knowledge management views interact. Adopting a very fitting approach is the ARIS house [9]. The model was developed to describe software application solutions, particularly enterprise resource planning systems. Using this idea and projecting it onto knowledge management provides five distinct views of how knowledge management fits into an organization. Fig. 1 depicts the proposed knowledge management house.

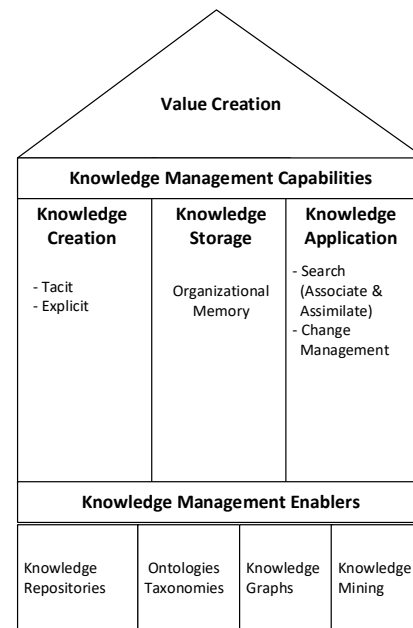


Fig 1 The Knowledge House

The roof of the house represents the knowledge value creation and contribution of all knowledge activities. It aligns all underlying initiatives to the common goal of value contribution, which is the ultimate objective of knowledge management.

The next layer represents the organizational knowledge capabilities. For demonstration purposes, the focus is on three fundamental capabilities: knowledge creation, storage, and application, recognizing that various different scenarios may demand more granularity.

The knowledge creation capability describes the interaction between explicit and tacit knowledge, through which new knowledge is created and must be stored. Knowledge creation is present in all corporate processes; it is not a question of if, it is a question of when and how to capture it. A practical example is the process of capturing lessons learned. In many organizations, this process is a necessary add-on to a finished project. However, if the objective is to derive knowledge from these lessons learned sessions, then capturing and storing their results is a critical part of enhancing organizational capabilities and delivering value.

Knowledge storage is the facility to store and retrieve knowledge that has been created within or applied to an organization. The description of knowledge storage is abstract since knowledge storage can take on a number of different forms and mediums.

The far right represents the functional applications of knowledge within an organization. One example could be the development of a corporate ontology to

better describe interrelationships between classes, and to develop reasoning for these relationships. Another example could be the implementation of domain-specific taxonomies and the relationships with a related ontology network.

The base of the house represents the knowledge enablers. These are the physical and tangible components that aid in developing a knowledge management infrastructure.

II KNOWLEDGE VALUE CREATION

In observing the evolution of knowledge management, three distinct generations have been identified [10]. The focus of the first generation was on knowledge sharing. The introduction of the Internet as a communication tool provided the capability of sharing information on a never-before-seen global scale [11]. But the sharing of knowledge is not only driven by the availability of knowledge (the supply side); it considers also the application of knowledge (the demand side). Through the emergence of required enabling technologies that were capable of addressing both the supply and demand sides and ensuring a reliable transfer between them, knowledge exchange or sharing was made possible.

The second generation was predominantly characterized by the creation of knowledge. By differentiating between explicit and tacit knowledge and observing the continuous interaction between them, new knowledge is created [12]. Knowledge sharing and creation are tightly linked. The underlying available knowledge technologies, together with the adoption of a mentality of continuous learning and improvement, provided a new focus on how to capture and acquire new knowledge. This knowledge in return needed to be ingested into the corporate knowledge repository.

The current third generation of knowledge management focuses on value creation [10]. This follows the general trend of stronger emphasis being given to measure and incorporate value creation into all aspects of process design. This generation is also the result of a changing understanding of the importance of classifying data as a corporate asset. What extends from this concept of data being an asset is corporate knowledge becoming an asset. Davenport and Prusak [13] anchor the creation of knowledge value in the existence of knowledge repositories, improved access capabilities to knowledge, enhancing cultural support for knowledge use, and considering knowledge as an asset. This view establishes a link

between knowledge management value creation and the necessary capabilities required to realize this value.

The emphasis on the importance of value creation has contributed to knowledge management initiatives being considered strategic elements in maintaining and even extending the competitive advantage. Expanding the definition of capital to include intellectual and social capital from human and financial resources demonstrates the importance that is being placed on general know-how, patents, process innovation, social collaboration, and communication in applying knowledge management. This therefore underlines the contribution that knowledge management makes to strategic planning and the development of new innovative business models. By utilizing and applying this corporate knowledge in discovering these new business models, corporate knowledge and the management of this knowledge demonstrates its value contribution and ensures it's a major contributor in corporate value creation.

Besides knowledge management being a value contributor to business model innovation, it also contributes to business process design and monitoring. Porter's core value chain [14] is a useful tool in observing the creation of value by knowledge management. In executing the core value chain, the product and service offerings should be attractive enough for the customer to utilize this offered value. Therefore, knowledge on customer product and service expectations as well as demand will flow backwards through the chain. Each stage contributes a portion of its value to the overall value creation process. Therefore, each value creation stage will require the knowledge necessary for continuous process improvement.

The path to value (P2V) describes resource utilization by applying capabilities to achieve value [15]. Borrowing this term from manufacturing, and given that corporate knowledge is a resource, it lends itself well to be included in measuring value contribution. This contribution can be tracked in all core as well as supporting business processes. Each knowledge management initiative should therefore be evaluated, monitored, and tracked in this key P2V context. If the P2V analysis is missed, it can quickly result in knowledge capabilities and technologies being applied with little or no value contribution. In this context it is important to stress that all evaluation be balanced against a holistic corporate view. Otherwise, a form of siloed knowledge management will emerge that inhibits maximized value creation.

Value creation, in whatever form it takes, requires a system of measurements by which this value can be measured. The measurement results can be in either

tangible or intangible form and should consist of multiple measurements in order to maintain objectivity. Since knowledge itself is an intangible resource, it provides significant challenges in attaching the right metric or metrics to derive a value measurement that is objective and demonstrates value contribution. There is no way to count, calculate, or add certain quantifiers in order to derive a percent contribution. Therefore, the value of knowledge management must be viewed contextually to all the other corporate activities.

A good approach is to determine knowledge management contribution dimensions. Dimensions provide a classification mechanism that can be assessed using indexes or weights. Examples of such dimensions are learning capabilities, intellectual property assessments, and decision support capabilities just to name a few. By using a dimensional approach, a measuring framework like the balanced scorecards [16] can be applied to determine value contribution. Smith and McKeen [17] propose a similar approach by differencing dimension or classifiers based on the tangible or intangible natures. Lee, Lee, and Kang [18] use an individual matrix to calculate an overall knowledge management performance index (KMPI).

In summary, there is no single approach that could satisfy all different knowledge management value contribution scenarios. The most critical consideration is that the selected dimensions represent a good cross section of an evaluation process and minimize biases as much as possible. In addition, it should be stressed that to achieve a functioning measurement system, continued monitoring is required which should result in adjusting, adding, or removing dimensions of the measurement framework. This in itself will apply knowledge to measure the path to value of the knowledge initiatives of a firm.

III. KNOWLEDGE MANAGEMENT CAPABILITIES

The previous discussion focused on the importance of value creation and measurement when knowledge management is being applied in a firm. A capability describes the ability to generate a certain outcome. This desired outcome is the contributor to the overall value generation process. Therefore, if knowledge management is to create value, it needs a number of core capabilities to achieve the value objective. Conceptually, the focus here will be on three foundational capabilities. The first is the capability of knowledge creation, the second the capability of

knowledge storage, and the third the capability of knowledge application.

First is the search capability. The search for information is critical in utilizing organizational knowledge since this capability should be able to retrieve information quickly across functional and departmental boundaries. The ability to ask questions in simple language and retrieve information with prioritized relevance is fundamental in applying and utilizing corporate knowledge for decision making, better process execution, and problem resolution. Efficient operation in this area will also ensure driving innovation for new value offerings to customers. Combining search capabilities with external resources and websites will enhance the value contribution of this capability. The objective is that information search should become the first go-to point in retrieving any information for knowledge creation. This capability should match the functionality and capability of current web search platforms, but with the intent of searching corporate content.

Following the search capability is the ability to associate or assimilate information in such a way that new knowledge can be gained, or existing knowledge can be applied to the decision and design processes of an organization. Today this is accomplished by using dashboards that allow for slicing or combining of information. Dashboards are still very rigid in supporting knowledge generation, so the emergence of machine learning and artificial intelligence engines that support these association and assimilation activities will become important in the application of knowledge management.

Finally, it is crucial for a firm to be able to rapidly adjust and adapt to economic conditions, competition, market demands, and technological advancements. Recognizing that changes are required, implementing the change, and managing and ensuring adoption of the change is a critical capability of knowledge management application. The knowledge and experience gained in developing processes to support organizational agility will ensure corporate survival. This will require open channels of communication and a willingness to adopt change. It is therefore important that companies elevate the status of their change management activities to ensure that corporate agility is maintained and improved.

IV. KNOWLEDGE MANAGEMENT ENABLERS

Knowledge management enablers are the power source for knowledge management capabilities. Enablers allow for the implementation of technologies so that a firm can utilize its knowledge management

capabilities to produce value. Whenever the discussion centres around the use of technology to support some business functionality, it is necessary to develop an architectural vision for this technology. Technology architectures provide a framework to describe how these components will be used, how they interact, and how they will support business functions. Technology architectures should consider three fundamental patterns in their design. The first is a central form of architecture in which all governance practices would be centralized. The second form is a federated approach in which there would be an overall governing body but with governance shared between enterprises and lines of business. The third is fully decentralized architecture in which governance would be distributed and only minimal alignment of governance maintained across business units.

In establishing the right knowledge management enablers, it is therefore critical to understand which architecture pattern should be applied as it will directly influence design and technology decisions for knowledge management. There are four fundamental enablers for knowledge management: the knowledge repositories; the ontologies and taxonomies with necessary semantic transformations; the use of knowledge graphs; and finally, the enablers for knowledge mining.

The knowledge repositories include document management systems, databases, and any other storage mediums that contain information and knowledge assets vital to the corporation. In order to achieve high efficiency in retrieving these assets, it is essential that a consistent storage approach be designed and adhered to. A bad design or practice is when artifacts for a given project are stored in several different repositories, with no clear communication or understanding as to why. It is also poor practice to not identify the latest version of an artifact, which makes retrieving the right version difficult and error prone. In these cases, knowledge assets are stored, but the retrieval is challenging and will deliver inconsistent results and therefore not contribute organizational value.

The development of corporate ontologies and taxonomies is essential for developing an effective knowledge management platform. These are often difficult to develop since the process involves the development of a corporate business glossary. In managing master and reference data it is essential to deliver a body of knowledge of common terms and usage that can be used across business units as a common denominator of understanding. In this context it is also important to consider data in motion within the corporation. As data moves between

applications it is critical to maintain a well-defined semantic transformation approach in order to sustain meaning across the application landscape. This must be supported with the ability to trace data lineage from source to target.

The design and implementation of knowledge graphs is the extension of ontologies and taxonomies. The linkage of information entities within a graph or network structure provides the capabilities necessary for effective knowledge retrieval. Graphs provide the agility to adjust to different business scenarios, quickly capture new knowledge, and incorporate this knowledge into the overall knowledge network. Traversing graphs allows for searching and associating knowledge nodes so that new knowledge can be derived. In addition, graphs are an ideal input to be processed with natural language processing or other artificial algorithms such as graph neural networks. Therefore, the knowledge graphs emerge not only to support search and traversal engines but also to provide the bases for extending knowledge creation and retrieval to artificial agents. The implementation and use of these new knowledge architectures will be a significant contributor in maintaining and expanding the competitive advantage. This will result in a direct impact on business model innovation by assisting in driving new value delivery concepts.

Finally, the term knowledge mining describes new forms of cognitive search and inference capabilities. Derived from current data mining practices, the new knowledge mining agents can search and retrieve knowledge by using rule sets to make inferences and recommendations [19]. The data sources are of heterogenous nature and the mining processes are able to index and classify these new data sources based on rules provided. A search is expressed in everyday language and the result sets are compiled through the use of intelligent agents. Tools like these will provide the next breakthrough for AI and will make knowledge management an integral part of the corporate value stream.

V. ADOPTING A KNOWLEDGE MANAGEMENT STRATEGY

Section 4 provided a discussion on the value creation and monitoring of knowledge management, the capabilities that are required for success, and the enablers that will support these capabilities; all of which are presented in the house of knowledge as an overall framework. The focus now shifts to developing a pragmatic approach in implementing these concepts with the objective of enhancing the knowledge management maturity of an organization.

Literature on the subject presents no shortage of potential strategic approaches. The fundamental question that needs to be answered is whether the strategy should be a top-down or bottom-up approach. In the top-down approach, corporate-wide strategy will be developed to move knowledge management maturity to the next desired level. The advantage here is that siloed initiatives are minimized and higher coherence across initiatives is achieved. The downside is that it requires significantly more engagement from stakeholders, and value generation can be delayed because of the longer implementation times.

The bottom-up approach is more pragmatic because it focuses on “low-hanging fruits,” and therefore the value generation cycle is significantly shorter. On the flip side, though, it could result in siloed and disjointed initiatives which could then lead to extra costs incurred later to harmonize processes and technologies.

Driving value with knowledge management will require that the related strategic plan be closely tied to the business plan and objectives, since the business plan is the driver for corporate value generation. Keeping these considerations in mind, the next step should be the prioritization of knowledge management initiatives. An excellent tool is the value versus effort matrix [20]. The matrix is a four-quadrant structure where the y coordinate is the value and the x coordinate is the effort. The four quadrants describe a space of prioritization. The first quadrant represents low value / low effort; the second quadrant represents high value / low effort; the third quadrant represents high value / high effort; and the fourth quadrant represents low value / high effort. Initiatives can be estimated based on potential value creation and effort and positioned in one of the four quadrants. Obviously, initiatives with high value and low effort are the “low-hanging fruits” that should be pursued first.

Finally, the result of the previous analysis should be presented in a road map that shows the expectation of these initiatives with their value contribution. The road map should have three distinct categories representing the value creation expectation, the capabilities that will be addressed with the initiatives, and the enablers that are required for the capabilities.

Fig. 2 provides an example road map. In the value section are the core processes for which knowledge management should contribute value. The diamonds reflect the starting of such value creation. One could size the diamond to the amount of value contribution that can be expected. The link between capability and value ensures that any capability is tied to a path to

value. As discussed, enablers drive capabilities. This road map ties all three knowledge management areas together into one overall presentation and is built on the concept of the knowledge house that was initially introduced.

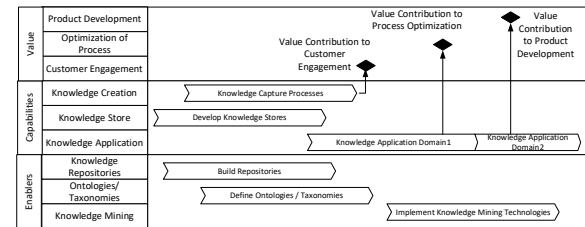


Fig 2. Knowledge Management Roadmap

VI. CONCLUSION

In conclusion, we proposed a knowledge management framework that is concerned with different perspectives on organizational knowledge management. These views focus on value generation, capabilities that contribute to value generation, and the enablers that ensure capabilities can be applied. The concept of such a framework then allows for formulation of a knowledge management strategy that is aligned with the overall business objectives; and by using the framework, a road map for implementation can be developed. This ensures that knowledge management initiatives are value focused and continuously contribute to new business model innovations.

REFERENCES

- [1] Zoltners, A. A., Sinha, P., & Lorimer, S. E. (2006). Match your sales force structure to your business life cycle. *Harvard Business Review*, 84(7/8), 80.
- [2] Joyce, A., & Paquin, R. L. (2016, November). The triple layered business model canvas: A tool to design more sustainable business models. *Journal of Cleaner Production*, 135, 1474–1486.
- [3] Lindgardt, Z., Reeves, M., Stalk, G., Jr., & Deimler, M. (2012). Business model innovation: When the game gets tough, change the game. In M. Deimler, R. Lesser, D. Rhodes, & J. Sinha (Eds.), *Own the future: 50 ways to win from the Boston Consulting Group* (pp. 291–298). The Boston Consulting Group.
- [4] Friedrich von den Eichen, S., Freiling, J., & Matzler, K. (2015). Why business model innovations fail. *Journal of Business Strategy*, 36(6), 29–38. <https://doi.org/10.1108/JBS-09-2014-0107>
- [5] Bellinger, G., Castro, D., & Mills, A. (2004). Data, information, knowledge, and wisdom. <https://www.systems-thinking.org/dikw/dikw.htm>
- [6] Nonaka, I. (2008) *The knowledge-creating company*. Boston: Harvard Business Review Press.
- [7] Nonaka, I. (1994). A dynamic theory of organizational knowledge creation. *Organization Science*, 5(1), 14–37.
- [8] Shongwe, M. M. (2016). An analysis of knowledge management lifecycle frameworks: Towards a unified framework. *Electronic Journal of Knowledge Management*, 14(3), 140–153.

- [9] Scheer, A. W. (1999). ARIS – Business process frameworks. Heidelberg, Germany: Springer-Verlag.
- [10] Vorakulpipat, C., & Rezgui, Y. (2008). Value creation: The future of knowledge management. *The Knowledge Engineering Review*, 23(3), 283–294. <https://doi.org/10.1017/S0269888908001380>
- [11] Koenig, M. E. D. (2002). The third stage of KM emerges. *KMWorld*, 11(3), 20–21.
- [12] Nonaka, I., Toyama, R., & Konno, N. (2000). SECI, Ba and leadership: A unified model of dynamic knowledge creation. *Long Range Planning*, 33(1), 5–34.
- [13] Davenport, T. H., & Prusak, L. (1998). Working knowledge: How organizations manage what they know. Boston: Harvard Business School Press.
- [14] Porter, M. E. (2008). Competitive advantage: Creating and sustaining superior performance. Free Press.
- [15] Mahlburg, C. (2004, December 21). Value creation -- The path to value. *Industry Week*. <https://www.industryweek.com/supply-chain/planning-forecasting/article/21937303/value-creation-the-path-to-value#:~:text=The%20path%20to%20value%20is,two%2C%2C%20three%2C%20or%20more>.
- [16] Kaplan, R. S., & Norton, D. P. (1992). The balanced scorecard—Measures that drive performance. *Harvard Business Review*, 70(1), 71–80.
- [17] Smith, H. A., & McKeen, J. D. (2003). Valuing the knowledge management function. In C. W. Holsapple (Ed.), *Handbook on knowledge management 2: Knowledge directions* (pp. 353–368). Springer.
- [18] Lee, K. C., Lee, S., & Kang, W. (2005). KMPI: Measuring knowledge management performance. *Information & Management*, 42(3), 469–482.
- [19] Chemchem, A., & Drias, H. (2015). From data mining to knowledge mining: Application to intelligent agents. *Expert Systems with Applications*, 42(3), 1436–1445.
- [20] Kukhnavets, P. (2018, April 24). Value/effort matrix: Lean prioritization for product managers. *Hygger*. <https://hygger.io/blog/lean-prioritization-approach-ongoing-pm-issues/>